BIG DATA BASED IMDB MOVIE REVIEW ENGINE



Submitted By

Aimen Atif **20-CP-26**
Shaher Bano Sani **20-CP-50**

Project Supervisor

Dr. Farhan Qamar

DEPARTMENT OF COMPUTER ENGINEERING
UNIVERSITY OF ENGINEERING AND TECHNOLOGY TAXILA

June 2024

BIG DATA BASED IMDB MOVIE REVIEW ENGINE



Authors/Submitted By
Aimen Atif (20-CP-26)
Shaher Bano Sani (20-CP-50)

Project Supervisor

Dr. Farhan Qamar
Name of Supervisor

DEPARTMENT OF COMPUTER ENGINEERING
UNIVERSITY OF ENGINEERING AND TECHNOLOGY
TAXILA

June 2024

ABSTRACT

The demand for effective review engines and movie recommendation systems has increased tremendously due to the exponential expansion of user-generated content and movie content. This project presents a comprehensive approach to developing a Movie Recommendation System and Review Engine utilizing sentiment analysis powered by machine learning and Big Data techniques. By using sentiment analysis, the system can automatically extract sentiments from textual reviews and classify them into positive, negative, or neutral categories. This module utilizes state-of-the-art natural language processing techniques to capture the underlying emotions and opinions expressed by users. To enhance the recommendation capabilities, the system employs a content-based recommender architecture along with cosine similarity to analyze movie features and user preferences, resulting in aligning similar movies in terms of movie cast, and themes. Moreover, with the explosion of user-generated content on social media platforms such as Twitter, Netflix, and YouTube, managing and processing large volumes of movie reviews becomes a challenging task and it is difficult to scan entire data manually. To address this problem, the use of Big Data technologies enables efficient storage, processing, and analysis of vast amounts of data, ensuring scalability and real-time performance. The evaluation of the system involves experimentation on a large movie dataset, comparing the performance of the system against existing approaches. The results demonstrate the efficacy of the proposed system in generating accurate and personalized recommendations while providing valuable insights into user sentiments, thus giving a platform to express their opinions, and facilitating an enhanced movie-watching experience.

UNDERTAKING

We certify that project work titled "*Big Data based IMDb Movie Review Engine*" is our own work. The work has not, in whole or in part, been presented elsewhere for assessment. Where material has been used from other sources it has been properly acknowledged/ referred.

Aimen Atif (20-CP-26)

Shaher Bano Sani (20-CP-50)

ACKNOWLEDGEMENTS

To everyone who helped us to successfully complete our final year project, we would like to extend our profound gratitude and appreciation. We would first like to express our gratitude to **DR. FARHAN QAMAR**, our project supervisor, for all of his help, support, and insightful advice during the project. Our research's direction and breadth were greatly influenced by their knowledge and support. Their help and brainstorming sessions have been invaluable to our project, and we are very appreciative to them. Finally, we thank the institution and its resources for offering a setting that is favorable to research and learning. We are thankful for the chance to have worked on this final year project, which has been a tremendous learning experience.

TABLE OF CONTENTS

Abstract ..... **i**
Undertaking ..... **ii**
Acknowledgements ..... **iii**
Table of Contents ..... **iv**
List of figures ..... **vii**
List of tables ..... **ix**
Abbreviations ..... **x**
Chapter 1: Introduction **1**
1.1 Introduction 1
1.2 Problem Statement 1
1.3 Background & Scope 2
1.4 Aims & Objectives 2
1.4.1 Aim 2
1.4.2 Objectives 2
1.5 Deliverables 3
1.6 Tools used 4
1.6.1 Libraries 4
1.7 Organization of Report 4
Chapter 2: Literature Review **5**
2.1 BIG DATA ANALYTICS 5
2.1.1 Volume 5
2.1.2 Variety 5
2.1.3 Velocity 5
2.1.4 Veracity 6
2.2 HADOOP ECOSYSTEM 6
2.2.1 Apache Hive 7
2.2.2 Apache Pig 8
2.3 RECOMMENDATION SYSTEM 9
2.3.1 Collaborative Filtering 10
2.3.2 Content-Based Filtering 10
2.3.3 Hybrid Approaches 12
2.4 ALGORITHMS USED 12
2.4.1 Sentiment Analysis 12
2.4.2 Convolutional Neural Networks (CNN) 14
2.5 RELATED WORK 15
Chapter 3: Methodology **18**
3.1 DATA COLLECTION 18
3.2 DATA ANALYSIS USING APACHE PIG: 19
3.2.1 Movie Recommendation System: 19
3.2.1.1 Data Preparation: 19
3.2.1.2 Filtering of movies with respect to genres: 19
3.2.1.3 Ordering and Limiting: 19
3.2.1.4 Storing the Results: 19
3.2.2 Movie Review Engine: 20
3.2.2.1 Data Refinement: 20
3.2.2.2 Tokenization: 20
3.2.2.3 Sentiment Dictionary: 20
3.2.2.4 Joining Tokens and Dictionary: 21
3.2.2.5 Calculating Average Ratings: 21
3.2.2.6 Filtering Positive and Negative Reviews: 21

3.2.2.7 Results Storage: 21
3.3 LIBRARIES USED 21
3.3.1 Pandas 21
3.3.2 Scikit-learn 21
3.3.3 Pickle 22
3.3.4 Matplotlib 22
3.3.5 Nltk 22
3.3.6 OpenCV 2 22
3.3.7 Face Recognition 23
3.4 MOVIE RECOMMENDATION SYSTEM 23
3.4.1 Data Inspection 23
3.4.2 Selection of Attributes 23
3.4.3 Text Vectorization 24
3.4.4 Applying Cosine Similarity 24
3.4.5 Recommendation 25
3.4.6 Data Serialization 25
3.5 USER REVIEW PREDICTION 25
3.5.1 Design Explanation 25
3.5.2 Data Preprocessing 26
3.5.3 Text Cleaning 26
3.5.4 Tokenization 26
3.5.5 Stop Word Removal 27
3.5.6 Word Cloud Visualization 27
3.5.7 Text Vectorization 27
3.5.8 Logistic Regression Analysis 27
3.5.8.1 Training and Testing 28
3.5.8.2 Confusion Matrix 28
3.5.9 Serialization of Data 29
3.5.10 Sentiment Prediction 29
3.6 ACTOR-BASED MOVIE RECOMMENDATION SYSTEM 30
3.6.1 Design Explanation 30
3.6.2 Dataset 30
3.6.3 Extracting Facial Features 30
3.6.4 Target Actor Face Detection 30
3.6.5 Creating Bounding boxes for identification 31
3.6.6 Matching Results and Recommendations 31
3.7 FRONT-END DESIGN 31
3.7.1 Home Page 31
3.7.2 About Us 32
3.7.3 Trending Movies 32
3.7.4 Contact Us 32
3.7.5 Review Prediction 33
3.7.6 Parental Guide 33
Chapter 4: Hardware Implementations/ Software Simulations 34
4.1 ANALYSIS OF MOVIE REVIEW ENGINE ON APACHE PIG 34
4.1.1 REGISTER Statement: 34
4.1.2 LOAD Statement for Review Data: 34
4.1.3 FOREACH Statement for getting details: 34
4.1.4 FOREACH Statement for tokenization: 35
4.1.5 LOAD Statement for Sentiment Dictionary: 36
4.1.6 JOIN Statement for Joining Tokens and Dictionary: 36
4.1.7 FOREACH Statement for Generating Ratings: 37
4.1.8 GROUP Statement for Grouping Ratings: 38
4.1.9 FOREACH Statement for Calculating Average Ratings: 39
4.1.10 FILTER Statements for Positive and Negative Reviews: 39
4.1.11 STORE Statement for Storing the Results: 41
4.2 ANALYSIS OF MOVIE RECOMMENDATION SYSTEM ON APACHE PIG 41
4.2.1 LOAD Statement for Movie Recommendation Data: 41
4.2.2 FOREACH Statement for Getting Details: 42

4.2.3 Extract Only Comedy movies: 42
4.2.4 Extract only Adventures movies: 45
4.2.5 Extract only crime movies: 45
4.2.6 Extract only Action movies: 46
4.2.7 Extract only dramatic movies: 46
4.2.8 Extract only Horror movies: 46
4.2.9 Extract only Thriller movies: 47
4.3 MOVIE RECOMMENDATION SYSTEM ON MACHINE LEARNING 47
4.3.1 Result Analysis: 50
4.4 REVIEW PREDICTION SYSTEM ON MACHINE LEARNING 50
4.4.1 User Input for Movie Review: 50
4.4.2 Positive Sentiment Prediction: 51
4.4.3 Negative Sentiment Prediction: 51
4.4.4 Adding Reviews to the Database: 52
4.4.5 Comparison of Models: 52
4.5 ACTOR-BASED RECOMMENDATION SYSTEM: 54
4.5.1 Movie Input: 54
4.5.2 Generating Face Encodings: 54
4.5.3 Object Detection and Bounding Boxes: 55
4.5.4 Face Identification: 55
4.5.5 Similarity Comparison and Resemblance Detection: 56
4.5.6 Result Evaluation: 56
4.6 USER FRIENDLY INTERFACE: 56
Conclusion & Future work 65
References 66
APPENDIX 69
Glossary 201

LIST OF FIGURES

Figure 2.1: 4 V's of Big Data……. 6
Figure 2.2: A general architecture of Hadoop Distributed File System.…… 7
Figure 2.3: Architecture of Apache Hive…… 8
Figure 2.4: Architecture of Apache Pig…… 8
Figure 2.5: Architecture of a recommendation system …… 9
Figure 2.6: Recommendation system based on Collaborative filtering …….... 10
Figure 2.7: Content-based Recommendation system …… 11
Figure 2.8: Cosine Similarity ……..... 11
Figure 2.9: Process of sentiment analysis ……..12
Figure 2.10: Process of tokenization ……... 13
Figure 2.11: Process of lemmatization ……... 13
Figure 2.12: A simple neural network ……... 14
Figure 2.13: Architecture of LeNet5 ……... 15
Figure 4.1: Loading Review Data ……... 34
Figure 4.2: Applying FOREACH statement ……... 34
Figure 4.3: Result of FOREACH statement ……... 35
Figure 4.4: Applying FOREACH statement for Tokenization……... 35
Figure 4.5: Result of FOREACH statement on Tokenization…….……... 36
Figure 4.6: Loading Sentiment Dictionary…..……... 36
Figure 4.7: Sentiment Ratings for words in dictionary……... 36
Figure 4.8: Applying JOIN statement……..……... 37
Figure 4.9: Result of JOIN statement……..……... 37
Figure 4.10: Applying FOREACH statement……... 38
Figure 4.11: Result of FOREACH statement……... 38
Figure 4.12: Applying GROUP statement……... 38
Figure 4.13: Result of GROUP statement…..……... 38
Figure 4.14: Calculating Average Ratings…..……... 39
Figure 4.15: Calculated Average Ratings…..……... 39
Figure 4.16: Applying Filter on Positive Reviews…..……... 39
Figure 4.17: Filtered Positive Reviews……..……... 40
Figure 4.18: Applying Filter on Negative Reviews……... 40
Figure 4.19: Filtered Negative Reviews……... 40
Figure 4.20: Applying STORE statement……..……... 41
Figure 4.21: Storing the reviews……... 41
Figure 4.22: Movie metadata……... 42
Figure 4.23: Extraction of Comedy Movies……...……... 42
Figure 4.24: FOREACH statement on Comedy Movies……... 43
Figure 4.25: Sorting of Movies in Descending Order……... 43
Figure 4.26: Top 10 movies using LIMIT function……... 44
Figure 4.27: Result of SORT function……... 44
Figure 4.28: Result of STORE function……..……... 45
Figure 4.29: Extraction of Adventure movies……... 45
Figure 4.30: Extraction of Crime movies……...……... 46
Figure 4.31: Extraction of Action movies……..……... 46
Figure 4.32: Extraction of Dramatic movies……..……... 46
Figure 4.33: Extraction of Horror movies……..……... 47
Figure 4.34: Extraction of Thriller movies……..……... 47
Figure 4.35: Calculating Feature Vector……... 48
Figure 4.36: Calculating Similarities……..……... 48
Figure 4.37: Prompt to enter Movie Title……...……... 48
Figure 4.38: Sorted Similar Movies……... 49
Figure 4.39: Calculating Similarity score based on searched movie……..…... 49
Figure 4.40: Movie Recommendations……...……... 50
Figure 4.41: Input User Review……..……... 51

Figure 4.42: Positive Review Prediction…….. 51
Figure 4.43: Negative Review Prediction……..…….. 52
Figure 4.44: Addition of Reviews in Database……..…….. 52
Figure 4.45: Performance Evaluation through Confusion Matrix…….…... 53
Figure 4.46: Movie input prompt……...…….. 54
Figure 4.47: Face Encoding…….. 55
Figure 4.48: Face Detection and Encoding…….. 56
Figure 4.49: Movie Recommendation…….. 56
Figure 4.50: Architecture of Front-end……..…….. 56
Figure 4.51: Front page of website…….. 57
Figure 4.52: Recommendation Server……...…….. 57
Figure 4.53: Movie Information…….. 58
Figure 4.54: Cast and Crew Information……...…….. 58
Figure 4.55: Actor's Information……..…….. 59
Figure 4.56: Reviews and sentiments Table……..…….. 59
Figure 4.57: Recommended Movies……..…….. 59
Figure 4.58: Log-In Page…….. 60
Figure 4.59: Register Page……..…….. 60
Figure 4.60: About Us Page…….. 61
Figure 4.61: Trending Movies…….. 61
Figure 4.62: Popular Movies……..…….. 62
Figure 4.63: Upcoming Movies…….. 62
Figure 4.64: Applauded TV Shows…….. 62
Figure 4.65: Contact Us Page…….. 63
Figure 4.66: Review Prediction Page …….. 63
Figure 4.67: Parental Guidance Page ……..……..64

LIST OF TABLES

Table 2.1: Recommendation System Processes……9

ABBREVIATIONS

CNN: Convolutional Neural Networks

CF: Collaborative Filtering

ETL: Extract Transform Load

IMDB: Internet Movie Database

ME: Maximum Entropy

MLP: Multi-layer perceptron

NLTK: Natural Language Tool Kit

RNN: Recurring Neural Networks

SQL: Structured Query Language

SVM: Support Vector Machines

TF-IDF: Term Frequency and Inverse Document Frequency

CHAPTER 1: INTRODUCTION

In the ever-expanding world of cinema, harnessing the power of big data propels the IMDb movie review engine into a new era of insightful analysis and unparalleled user experiences [4]. Efficient performance is a vital component of software engineering, especially when working with constrained hardware resources. It falls upon software engineers to discover methods that optimize the utilization of available hardware, enabling increased computational capabilities and task handling. The Movie Recommendation System and Review Engine is a cutting-edge solution that combines state-of-the-art sentiment analysis, Convolutional Neural Networks (CNN) and Big Data techniques to tackle the challenges encountered by movie viewers in their quest to find relevant movies based on user reviews [6]. This project makes a significant contribution to the field of movie recommendation systems and review engines, enhancing the overall movie discovery process and empowering viewers with accurate recommendations and a look into user reviews. By empowering viewers with comprehensive insights into user sentiments, it revolutionizes the way we navigate the vast landscape of cinema, making every movie-watching experience a truly immersive and personalized journey.

1.1 Introduction

In today's digital era, the movie industry has witnessed exponential growth in terms of content availability and user-generated reviews. With an overwhelming number of movies being released each year, it has become increasingly challenging for viewers to navigate through the vast landscape of cinema and make informed decisions about which movies to watch [5]. Movie recommendation systems and review engines play a crucial role in assisting viewers by providing personalized recommendations and insights into user sentiments. Indeed, the integration of sentiment analysis and deep learning techniques, such as CNN, significantly strengthens our movie recommendation system, ensuring user satisfaction. For achieving greater scalability in processing large datasets, big data techniques are employed in the movie review engine. These techniques are essential for efficiently handling the ever-growing volume of user reviews and enabling comprehensive analysis of textual data. The maturing big data technologies have provided researchers with valuable tools for extracting insights from large and diverse datasets [3]. When dealing with big data techniques, it is crucial to select appropriate technology tools capable of handling the characteristics of these datasets, including their variations and the need for data correctness. Apache Hive and Apache Pig are two prominent big data technology tools that excel in addressing the challenges associated with large and complex data.

1.2 Problem Statement

The exponential growth of movie content and user-generated reviews has led to information overload and decision paralysis for movie enthusiasts. Users face the challenge of finding personalized movie recommendations based on their preferences and sentiments. Additionally, accurately extracting meaningful insights from user reviews and classifying them into different types of sentiments is vital for informed decision-making. The existing movie recommendation systems often face limitations in accurately capturing user

preferences and sentiments due to the complexity and subjectivity of movie reviews. For this, users require effective systems that can recommend personalized movies based on their preferences and sentiments.
In the initial movie recommendation system built using machine learning techniques, such as sentiment analysis and Convolutional Neural Networks (CNN) we can encounter difficulties in efficiently handling large datasets. Although machine learning approaches excel at providing accurate recommendations based on user behavior and preferences, scaling them to cope with the ever-increasing volume of data has become a concern. To address these challenges, the integration of Big Data techniques is necessary. Therefore, there is a need to develop an advanced movie review engine that leverages Big Data techniques to effectively process and analyze vast amounts of movie-related data, enabling accurate recommendations and comprehensive insights into user sentiments.

1.3 Background & Scope

Machine learning techniques, such as sentiment analysis and CNN, have shown promising results in various natural language processing tasks. By integrating these techniques with Apache Hive and Pig, which are widely adopted Big Data tools, it becomes possible to process and analyze large volumes of movie data efficiently. The movie recommendation system and review engine leverage these technologies to provide personalized movie recommendations, extract sentiments from textual reviews, and classify them accurately. Additionally, the system can benefit from the scalability and parallel processing capabilities of Apache Hive and Pig, enabling efficient storage, retrieval, and analysis of movie data. The scope of this project encompasses the development of a scalable, efficient, and accurate review that can adapt to the dynamic nature of user preferences and sentiments.

1.4 Aims & Objectives

1.4.1 Aim

The aim of this project is to develop a Review Engine based on Big Data techniques that aims to leverage the capabilities of Apache Hive and Pig for efficient storage, retrieval, and processing of large movie datasets. This involves designing and implementing the necessary data pipelines and workflows to handle the volume and variety of movie data. The ultimate goal is to empower users with data-driven insights, enabling them to make informed decisions about movie choices based on the collective sentiments expressed in the reviews.

1.4.2 Objectives

The Objective of using machine learning techniques in our project is:

- To Develop a content-based recommendation system that will focus on the content of movies, such as genre, director, actors, and plot, to identify similarities and patterns.
- Implement sentiment analysis techniques to extract sentiments from textual reviews and classify them into positive, negative, or neutral categories.
- Develop a CNN model that uses face recognition techniques to analyze movie cast features and user preferences to generate accurate movie recommendations.

As for achieving more scalability, Big Data techniques are used that efficiently process and analyze large volumes of user-generated reviews, utilizing sentiment analysis to provide comprehensive insights and facilitate informed decision-making.

- To optimize data storage, indexing, and querying operations, enabling fast and scalable access to movie data.
- Utilize Apache Hive and Pig to streamline the processing of large movie datasets. By leveraging their data processing capabilities, the objective is to enable efficient and parallel execution of data transformations, aggregations, and analytics.
- To ensure the scalability and performance of the deployed system and that it can efficiently process and analyze large movie datasets without compromising performance. This involves benchmarking the system against existing approaches to identify areas for optimization and improvement.
- To optimize and improve the system's performance. This may involve fine-tuning data storage configurations, optimizing query execution plans, or leveraging advanced optimization techniques specific to Apache Hive and Pig.

1.5 Deliverables

The deliverables for the project are as under:

- **Movie Recommendation System:**
  A fully functional content-based Movie Recommendation System that incorporates sentiment analysis and CNN. Based on the metadata, such as genres, actors, directors, and plot summaries, the system will recommend movies to the user according to their taste.
- **Sentiment Analysis:**
  The system will utilize sentiment analysis on movie reviews to predict the sentiments so that the users can not only view the reviews of other people from around the world but also the sentiments associated with them.
- **Feedback and Reporting:**
  The system will also provide the feature of feedback from the users by the help of sentiment analysis, so that if any user wants to provide a review for a particular movie, he/she can do so, and the sentiment related to that review will also be predicted at run time.
- **Integration with IMDb Database:**
  The engine will be integrated with the IMDb database for seamless access to the details of movies from around the world, including cast and crew, plot summaries, ratings and reviews.
- **User Interface:**
  The system will provide a friendly user interface that will allow the users and audience to search for their favorite movies and can also access the personalized recommendations along with the movie reviews, ratings and all the particular details in a hassle-free manner.
- **Scalability and Performance Optimization:**

The engine will be designed in such a way that it will efficiently handle larger datasets of movies and reviews. The deliverable will basically be scalable architecture with performance optimization techniques.

1.6 Tools used

The project utilizes a combination of machine learning tools/frameworks alongside Apache Hive and Apache Pig from the Big Data ecosystem to optimize data processing and analysis.

- Apache Pig
- HDFS (Hadoop Distributed File System)
- Jupiter Notebook
- Visual Studio (VS)

1.6.1 Libraries

- Piggybank
- OpenCV
- Scikit-learn
- TensorFlow
- NLTK (Natural Language Toolkit)

1.7 Organization of Report

This report is organized into several chapters to provide a comprehensive understanding of Big Data based IMDb Movie Review Engine.

Chapter 1 introduces the project and provides an overview of the objectives, significance, and scope. It outlines the need for a content-based movie recommendation system and review engine, highlighting the importance of using Big Data techniques to enhance the movie discovery process.

Chapter 2 presents an in-depth review of related works and research in the field. It explores existing literature, methodologies, and technologies used in movie recommendation systems, sentiment analysis, CNN, and Big Data analytics.

Chapter 3 describes the methodology employed in the development of the system. It explains the implementation details of sentiment analysis, CNN, and Big Data technologies. The chapter also provides a detailed explanation of the tools and technologies used in the project, such as Hive and Pig.

Chapter 4 presents the hardware implementations or software simulations conducted as part of the project. It outlines the experimental setup, data collection, and preprocessing techniques utilized. This chapter also includes the evaluation of the system's performance, discussing metrics and results obtained during testing.

Finally, the last part of the report concludes with a summary of the findings, limitations, and future work. It summarizes the key findings and contributions of the project. It suggests potential extensions or enhancements for future iterations.

CHAPTER 2: LITERATURE REVIEW

2.1 BIG DATA ANALYTICS

The term Big Data refers to the massive amount of both structured and unstructured data produced by a variety of sources, including social media and internet platforms, electronic devices. It signifies a revolutionary shift in the manner we collect, store, and analyze data. It comprises of data that is too expensive and complicated to be evaluated properly with traditional data processing techniques. It is one of the major applications of parallel and distributed computing [3]. The concurrent processing of numerous tasks or processes to solve a computer problem can be defined as parallel computing. In contrast, distributed computing makes use of multiple computers, or nodes, collaborating to complete a task. Capturing, storing, analyzing, sharing, transferring, visualizing, querying, updating, information privacy, and data source are some of the concerns associated with big data analysis [31]. It describes the process of applying sophisticated data analytics methods to mine vast volumes of data for valuable insights, including predictive analytics and user behavior analytics. One of the major distinctive characteristics of big data includes its "4Vs" generally referred to as volume, variety, velocity, and veracity of data [5].

2.1.1 Volume

Volume in the dictionary of big data refers to the vast amount of data generally ranging in Zeta bytes or Petabytes or even Exabytes, being generated and collected. Big data sets are defined by their massive size, and include the streaming data from Social Media platforms, data from sensors and machines and the transaction-based data stored over the years [5]. This rapid growth in data generation has necessitated the development of storage and processing systems that can scale to accommodate and handle such vast volumes of data.

2.1.2 Variety

Variety, in big data, emphasizes the wide range of data sources and formats involved. It refers to the different forms in which the data exists. This encompasses structured data, that consist of a well-organized structure such as databases and spreadsheets, semi-structured that does not adhere to the tabular format of data models and lacks a fixed schema such as XML or JSON and unstructured data, including text documents, images, videos, and social media posts. Big data processing often requires the integration and analysis of data from diverse sources and formats, presenting challenges in terms of data integration and interoperability.

2.1.3 Velocity

Velocity pertains to the rate at which data is produced, processed, and analyzed. Big data is frequently generated in real-time or close to real-time, necessitating swift processing and analysis to derive meaningful insights and facilitate timely decision-making. Illustrative examples of high-velocity data include streaming data from social media feeds, sensor networks, and financial transactions. Managing high-velocity data necessitates efficient mechanisms for capturing and processing data promptly.

2.1.4 Veracity

Veracity in big data refers to the degree of inevitability and trustworthiness of the data. The inspection of data quality and reliability is necessary since big data frequently exhibits noise, incompleteness, and inconsistency [3]. In order to avoid incorrect conclusions and taking wrong actions, veracity highlights the importance of evaluating the data's quality, dependability, and trustworthiness. The significance of guaranteeing accurate data and reliable sources in upholding the validity of analysis and decision-making procedures is emphasized. Figure 2.1 shows the 4Vs of Big Data.

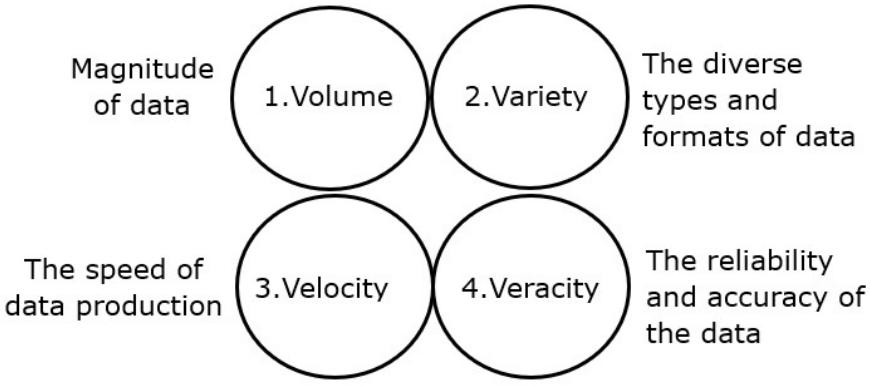


Figure 2.1 4 V's of Big Data

2.2 HADOOP ECOSYSTEM

Apache Hadoop, a free to use and open-source platform enables the users to process, analyze and store large amounts of data in a distributed way. The Hadoop framework itself is mostly written in Java, while shell scripts and some native C code are utilized for command line utilities. It makes use of commodity hardware integrated with each other. Enterprises can handle and store large amounts of data with Hadoop's fault tolerance, scalability, and cost. It is predicated on the fundamental idea of the Hadoop Distributed File System (HDFS) and the Map Reduce distributed processing methodology [3].

HDFS is a fault-tolerant file system for the processing of enormous files. Developed for the Hadoop architecture, it is a distributed, scalable, and portable file system with a Java foundation. It works by partitioning huge files into blocks and then distributing them over the cluster [5]. This approach makes the use of the data locality phenomenon, which occurs when nodes alter the data that they have access to. This allows for faster and more efficient handling of the information as compared to a more conventional supercomputer design that relies on a parallel file system and disperses computation and data over high-speed networking. A general architecture of HDFS is shown in Figure 2.2.

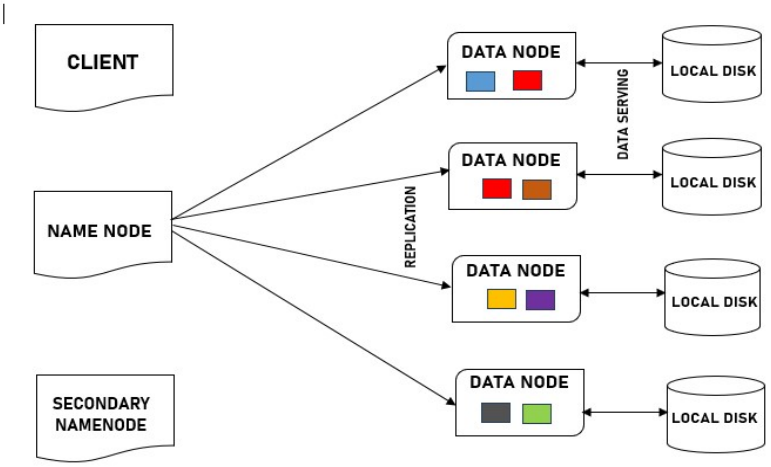


Figure 2.2 A general architecture of Hadoop Distributed File System

It is developed to evolve from a single server to thousands of devices, each of which may handle local storage and processing. The library itself is built to identify and manage failures at the application layer, providing a highly available service on top of a cluster of computers, each of which may be prone to failures, as opposed to depending on hardware to offer high availability.

MapReduce is a low-level programming technique, that uses a distributed, parallel algorithm on a cluster to process and generate large data sets. It directly negotiates with hardware to perform the computations. The evaluation in MapReduce is done by breaking a big problem into smaller chunks which is then carried out by multiple computers integrated with each other [3]. It consists of two parts, mappers, and reducers. Mappers are responsible for taking input data and processing it by generating intermediate key-value pairs in parallel across a distributed cluster. While reducers perform the aggregating or concatenating task to produce the final output.

2.2.1 Apache Hive

Apache hive is the data warehouse and high-level programming language of Hadoop. The interaction of Hadoop with Hive makes it easy to manage the big data. It is used to store, query, and analyze data. It transforms semi-structured data into the structured format by creating schemas and tables of the data [5]. It is basically used for batch processing over huge datasets of immutable data. With Hive's schema-on-read technique, which applies the schema at the time of querying, managing varied and dynamic data is made flexible. It offers choices for optimizing query performance and storage economy by supporting a wide range of data store formats, including text files, Avro, Parquet, and more. In order to process structured and semi-structured data efficiently, it provides data serialization and deserialization capabilities, supporting sophisticated data types like arrays, maps, and structs. Users may add new logic and operations to their Hive queries by extending Hive with user-defined functions (UDFs), aggregates (UDAs), and table functions (UDTFs). The architecture of Hive is shown in Figure 2.3.

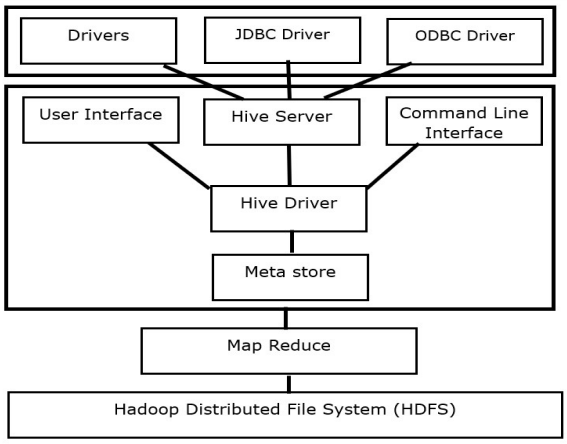


Figure 2.3 Architecture of Apache Hive

It uses its own language for the execution and processing of queries known as Hive-QL. It is more or less similar to the Structured Query Language (SQL) and is based on SQL-92 specifications. It helps to process billions of rows of dataset smoothly by making Hive run over Map Reduce.

2.2.2 Apache Pig

Apache Pig is a high-level data flow language and open-source processing model that is used for the analysis of larger datasets in an easy and faster way. Pig has the ability to run its Hadoop tasks in Apache Spark, Apache Tez, or MapReduce. It enables effortless data summarization, reporting, querying and analysis of large amounts of data. It provides abstraction over MapReduce and provides an engine for executing dataflows parallelly. Pig's optimizer looks at the data flow and reorganizes the process to boost performance. Utilizing the parallelism of the underlying Hadoop architecture, dynamically splitting the data, and refining the execution method are the keys to efficient and scalable data processing [3]. The architecture of Apache Pig is shown in Figure 2.4.

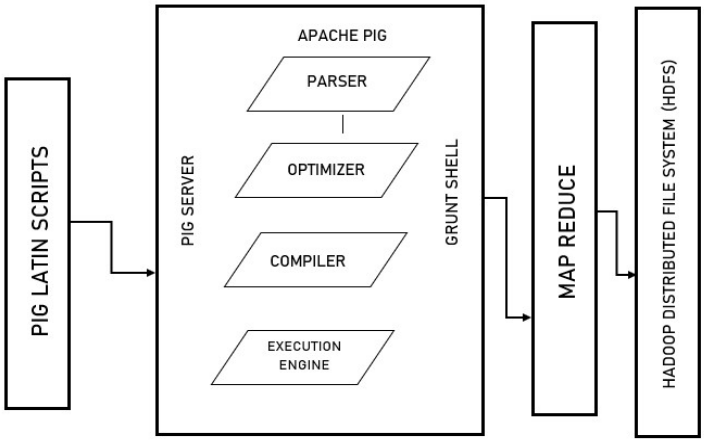


Figure 2.4 Architecture of Apache Pig

Pig Latin is the scripting language of Pig that allows the users to perform data transformations and computations in a clear and understandable way. Its purpose is to make handling big datasets on Hadoop easier and more automated. Pig Latin scripts take use of Hadoop's scalability and fault-tolerance by running concurrently throughout a distributed cluster. Moreover, it offers user-defined functions (UDFs), which let users add new data processing logic to Pig to increase its capability. It is extensively used for many different types of data processing jobs, such as ad hoc analysis, data integration, data cleaning, and ETL (Extract, Transform, Load). It offers a high-level abstraction that facilitates the creation and upkeep of workflows for data processing, enabling users to interact with huge data on Hadoop more easily.

2.3 RECOMMENDATION SYSTEM

A recommendation system is an information filtering system designed to anticipate and propose relevant items or content to users [3]. Its main objective is to offer personalized suggestions that align with the preferences and interests of users that are based on past similarities with other users as well. It is a kind of information filtering system that suggests items based on what is most relevant to a certain user. When a person has to select an item from a potentially overwhelming number of options that a service may give, recommender systems are proved helpful [25]. They have wide-ranging applications across various domains, including e-commerce, streaming services, social media platforms, and online content platforms. Figure 2.5. shows the logical process of a modern recommendation system.

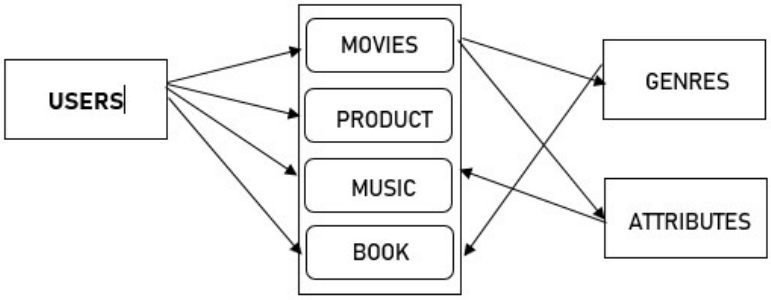


Figure 2.5 Architecture of a recommendation system

By employing machine learning concepts and techniques, a recommendation system can effectively harness the IMDB dataset to generate personalized movie recommendations [31]. Through analysis of the extensive data available in the IMDB dataset, machine learning models can provide precise and pertinent recommendations, enriching the user experience and facilitating the exploration of movies. There are several approaches and techniques employed in recommendation systems:

2.3.1 Collaborative Filtering

Collaborative filtering is a technique used in recommendation systems to provide personalized recommendations to users. It relies on analyzing the behavior and preferences of a group of users to make recommendations to other users [4]. The underlying idea is that users who have similar preferences or exhibit similar patterns of behavior are likely to have similar tastes and preferences for items. Figure 2.6 shows the recommendation system based on collaborative filtering technique.

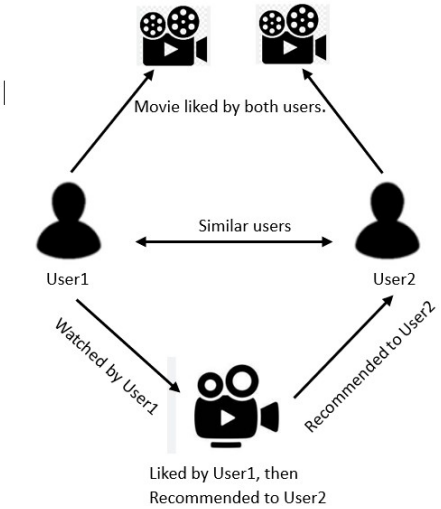


Figure 2.6 Recommendation system based on Collaborative filtering

There are two main types of collaborative filtering: user-based filtering and item-based filtering.

User-based filtering: In user-based collaborative filtering, recommendations are made based on similarity between the users. The system identifies users who have similar preferences or behaviors to the target user and recommends items that those similar users have liked or interacted with.

Item-based filtering: In item-based collaborative filtering, recommendations are made based on the similarity between items. The system identifies items that are similar in terms of user preferences and behavior and recommends items that are like the ones the user has already interacted with.

Collaborative filtering methods do not require explicit knowledge about the attributes or content of the items being recommended. Instead, they rely on the collective behavior and preferences of users to make recommendations.

2.3.2 Content-Based Filtering

Content-based recommendation systems analyze the similarity between different users or items to provide recommendations [25]. These systems examine the attributes and descriptions of items to identify common characteristics among a user's favorite items, which are then stored in the user's profile. In a specific case, a content-based recommender utilized cosine similarity as a prediction method. It considered various attributes such as

keywords, taglines, director, and cast from the sources. A general flow diagram of content-based recommendation system is given in Figure 2.7.

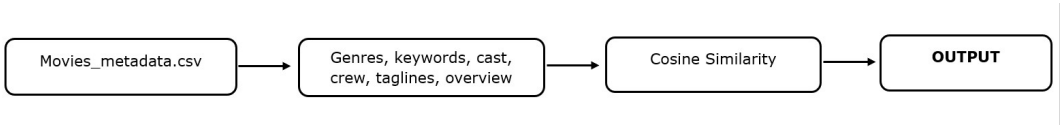


Figure 2.7 Content-based Recommendation system

In the case of content-based recommendation systems, the similarity between the two items is calculated by means of Cosine Similarity Function. It is a similarity metric commonly used in content-based recommendation systems [17]. It measures the similarity between two items by calculating the cosine of the angle between their feature vectors. In the case of Movie Recommendation Systems, the feature vector represents the attributes or content of the movies. The cosine similarity value ranges from 0 to 1, where 0 indicates no similarity, and 1 represents perfect similarity. Equation 2.1 shows the Cosine Similarity between two items:

$$\frac{\sum x_i y_i}{\sqrt{\sum {x_i}^2}\sqrt{\sum {y_i}^2}} \qquad (2.1)$$

where x_i and y_i are the two items whose similarity scores are being calculated. To utilize cosine similarity in a content-based recommendation system, the system first constructs feature vectors for each item using relevant attributes. Then, when a user expresses their preference for a specific item, the system calculates the cosine similarity between that item and other items in the dataset. The items with the highest cosine similarity scores are considered most similar to the user's preferred item and are recommended accordingly [17]. Figure 2.8 illustrates how the cosine similarity technique is used to calculate the similarity vector between two movies M1 and M2.

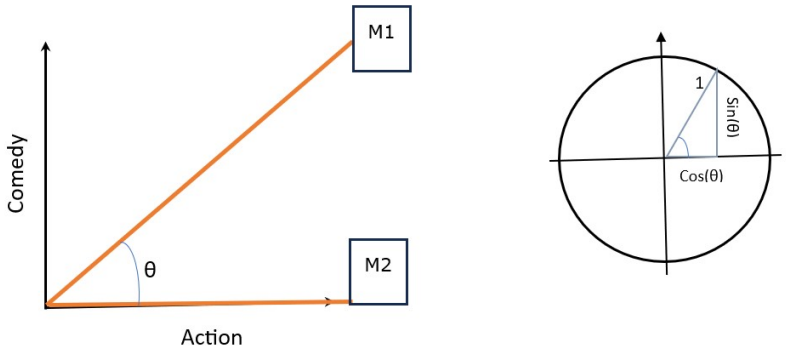


Figure 2.8 Cosine Similarity

2.3.3 Hybrid Approaches

The hybrid recommender system combined the concepts of both content-based and collaborative recommendation systems to generate predictions. Content-based filtering analyzes the metadata to identify similar movies based on attributes such as keywords, taglines, and director. Collaborative filtering, on the other hand, considered the ratings data to identify users with similar preferences [25]. The output of the hybrid recommender system is a list of items, sorted based on the expected ratings by the specific user. This pair enables the algorithm to provide more thorough suggestions by capturing item features as well as user preferences.

2.4 ALGORITHMS USED

2.4.1 Sentiment Analysis

Sentiment analysis, also referred to as opinion mining, is a computational approach employed to ascertain the sentiment or emotional tone conveyed within text-based data [8]. It entails the examination of textual content to identify and extract subjective information, including positive, negative, or neutral sentiments, as well as specific emotions like happiness, anger, or sadness. Through the analysis of sentiment expressed in user interactions, we can acquire valuable insights into their preferences and levels of satisfaction. This valuable information enables us to customize our recommendations to align with their individual tastes and requirements more accurately [19]. Figure 2.9 shows the whole process of sentiment analysis algorithm.

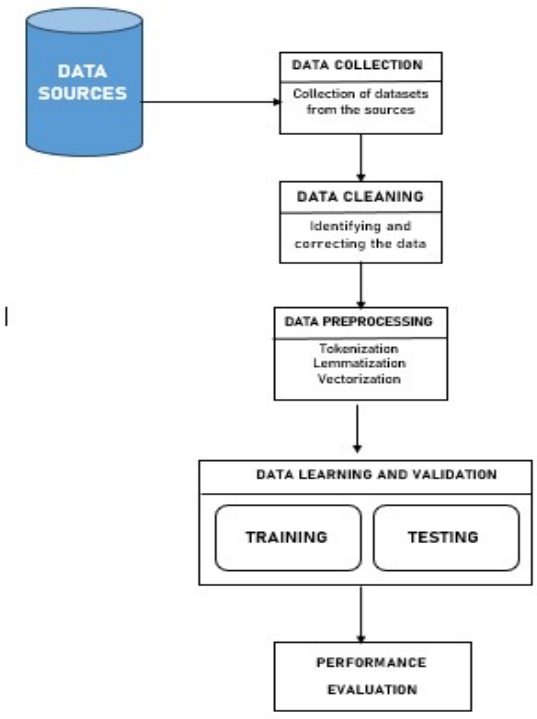


Figure 2.9 Process of sentiment analysis

For sentiment classification, machine learning algorithms such as Logistic Regression, Random Forest, and Support Vector Machines (SVM) can be utilized. The process can involve multiple steps, including data collection, cleaning, and preprocessing, feature extraction, sentiment classification, and evaluation, utilizing techniques like tokenization, stemming/lemmatization, part-of-speech tagging, and sentiment lexicons, conducting of n-gram analysis to explore word sequences, and transforming the text data into numerical vectors using vectorization techniques.

Sentiment analysis can be applied to diverse textual data types, including social media posts, customer reviews, survey responses, and news articles. In this process, word tokenization is the process where sentences or text are divided into individual words or tokens. This step breaks down the text into smaller units, enabling further analysis [5]. For instance, a sentence would be tokenized into a list of words or tokens. Figure 2.10 shows the process of tokenization.

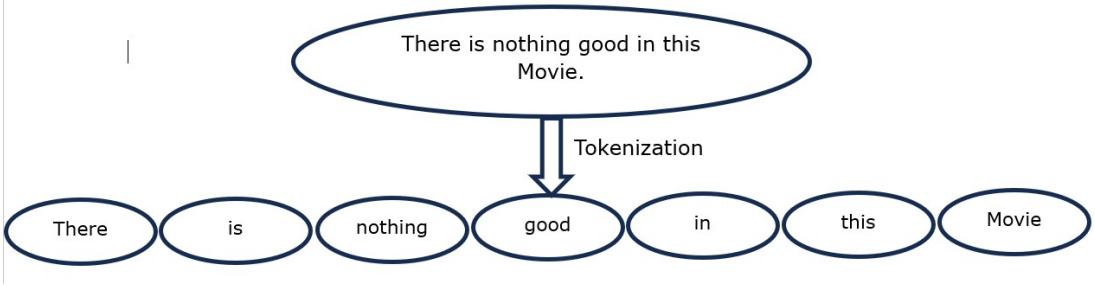


Figure 2.10 Process of tokenization

Lemmatization refers to locating a particular word's lemma or dictionary form. Lemmatization reduces a word's inflectional and occasionally derivationally related forms to its base form. Figure 2.11 shows the process of lemmatization.

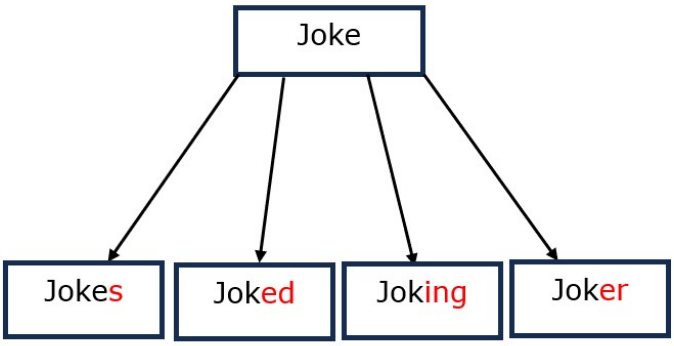


Figure 2.11 Process of lemmatization

To extract features from the tokenized and lemmatized text data, Count Vectorizers are employed. Count Vectorizer is a feature extraction technique that converts text documents

into numerical representations. It counts the occurrence of each word or token in the text and creates a feature vector for each document or sample [30]. These feature vectors can then be used as input for machine learning models.

2.4.2 Convolutional Neural Networks (CNN)

CNN or conventional neural network is a common artificial neural network that serves as the foundation for many deep learning models, often referred to as a feedforward neural network or a multi-layer perceptron (MLP). Its structure is composed of multiple layers of sequentially arranged, networked nodes, or neurons. Each neuron becomes an input for the layer of neurons below it after utilizing inputs and generating an output [20]. The network of CNN is usually made up of three layers including the input, one or more hidden layers, and output.

The data which can be converted into numerical representation is fed into the input layer such as numerical features. Every neuron in the input layer illustrates a feature or an input variable. The values from the input layer are sent to the subsequent levels by the network. The term "hidden layers" refers to the layers that sit between the input and output layers. Every neuron in a hidden layer changes to create an output after getting input from the layer above. An activation function is often performed after the weighted sum of the inputs to complete the transformation. The activation function introduces non-linearity into the data, which allows the network to identify complex patterns and relationships. The network's output layer uses the data processed by the hidden layers to provide the final prediction or classification [20]. Depending on the specific task, the number of neurons in the output layer changes. A single neuron would represent each class in a binary classification problem, for example. However, in a multi-class classification problem, the output layer would include several neurons, each of which would represent a different class. A simple neural network is shown in Figure 2.12.

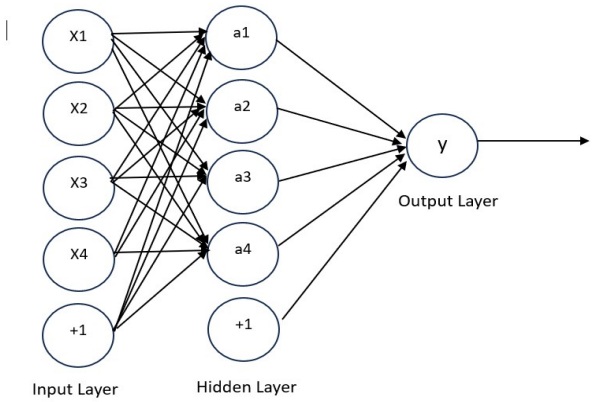


Figure 2.12 A simple neural network

In order to reduce the discrepancy between the anticipated output and the ground truth, a typical neural network modifies the weights and biases connected to each neuron throughout the training phase. This is accomplished by using a loss function, which measures the network's performance, to compare the projected output with the anticipated output.

Iteratively enhancing the network's performance, the backpropagation algorithm—a kind of gradient descent—is frequently employed to update the network's weights and biases.
Currently, there are several prominent architectures in the field of neural networks, including LeNet5, AlexNet, ZF Net, GoogLeNet, and VGGNet. In this analysis, we will focus on the LeNet5 architecture, shown in Figure 2.13 which is a well-known and classic CNN structure primarily used for handwritten font recognition. LeNet5 consists of a total of five layers. Apart from the input layer, each layer has trainable parameters, and multiple feature maps are present in each layer. The purpose of the convolutional layers is to extract input features using convolution kernels. Each feature map within a layer contains multiple neurons. LeNet5's architecture was specifically designed for recognizing handwritten digits. It starts with the input layer, which receives the pixel values of the input image. The subsequent layers consist of convolutional layers, pooling layers, and fully connected layers. The convolutional layers perform convolutions on the input features using trainable filters to extract meaningful patterns and features. This allows the network to learn hierarchical representations of the input data. The pooling layers reduce the spatial dimensions of the feature maps, helping to capture more robust and invariant features. The fully connected layers connect all the neurons from the previous layers to the output layer, enabling the network to make predictions or classifications based on the learned features [29].

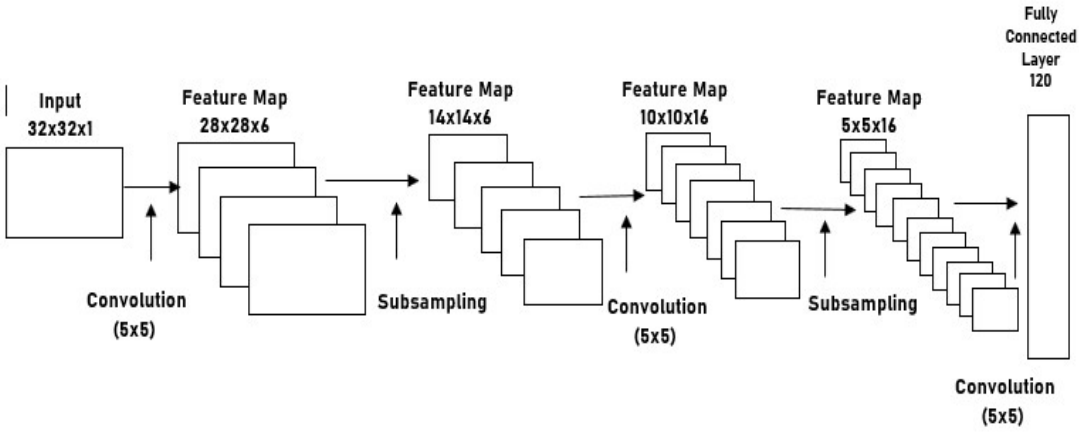


Figure 2.13 Architecture of LeNet5

Conventional neural networks have shown benefits for various applications, including natural language processing (NLP), picture categorization, and time series analysis. But still, because of their limits in capturing intricate patterns and long-range relationships, they may have difficulties managing large-scale and complicated information. As a result of this, more advanced structures, such Recurrent Neural Networks (RNNs), have been developed to handle certain tasks and data types.

2.5 RELATED WORK

Verma, [5] proposed a recommendation system using Hadoop architecture to analyze and process large amount of data from social media platforms in the form reviews, opinions, and remarks. Table 2.1 shows the steps that were required for the designing of the recommendation system. It defined how different types of textual data could be handled to

extract meaningful information from it and also presented a hybrid filtering approach for the system. They had used Apache Hive for the processing of large movie ratings dataset and designed its Mahout Interfaces to analyze the performance with different file sizes in a distributed computing environment using the Hadoop framework.

Table 2.1: Recommendation System Processes

ETL
Feature Generation
Recommendation Algorithms/Model Generation
Workflows & Scheduling
Support Testing of the model
Measure Performance

Sarker, Kamal Uddin, et al. [1], focused on the role of data in AI, machine learning, and data science applications, particularly through the use of web data extraction and Web Scraping techniques. They discussed the implications of biased user ratings and conducted an analysis of movie ratings from IMDB and Amazon. They emphasized the importance of data quality and quantity in influencing machine learning algorithms. The paper presented a novel clustering model based on two different rating systems of IMDB movie data and found that only 42.4% of the extracted dataset records were accepted for research purposes after cleaning.

Rajarajeswari, S., et al. [2] discussed three fundamental approaches commonly used in recommender systems: simple recommender, collaborative filtering, and content-based filtering. They described the basic idea of movie recommendation systems, with ideally analyzed factors such as reviews, cast, plot, crew, genre, and popularity to generate optimal recommendations. The paper also provided predictions based on cosine similarity. The paper also explored the concept of a hybrid approach, which combines these approaches to create a more optimal solution. It described the basic type of recommendation system that compared metrics or weights for all available entities and provided recommendations based on this comparison.

Peng, Y. [3] described that Collaborative Filtering-based (CF) methods were primarily utilized in big data processing platforms due to their parallelization characteristics. The paper presented the basic principle of the recommendation system based on collaborative filtering. The algorithm analyzed item descriptions to identify common attributes in a user's favorite items and stored those preferences in the user's profile.

Wei, Chutian, et al [4] focused on the utilization of the content-based recommendation model to calculate the similarity index between movies and discover users' potential interest in each movie, emphasizing the importance of efficient movie recommendation systems in the digital media platform. The article also discussed the limitations of using cosine distance and proposed the utilization of the Pearson correlation coefficient as an alternative method. They developed a fully content-based filtering recommendation engine using data from IMDB, provided a detailed methodology for finding correlations between movies using the

Pearson correlation method, and evaluated its performance using a sample of user viewing records. The article concluded by discussing potential further work and suggested improvements for the recommendation system.

Singh, Ramni Harbir, et al. [17] gave the concept of cosine similarity to calculate the likeliness between item vectors [18]. They explained that angle theta between the two movies will determine the similarity between them. The similarity between the two vectors increases with decreasing angle. If the value of the theta is near 1 then it is most similar and if it's near to 0 then it is least similar. The movie will be recommended if it is close to 1, otherwise there would be no similarity between them.

Pang et al. [14] classified the emotional polarity of the movie commentary text into positive and negative categories using the maximum entropy (ME) and part of speech (POS) N-gram grammar (n-gram). This was done using supervised learning techniques in machine learning. Turney used machine learning's unsupervised learning technique to investigate the polarity of text emotions [15]. The movie reviews dataset yielded an accuracy of 65.83% for the approach.

Wang, Y. [8] proposed a Sentiment-Enhanced Hybrid Recommender System. They used Euclidean distance to measure the similarity between the items.

$$sim(u_x, u_y) = \sqrt{\frac{\sum_{i=1}^{n}(x_i - y_i)^2}{n}} \quad (2.2)$$

where x_i, y_i are the ratings of movies u_x, u_y respectively.

Wang, Jie, and Zihao Li [20] proposed in their paper, that the image processing based on convolutional neural networks (CNN) required collecting a large number of pictures for computer learning. Many images were collected, and irrelevant parts of the face were cropped. Face detection was used to identify and save the cropped images in a specific folder. The constructed CNN model followed the structure of LeNet5, with layers including the input layer, convolutional layers, pooling layers, fully connected layers, and output layer.

Using a Support Vector Machine (SVM) classifier, Reddy Boyapally and Shashank [22] compared faces to the available faces. In order to improve face recognition, the SVM was trained with the 128 facial features that were obtained. The objective was to find a most extreme division hyperplane that separated the dataset into particular classes, limiting disarray during examination.

CHAPTER 3: METHODOLOGY

The proposed system utilizes three effective algorithms that overall enhance the movie watching experience. The content-based movie recommendation system makes the use of movie metadata and generate recommendations based on genres, and overviews. Other than this, the algorithm of sentiment analysis collects the reviews of users from all over the world and predicts them as either good or bad. It also allows the users to write their owns reviews for the searched movie which are processed and integrated into the system's database. It predicts the sentiments of newly submitted reviews, thus providing real-time feedback. Furthermore, the face recognition model applies convolution neural networks and deep learning concepts to identify the actors from the movie posters by extracting the facial features and recommends movies of the similar actors.
The proposed methodology consists of the following steps:

- Data Collection
- Data Analysis in Hive and Pig
- Recommendation System
- Sentiment Analysis
- Face Recognition using CNN

3.1 DATA COLLECTION

Internet Movie Database (IMDb) is an online database of information related to movies, pictures, video games, podcasts, television series, which contains their cast and crew, personal biographies, plot summaries, ratings, fan and critical reviews. It includes a large amount of data, with millions of users globally, for a wide range of uses, including trend research and recommendation systems. To build a huge dataset for analysis and recommendation engine creation, the data collecting method includes gathering IMDb movie reviews and related movie metadata. IMDb is among the very reliable and upright sources from which movie metadata is first gathered. The reason for using this source is that people from all over the world uses Netflix or other recommendation engines to watch their favorite movies and all these platforms use data from this database. The metadata includes the information of titles, genres, production companies, directors, performers, overviews, runtime, and release dates.
This data gives the engine an initial point for comprehending the features and qualities of the films in its collection. With its extensive database of user-generated reviews and ratings, IMDb offers important insights into the opinions of viewers. Web scraping techniques are used to gather reviews by extracting reviews, ratings, and related metadata. This data serves as a valuable resource for sentiment analysis and user preference comprehension. Additionally, the IMDB movie posters that are gathered via web scraping methods make up the dataset for the facial recognition system. To retrieve the necessary photos or posters, it includes programmatically exploring the web sites. The training of the algorithm uses this dataset as input so that it can recognize patterns and structures in the movie posters.

3.2 DATA ANALYSIS USING APACHE PIG:

The **LOAD** command is used to load the dataset into Pig Latin. The dataset, which can be saved in CSV, TSV, or JSON formats, is identified by its\ location and format in this statement. The Piggybank jar must be registered in order to begin the analysis, adding additional features and tools for Pig Latin. Furthermore, throughout the loading process, the dataset's schema, including the names and types of the columns, should be established.

3.2.1 Movie Recommendation System:

The analysis's goal is to pinpoint and examine each genre's highest-rated films. By filtering the dataset, arranging the films according to vote average, and choosing the highest rated films for each genre, it solves the problem of finding highly rated films within particular genres. This study makes it possible to have a thorough grasp of how well films perform and are popular across many genres, which helps with suggestions for films, marketing tactics, and content production. It is easier to access and do additional research when the findings are stored individually. This makes it possible to conduct in-depth analyses of the traits, themes, and patterns of the highest-rated films in each genre.

3.2.1.1 Data Preparation:

The FOREACH statement is used to pick appropriate columns for analysis once the data has been loaded. Index, genres, keywords, original title, summary, release date, and vote average are some of these columns. 'each_movies' is the name of the data subset that will be utilized in further analysis.

3.2.1.2 Filtering of movies with respect to genres:

The study centers on many movie genres, beginning with the comedy genre. To refine the selection of comedy movies, the FILTER statement matches rows that include the term "comedy" in either the 'genres' or 'original_title' column. The "filterComedy" dataset is the result of the filter. The 'original_title,' 'genres,' and 'vote_average' columns are taken from this dataset and added to the 'comedyMovies' dataset.

3.2.1.3 Ordering and Limiting:

The 'comedyMovies' dataset is sorted using the ORDER statement in decreasing order by 'vote_average' in order to determine which comedies are the most rated. Then, using the LIMIT statement, the top 10 comedic films are chosen and added to the 'topComedyMovies' dataset.

3.2.1.4 Storing the Results:

The findings of the analysis are stored separately for each genre. For instance, the top-rated comedy movie dataset, 'topComedyMovies,' is constructed in the case of comedies. The 'original_title,' 'genres,' and 'vote_average' columns of the highest-rated comedies are included in this dataset. Next, a CSV file containing the 'topComedyMovies' dataset is saved in the specified HDFS location.

Other genres including drama, action, thriller/mystery, horror, and criminal films go through a similar process. Following the analysis, distinct files for each genre—adventure, action,

thriller/mystery, drama, horror, and crime—will be included in the output. These files will all have the top 10 films in that particular category. Every genre is then kept in distinct files within the HDFS after being sorted, filtered, and restricted to the top 10 films. These files will be kept on the HDFS for convenient access and additional examination. Now for example, if anyone is looking for the top-rated adventure films, they may move towards the file that is especially been created for that particular genre. It consist of the list of the top 10 adventure films, along with the genres in which they fall and the average number of votes for each. In a similar manner, by consulting the corresponding files, they may easily view the highest rated films for different genres.

3.2.2 Movie Review Engine:

Film reviews are widely available in the age of social media and internet platforms. However, it takes a lot of time and effort to manually analyze and classify these evaluations. For this reason, processing and classifying movie reviews effectively might benefit immensely from an automated method that makes use of sentiment analysis tools. The data processing language Pig Latin makes it easier to analyze big datasets. The goal is to develop an efficient and effective sentiment analysis model that can reliably classify movie reviews as positive or negative by utilizing Pig Latin's capabilities. Several phases are involved in the approach for creating the Pig Latin-based sentiment analysis model:

3.2.2.1 Data Refinement:

The dataset must be loaded, and then the data must be ready for further examination. Setting the schema and giving the columns the proper data types are the tasks of this stage. To illustrate, the schema will define review ID and review text columns as strings and integers, respectively, if the dataset has these types of columns. This phase can also be used to handle missing values and other data cleaning and preparation activities, such eliminating unnecessary characters.

3.2.2.2 Tokenization:

The process of dividing the review text into discrete words, or tokens, is called tokenization. To loop over each entry in the loaded dataset, use the **FOREACH** command in Pig Latin. A UDF (User-Defined Function) or tokenization function is applied to the review text column within the FOREACH expression. Using certain delimiters, such spaces or punctuation, this function divides the text into tokens.

3.2.2.3 Sentiment Dictionary:

A sentiment dictionary is used to apply sentiment evaluations to the tokens in the analysis. A dictionary-style list of words is generated, along with the corresponding emotion scores. To get the sentiment rating of each word in the tokenized reviews, the lexicon is checked. Every word in the sentiment dictionary has an associated sentiment rating, and each item can be retained as a stand-alone dataset.

3.2.2.4 Joining Tokens and Dictionary:

A join operation is carried out between the tokenized reviews and the sentiment dictionary in order to link each token with its matching sentiment rating. The two databases are combined using the **JOIN** command in accordance with terms that match. As a consequence, a new dataset is created, in which every record has a token, its sentiment rating, and additional pertinent data from the first evaluations.

3.2.2.5 Calculating Average Ratings:

The data is sorted by review ID and text to get the average sentiment rating for each movie review. The sentiment rating column in the **FOREACH** statement is subjected to the **AVG** function following the data grouping process. This function creates a new dataset containing the review ID, text, and average rating after calculating the average rating for each group.

3.2.2.6 Filtering Positive and Negative Reviews:

Reviews that are categorized as favorable or negative are segregated in the filtering stage according to their average sentiment ratings. Two distinct datasets, one with good ratings and the other with negative reviews, are produced using the **FILTER** command. One way to specify the criterion for classifying reviews as good or negative is to use a threshold number for the average sentiment rating.

3.2.2.7 Results Storage:

After the filtering process, the positive and negative reviews are kept in different files for later examination. The datasets are saved into the file system using the Pig Latin **STORE** command. The output files' location and format—such as CSV or JSON—are specified in the statement. The review ID, review text, and average sentiment rating for each good or negative review will be included in each file.

3.3 LIBRARIES USED

3.3.1 Pandas

The Pandas library is widely used in machine learning algorithms for the purpose of data manipulation, transformation, filtering, and analysis. It allows for the handling of structured data such as tabular data with the help of data structures and functions. Here it is used to load the dataset files into the data frames so that we can preprocess and clean the metadata by removing the irrelevant columns, handling missing values, and converting data types.

3.3.2 Scikit-learn

Scikit-learn or sklearn is a python library that provides a comprehensive set of tools for various tasks including regression, classification, and model selection. It supports both supervised and unsupervised learning techniques. In this algorithm, it is used to import Count Vectorizer and Cosine Similarity Function. Count Vectorizer, a feature extraction function is used here to convert the textual movie attributes such as movie title, genre, overviews into numerical representations that can be used by the algorithm. While the cosine similarity function, imported by the scikit-learn is used here to calculate the

similarities between two movies by utilizing their numeric information generated through the count vectorizer.

3.3.3 Pickle

The Pickle library in python is used to save the trained machine learning models to the local storage and can also load them back when needed. In this algorithm, it is used to create files of movie list and similarity. The movie list file contains the new data that includes the movie title combined with index and the similarity file consists of the calculated values of cosine similarities for all the movies in the dataset.

3.3.4 Matplotlib

The matplotlib library is a popular data visualization library in Python programming. It helps to create line plots, scatter plots, bar plots, histograms, pie charts, and many other types of visual representations of data. It offers a flexible and customizable interface to control various aspects of the plots, including colors, markers, labels, titles, axes, and legends. In sentiment Analysis we have used it to create and display the word cloud visualization which is basically generating two different objects for both positive and negative reviews. It will help to classify the reviews and extract sentiments out of it.

3.3.5 Nltk

NLTK offers functionalities for essential NLP tasks like tokenization, stemming, lemmatization, part-of-speech tagging, parsing, and semantic reasoning. Additionally, it provides comprehensive corpora, lexicons, and language models tailored to different languages, empowering researchers, developers, and data scientists to efficiently perform a wide array of NLP tasks. In the realm of sentiment analysis, NLTK proves invaluable by offering sentiment lexicons, including **WordNet-Affect** and **SentiWordNet,** which associate sentiment scores with words or phrases. These lexicons facilitate the determination of sentiment polarity within a given text. NLTK also supports machine learning approaches for sentiment analysis, providing wrappers around popular algorithms like Naive Bayes making the training and evaluation of sentiment analysis models more accessible. Furthermore, NLTK offers datasets such as the Movie Review Corpus, which can be utilized for training and testing sentiment analysis models. Its seamless integration with other libraries, such as scikit-learn, enables the combination of functionalities for more advanced sentiment analysis tasks.

3.3.6 OpenCV 2

OpenCV (Open-Source Computer Vision Library) is a widely used computer vision library that provides various functions and algorithms for image and video processing. In the actor-based recommendation system, CV2 can be used for tasks such as reading and preprocessing images, detecting, and cropping faces from movie posters, and performing image transformations on it. CV2 offers a rich set of functionalities that can assist in data preprocessing and preparing the input images for training or inference with the CNN model.

3.3.7 Face Recognition

The face recognition library is specifically designed for face-related tasks, such as face detection, face alignment, and face recognition. This library provides pre-trained models and functions that simplify the process of facial feature extraction and recognition. In this recommendation system, the face recognition library can be utilized to detect and align faces in the movie posters, extract facial features from those aligned faces, and compare the features for similarity matching. The library may also offer convenient methods for generating face embeddings or encoding facial features that can be used as input to the CNN model.

3.4 MOVIE RECOMMENDATION SYSTEM

3.4.1 Data Inspection

This step consists of the detailed exploration of the dataset of movies metadata. By means of different commands from the Python environment, we can check and determine, how the actual dataset looks like. This is done through various strategies including visualizing the whole dataset or by looking for different types of uncertainties that may occur in it so that we can remove them in initial steps. Firstly, the dataset file is loaded into python and then by using various commands, it is inspected.

film.head command here is used to comprehend the attributes of the dataset, their values, and datatypes. We have used it to display the initial ten movies including their ID, Title, genre, collection, overview, popularity, original language, release date, runtime, vote average, and vote counts from the dataset. The purpose behind using this is to quickly get a glimpse of how the data looks like.

film.describe function is used to manipulate the tabular dataset to generate its descriptive statistics. By using it we have displayed the count, mean, standard deviation, minimum and maximum in terms of ID, popularity, vote average and vote count. These statistics can give the idea of the overall distribution is valuable for understanding the characteristics of the data in designing and implementing the recommendation algorithm.

film.info is used to illustrate the range index of the dataset file, by displaying the total number of entries or rows in it, total columns, count of the entries, non-null entries, and their count in terms of d-types. It also helps to find the total memory usage that dataset file is acquiring.

The dataset is further inspected by using the **film.isnull( ).sum( )** command which basically displays the number of null values within all of the attributes or columns of the file. The primary purpose of using it is to check whether there exists any empty or invalid value within the dataset so that we can apply any kind of preprocessing to cope up with it.

3.4.2 Selection of Attributes

In this part, we basically have to extract those columns which will be used in the designing of the recommendation system. For example, the dataset includes the attributes of ID, Title, genre, collection, overview, popularity, original language, release date, runtime, vote average, and vote counts. But the system has to recommend movies based on the genres and overviews of the searched movie, and the result will be in the form of movie ID and their

respective titles. The genres give the details of characteristics, themes, and styles of the movie, while the overview refers to a summary of the respective movie that provides the general interpretation of movie's storyline, main characteristics and key elements. Both attributes are combined to provide comprehensive and enhanced recommendations. So, we only have to use the features or attributes of ID, title, genres and overviews.

Firstly, we have displayed the columns of dataset by using the **film.columns** command. It gives information of the data types of all the attributes along with them as well. Then, we extracted the ID, title, genre, and overview columns from the dataset, that will be used to recommend the movies. And from these, the genre and overview attributes are combined together under the tag column. This single column is used for the conversion and extraction purpose in the next steps.

3.4.3 Text Vectorization

The process of transforming textual data into a numerical representation suitable for analysis and machine learning applications is known as text vectorization. It involves turning unstructured text materials into a format that is organized and readable by algorithms. These vectors are then utilized in calculating the similarities between the movies for the recommendation purpose. For the feature extraction purpose, we have used sklearn library, to import the Count Vectorizer. It is basically used to convert the textual data that is the combination of genre and overview attribute into the numeric representation. These numeric feature matrices are used by the system for further processing. The maximum rows of the dataset and the stop words language, both are defined. According to the number of rows, the dimensions of the vector have resulted in 45000x45000. Lastly, the object vector fits into the text data. This step analyzes the text and builds the vocabulary (a set of unique words or terms) based on the 'tags' column of the new_data data frame. It learns the vocabulary and assigns a unique index to each word. The **.toarray()** method is then used to convert the resulting sparse matrix representation into a dense array format.

3.4.4 Applying Cosine Similarity

Cosine similarity technique is used here to calculate the similarity between different movies based on their genres and overviews that were previously combined into a single attribute. It measures the similarity by calculating the cosine of the angle between their feature vectors. Its value ranges from 0 to 1, where 0 indicates no similarity, and 1 represents perfect similarity. The array that will be formed as a result of this will be used to recommend the movies for the searched ones. The movies with the highest cosine similarity scores are considered most similar to the user's preferred ones and are recommended accordingly.

Firstly, we have imported the cosine similarity function from the sklearn metrics pairwise library. Then, the vector which was created in the text vectorization step is fed into the function and the similarity is calculated. These similarity values are then used to predict the most similar movies in the form of recommendations.

3.4.5 Recommendation

This step is used to create a python function for the recommendations. The similarity values from the above step are utilized in recommending movies to the users. The function first finds the title column of the dataset that is accessed by the function so that it can be used to match the titles of movies for analysis. After that a movie title is given to the function in the first step, it finds the index number of it from the whole dataset. This creates a Boolean mask by comparing each element in the 'title' column with the string of the given movie title The result is a series of True and False values, where True indicates a match. Then, it retrieves the similarity scores for that movie from the similarity matrix. The key=lambda vector: vector [1] command specifies that the sorting should be based on the second element of each tuple, which is the similarity score. The reason is to recommend movies that are highly similar. A tuple is created containing the index of the given movie and its similarity score using the enumerate(similarity) function. The similarity scores are sorted in descending order, excluding the first entry (which corresponds to the movie itself). The top N similar movies are selected based on the sorted similarity scores. Finally, the function retrieves the titles of the top similar movies and prints them as recommendations for the given movie title.

3.4.6 Data Serialization

Data serialization is the process of converting complex data structures, such as objects or data collections, into a format that can be stored or transmitted. We have used the pickle library from the python environment to serialize the calculations done above into the binary format to store them in the form of different files. This step consists of creating two files from the above system that are the movie list and similarity files.

- Movie list file consists of the new_data that includes the movie ID, title and their genres and overviews combined together.
- Similarity file includes the similarity scores of all the movies in the new_data.

Both files are used in the recommendation system to recommend the movies to the users based on the one they have searched for.

3.5 USER REVIEW PREDICTION

3.5.1 Design Explanation

The design of the sentiment analysis system has incorporated various essential components, utilizing the vast amount of user reviews from the IMDB dataset. The system has efficiently collected a significant volume of user reviews from the dataset, serving as the training data for the sentiment analysis model. These reviews have been annotated with sentiment labels, including positive, negative, or neutral. Through the application of advanced machine learning techniques, the model has undergone training on this dataset, acquiring the ability to recognize intricate patterns and distinguishable features that indicate sentiment. After the sentiment analysis model has been trained, it has the capability to analyze forthcoming user reviews. When a user submits a review in the comment section, the system processes the

text and employs the sentiment analysis model to classify the sentiment of the review as bad, good, or normal. This classification is determined by utilizing the patterns learned from the training data, ensuring accurate sentiment analysis of new user reviews. Additionally, the system has integrated a server that serves as a centralized storage and management repository for all user reviews, including newly submitted ones. This server plays a crucial role in collecting and organizing the reviews efficiently. Each individual review's sentiment classification, along with the corresponding review text, has been stored in the server, ensuring that the sentiment analysis system maintains a comprehensive record of user feedback in it.

3.5.2 Data Preprocessing

During this stage, duplicate entries are removed, missing values are handled, and formats are standardized to ensure data consistency and accuracy. This cleaning process is crucial in preparing the data for subsequent analysis and ensuring the integrity of the information within the dataset.

3.5.3 Text Cleaning

In sentiment analysis, the dataset we have used is typically consisting of texts or documents along with their corresponding sentiment labels. However, challenges have arisen from duplicate entries and missing values, which can affect the accuracy and reliability of the sentiment analysis model. Data cleaning steps have included handling missing values and removing duplicate entries. Missing values may have occurred when sentiment labels were incomplete or unavailable for certain texts, and duplicate entries may have arisen due to various reasons, such as data collection errors or multiple instances of the same text with different sentiment labels.

Dataset.dropna() This function has been used to remove rows or columns that contain missing values. By eliminating these missing values, the dataset has become more consistent and complete, providing reliable data for training the sentiment analysis model.

Dataset.drop_duplicates() This function is used to identify and drop any duplicate entries, ensuring that each text has been represented only once in the dataset.

3.5.4 Tokenization

The tokenization process in the model begins with receiving a text document as input, which could be a sentence, paragraph, or entire document. If there have been multiple sentences, the text has optionally been segmented into individual sentences. Tokenization has then been applied to divide the text into individual words or tokens. Tokenization techniques include whitespace tokenization, which splits the text based on spaces, or more advanced approaches using regular expressions or pre-trained models for word boundary detection. Tokenization is also used for handling special cases like contractions, where “don't” have been tokenized as separate tokens “do” and “not,” and hyphenated words like “state-of-the-art” have been divided into “state,” “of,” “the” and “art”. The output of tokenization results in a sequence of tokens that can be further processed for feature extraction, vectorization, or classification.

3.5.5 Stop Word Removal

Stop words are those words that commonly occur in a sentence (e.g., “and,” “the,” “is”) that does not carry significant meaning for sentiment analysis. Removing stop words helps to reduce noise and has focused attention on more meaningful words. By removing these commonly occurring words, our sentiment analysis process has become more efficient and effective in identifying the words that carry sentiment or contribute to the overall meaning of the text.

3.5.6 Word Cloud Visualization

The word cloud visualization provides a visual summary of the most prominent words in the analyzed text data. We have used word cloud visualization in our model because it enables users to quickly grasp the overall sentiment or key themes present in the text documents. The larger and more prominent words in the word cloud represent the most frequently occurring or significant terms related to sentiment, making it easier to identify major sentiments or topics. Word cloud visualizations have been frequently employed in sentiment analysis to gain insights, detect trends, or effectively communicate findings in a visually appealing and accessible manner. In output it gives us a separate document for positive and negative sentiment which consist images of the words in different size format. For positive documents in sentiment analysis, it displays the most frequent and representative positive sentiments or words in a visually appealing manner and same for the negative documents it displays the negative sentiments or words.

3.5.7 Text Vectorization

Text vectorization has remained a fundamental process in sentiment analysis, allowing for the conversion of textual data into numerical representations suitable for machine learning models. We have used **Tfidf Vectorizer** technique that transforms text documents into a matrix of numerical features, considering the essential aspects of term frequency (TF) and inverse document frequency (IDF) in the text. Term frequency represents the number of times a word has appeared in a document, while inverse document frequency has measured the rarity of a word across all documents in the dataset. The combination of term frequency and inverse document frequency, achieved through a multiplication operation, has continued to yield the TF-IDF score, representing the importance of each word in a document relative to the entire dataset. The TF-IDF scores for all words in a document have been organized into a vector, creating a numerical feature representation for the document. Each dimension of the vector has corresponded to a unique word, and the value in each dimension has represented the TF-IDF score of that word in the document.

3.5.8 Logistic Regression Analysis

Logistic regression, serves as a fundamental machine learning algorithm, has been extensively utilized in our sentiment analysis model for the purpose of analyzing and classifying text data based on sentiment. Within this model, we have used logistic regression algorithm that has successfully determined a decision boundary that separates the input data points into two distinct classes: positive sentiment and negative sentiment. By examining the TF-IDF vector representations of the text documents, the logistic regression model has

effectively approximated the probabilities associated with each document belonging to either the positive or negative sentiment category. In logistic regression, the input features are combined linearly by coefficients (weights) and an intercept term. This linear combination is subsequently transformed via the logistic function, generating a probability value ranging between 0 and 1. This probability signifies the likelihood of the instance being associated with the positive class.

3.5.8.1 Training and Testing

We have utilized Logistic Regression for training and testing purposes. Throughout the training phase, the model has been exposed to a labeled dataset containing text documents and their corresponding sentiment labels. By analyzing the TF-IDF vector representations of the text data, the logistic regression model has learned the optimal weights and coefficients that have best fit the training data. This learning process, involving the minimization of a cost function through iterative optimization techniques like gradient descent, has aimed to discover the decision boundary that has effectively separated positive and negative sentiment instances.

Once the model has been trained, it is assessed using a separate testing dataset. The objective of this testing phase is to evaluate the model's generalization capability and its performance on unseen data. The logistic regression model has predicted the sentiment labels for the testing documents based on their TF-IDF representations. Then we compared these predictions to the true sentiment labels to measure the model's accuracy, precision, recall, and F1 score. By conducting testing on a distinct dataset, we have been able to evaluate the logistic regression model's performance within real-world scenarios. The testing phase has offered insights into the model's ability to generalize its learned patterns to new, unseen instances and make precise sentiment predictions. This evaluation has played a crucial role in assessing the reliability and effectiveness of the logistic regression model for sentiment analysis tasks.

3.5.8.2 Confusion Matrix

The confusion matrix is a vital tool in sentiment analysis models as it presents a tabular summary of the model's predictions in comparison to the actual sentiment labels of the test dataset. It consists of four key elements: true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN), which represent the frequencies or counts of the model's predictions relative to the actual sentiment labels. Accurate predictions of positive sentiment instances are classified as true positives (TP), indicating that the model has correctly identified instances with positive sentiment. Likewise, true negatives (TN) represent instances where the model has correctly predicted negative sentiment, accurately identifying instances with negative sentiment. False positives (FP) occur when the model mistakenly predicts positive sentiment, incorrectly identifying instances as positive when they are negative. Conversely, false negatives (FN) arise when the model incorrectly predicts negative sentiment, misclassifying instances as negative when, they are positive. The confusion matrix's primary objective in sentiment analysis is to offer a comprehensive overview of the model's predictive performance across different sentiment classes, typically positive and negative. We have used it to assess the model's accuracy, precision, recall, and other performance metrics.

By analyzing the values within the confusion matrix, we can gain insights into the model's strengths and weaknesses. It allows us to evaluate the model's ability to accurately predict sentiment labels and assists in making informed decisions to improve its performance and accuracy in classifying sentiment in text data.

3.5.9 Serialization of Data

The process of data serialization in sentiment analysis models using the pickle library involves saving the trained model and associated data structures into a serialized binary file. This ensures easy storage, sharing, and deployment of the model for sentiment analysis tasks. After training the model with a labeled dataset and performing necessary preprocessing steps, the pickle library is utilized for serialization and deserialization. Serialization is carried out using the **pickle.dump()** function, which takes the trained model object and a file object to write the serialized data. The serialized model is saved as a binary file, typically with a **". pkl"** extension. Other relevant data structures like preprocessed data or feature representations can also be serialized in a similar manner. Deserialization is performed by loading the serialized file using **pickle.load(),** which returns the deserialized model object for sentiment analysis predictions. This serialization process enables convenient storage, sharing, and reuse of trained sentiment analysis models, preserving their parameters and data structures for consistent and reproducible sentiment analysis tasks.

3.5.10 Sentiment Prediction

Once the sentiment analysis model is serialized and saved, it can be loaded to make predictions on user reviews. By reconstructing the serialized model using **pickle.load(),** it becomes possible to predict the sentiment labels (positive or negative) of new user reviews. These sentiment predictions are obtained by feeding the new reviews through the loaded model. The output of sentiment prediction is typically a set of sentiment labels assigned to each user review. These labels indicate whether the expressed sentiment in the review is positive or negative. Positive sentiment labels signify satisfaction, appreciation, or a favorable opinion, while negative sentiment labels indicate dissatisfaction, criticism, or an unfavorable opinion.

To facilitate further analysis and utilization, the sentiment prediction output can be stored in a dataset file or a database. This allows for easy management and examination of sentiment-related information. The saved data can be organized based on user reviews or other relevant criteria, enabling tasks such as sentiment analysis aggregation, trend analysis, or decision-making based on sentiment.

In summary, after serializing and saving the sentiment analysis model, it can be loaded to predict the sentiment labels of new user reviews. The output of sentiment prediction comprises sentiment labels indicating positive or negative sentiment. This output can be stored in a dataset file or a database, facilitating management, analysis, and subsequent tasks related to sentiment analysis.

3.6 ACTOR-BASED MOVIE RECOMMENDATION SYSTEM

3.6.1 Design Explanation

To design a system for face recognition using Convolutional Neural Network (CNN) in the context of recommending movies based on the same actor, we start by creating a dataset of labeled images containing actors' faces from movie posters. The CNN architecture is defined with convolutional layers for feature extraction, pooling layers for down sampling, and fully connected layers for classification. CNN is trained using the labeled face images, enabling it to recognize and differentiate between different actors. Once we have trained our model, it extracts features from the face of an actor on a movie poster, which are then compared with features of other actors in the dataset using similarity measures like cosine similarity or Euclidean distance. The movie ID with the closest matching actor features is returned as the output, indicating a movie featuring the same actor. This process capitalizes on CNN's ability to learn facial features, allowing for personalized movie recommendations based on actor resemblance.

3.6.2 Dataset

Due to the lack of an appropriate dataset containing movie posters in IMDB, we have constructed the dataset for our recommendation face recognition system. This self-built dataset includes movie-specific information such as movie ID, actor name, movie title, and a dedicated column for the movie poster image itself. By creating this dataset, we ensured the incorporation of crucial details required for training and evaluating the face recognition system, enabling it to recommend movies based on actor similarity. The poster column provides visual representations of the movie posters, which are helpful in utilizing Convolutional Neural Networks (CNNs) to extract facial features and achieve accurate actor recognition.

3.6.3 Extracting Facial Features

In this model, the face encoding process begins by identifying the poster image related to the movie title entered by the user. The face recognition library is then utilized to perform face encoding on the target actor's face within the poster, generating a numerical representation of their unique fac ial features. Subsequently, the faces of all other actors in the dataset are encoded by iterating through each row and loading the respective image files. The face encoding function is applied to extract facial features, capturing the essence of each actor's face. These face encodings, along with their corresponding actor names, are stored in a list. Ultimately, the output contains the list of encoded faces of all actors in the dataset, enabling comparison with the searched actor's face encoding.

3.6.4 Target Actor Face Detection

The face encodings are utilized to assess the similarity of facial features among actors, employing methods like cosine similarity. It analyzes a dataset containing movie titles, poster image paths, and encoded actor faces. By comparing the face encodings, the model will identify actors whose facial characteristics closely resemble those of the target actor. The process involves iterating through each dataset row, loading poster images, and

extracting the encoded faces using face recognition techniques. These encodings are then compared to the target actor's face encoding using a specified tolerance threshold. When a match is discovered, the corresponding actor ID is recorded in a list of matching results. In the end, the output is a list of actor IDs associated with posters that exhibit face encodings like those of the target actor.

3.6.5 Creating Bounding boxes for identification

The purpose of creating bounding boxes is to visually emphasize the detected faces within an image, aiding in their identification and differentiation from the surrounding content. These bounding boxes serve as a visual representation of face locations, facilitating face recognition, analysis, and subsequent processing. To create these bounding boxes, the model requires two parameters: the file path to the movie poster image and the associated actor's name. The code initially loads the image and converts it from RGB to BGR color space using OpenCV. It then detects the face locations within the image and draws rectangles (bounding boxes) around each face using **cv.rectangle**, with specified coordinates, color, and thickness. Finally, the actor's name is displayed on the image with designated font properties.

3.6.6 Matching Results and Recommendations

The output from our model is used to identify and recommend matching results. It begins by using the target image to perform face recognition and comparison, resulting in a list of actor IDs that closely resemble the target actor's face. These matching results are then used to provide recommendations based on actors with similar facial features.
If the length of matching results is greater than zero, indicating that there are matching results available. It selects the first matching result as the primary actor ID for further processing. Finally, the model utilizes the provided image path and actor's name to display the movie poster. It overlays a bounding box around the identified face and includes the actor's name on the image.

3.7 FRONT-END DESIGN

To facilitate seamless user interaction, both the content-based recommendation and sentiment analysis algorithms are integrated into a user interface. This interface empowers users to not only watch movies and receive personalized recommendations based on their preferences but also provides the functionality to predict their own reviews using sentiment analysis. The user can have a detailed view of information related to the cast and crew, actors' details, and other similar movies as recommendations to the movie they have searched for. Other than this, the sentiment analysis feature used on the website enables the users to have a look at the reviews along with their associated sentiments below every movie which they search for. This makes it easy to them to analyze whether the movie is according to their choice or not.

3.7.1 Home Page

This is the starting page or front page of the website where the users are greeted to get into the seamless browsing experience related to their favorite movies. Their interests in movies

and film-related content are catered to by this dynamic home page. Via several links that take users to other sections of the website, it provides an abundance of choices. These connections provide doors to a wealth of interesting stuff, such as news, interviews, movie reviews, and genre-specific articles. Additionally, there is a subscription button that is positioned prominently on the main page in an effort to draw people in. Users only need to click to be sent to the subscription page, where they can take advantage of the exclusive opportunity to remain in touch and get the most recent updates about movies and films. In addition to maintaining customer engagement, this subscription service offers excellent chances for focused marketing campaigns. A login button is also included on the main page, giving users the option to create a new account or log in using their current user ID. This feature allows users to watch movies and make use of personalized features on the website more easily. Our website strives to provide an immersive and user-friendly environment that meets the many demands of movie aficionados by smoothly combining these capabilities.

3.7.2 About Us

The two columns that make up the "About Us" page each have a different function that helps visitors get a full picture of the website and what it has to offer. The objective and value proposition of the website are briefly introduced in the left column, drawing readers in and encouraging them to explore deeper. The purpose of this introduction is to spark attention and trust in the audience right away. Two buttons are positioned in the right column to provide visitors with quick access to the suggestion and review pages. These buttons act as unambiguous navigational indicators, directing visitors to the precise areas containing well-curated suggestions and in-depth reviews. Furthermore, social media icons have been included into the page so that readers can easily interact with the social media platforms that are linked to the website. With the help of these icons, anyone can easily keep up with the most recent reviews and suggestions and interact with the active community on the website.

3.7.3 Trending Movies

A selection of films from different genres are showcased on the "Trending Movies" page, which is arranged into several categories such popular movies, applauded TV series, and upcoming movies. It is simple for users to look for their favorite movies or find out about new and well-liked ones. A "Watch Trailer" button appears next to every film, enabling viewers to have a sneak peek and decide whether or not to watch it. The website seeks to give cinema buffs an easy-to-use interface via which they can find, examine, and assess films according to their tastes and inclinations.

3.7.4 Contact Us

The Contact Us page is a vital part of the website's capacity to interact with users in an efficient manner. Several features on this important website are designed to make getting in touch simpler. Regardless of the queries, suggestions, or remarks, the user-friendly interface ensures a seamless and pleasant experience. The Contact Us page has an easy-to-use, although basic, contact form. By entering username, email address, and message in this form, the user may directly message or ask questions.

3.7.5 Review Prediction

A container element encloses the primary content part, offering structure and design. It consists of two main parts: a star-rating widget and a movie review form. Users can submit their movie reviews using the movie review form. It has an input field, which is usually a text field that may hold several lines of text. Users are able to express their ideas, comments, and criticism on a film. There is a "Predict" button accessible beneath the input box. The JavaScript code predicts the tone of the movie review when users click this button by processing the form submission event. The code pulls the movie review from the input field, stops the usual submission action, and uses a machine learning method to figure out how sentiment is expressed. An element with the id "sentiment Result" displays the sentiment prediction result, which might be positive or negative. Furthermore, the algorithm dynamically modifies the star-rating symbols' hue according to the anticipated attitude. Users can rate the film using the star-rating widget. It has five radio buttons, each of which represents a distinct star rating. To express their rating, users can choose from one of the radio buttons.

3.7.6 Parental Guide

The purpose of the "Parental Guidance" page is to help parents choose the information that is best for their kids. This website provides a thorough resource for learning whether films, TV series, and other media are appropriate for which age groups. Every piece of material undergoes a thorough evaluation process before being assigned a category based on suitability, which considers elements like language, violence, and mature themes.

The platform is simple for parents to access and offers a wealth of options for age-appropriate media. The information is divided into age categories so that parents can quickly locate the best advice for the particular requirements of their kids. This website contains guidance as well as tools and pointers for encouraging responsible media usage and starting media-related talks with kids.

CHAPTER 4: HARDWARE IMPLEMENTATIONS/ SOFTWARE SIMULATIONS

4.1 ANALYSIS OF MOVIE REVIEW ENGINE ON APACHE PIG

4.1.1 REGISTER Statement:

The REGISTER statement in Pig Latin is used to include external libraries or JAR files that provide additional functionalities. In this thesis, the Piggybank JAR file is registered to access the built-in UDFs required for tokenization and other data processing tasks.

4.1.2 LOAD Statement for Review Data:

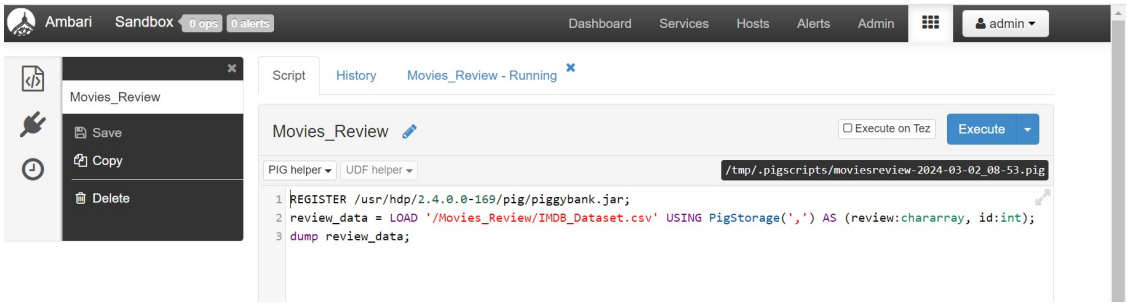


Figure 4.1 Loading Review Data

Here it loads the data from the CSV file '/Movies_Review/IMDB_Dataset.csv' using a comma as the delimiter and assigns the loaded data to the relation 'review_data' with two columns: 'review' of type chararray (string) and 'id' of type int (integer).

4.1.3 FOREACH Statement for getting details:

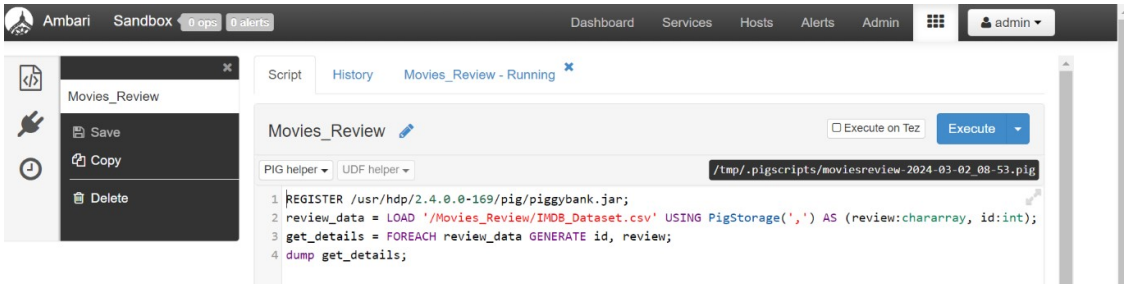


Figure 4.2 Applying FOREACH statement

It creates a new relation called 'get_details' by selecting the 'id' and 'review' columns from the 'review_data' relation.

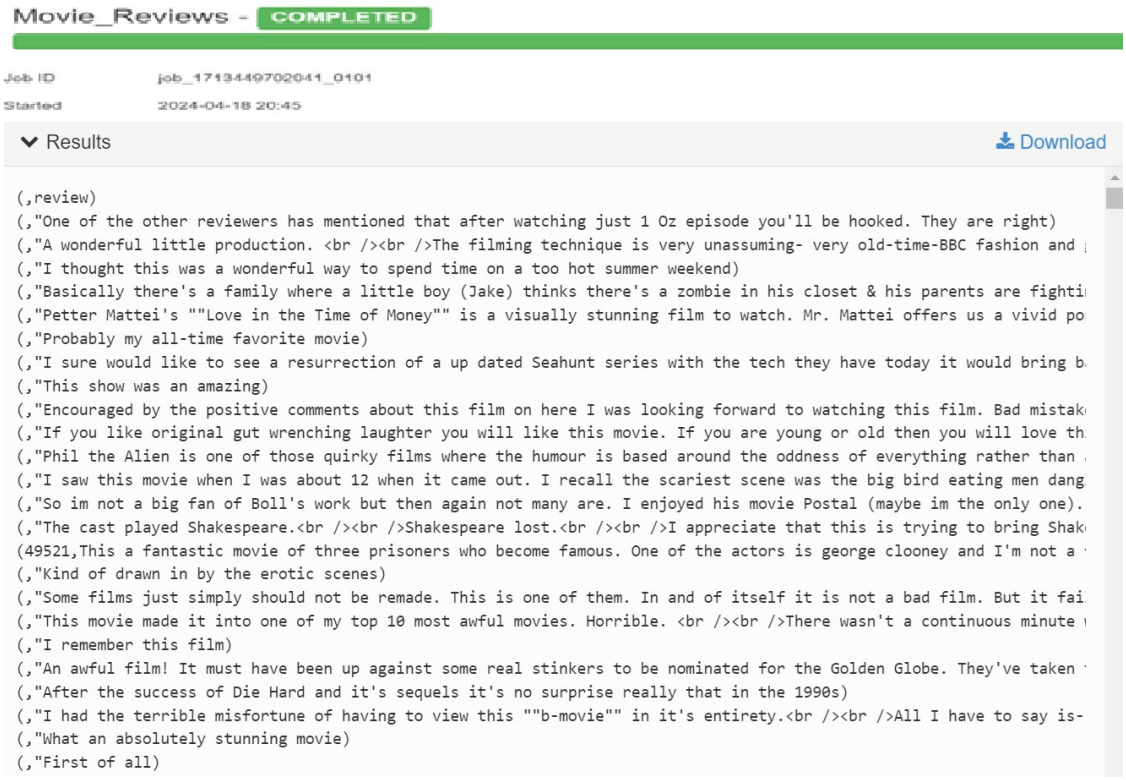


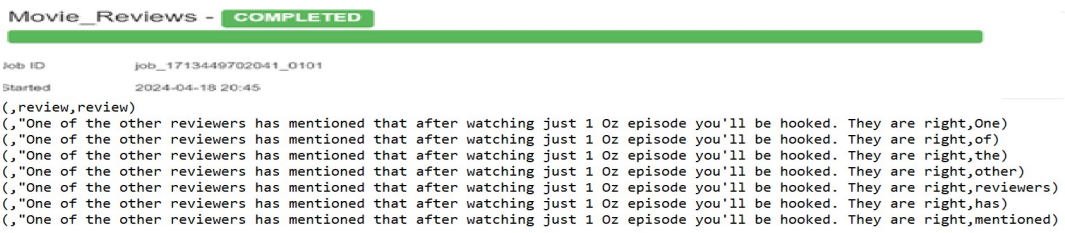
Figure 4.3 Result of FOREACH statement

4.1.4 FOREACH Statement for tokenization:



Figure 4.4 Applying FOREACH statement for Tokenization

It will extract the 'id' and 'review' columns from the 'get_details' relation and applying the TOKENIZE function to the 'review' column, resulting in a flattened relation with an additional column named 'words' containing the individual tokens from the 'review' text.



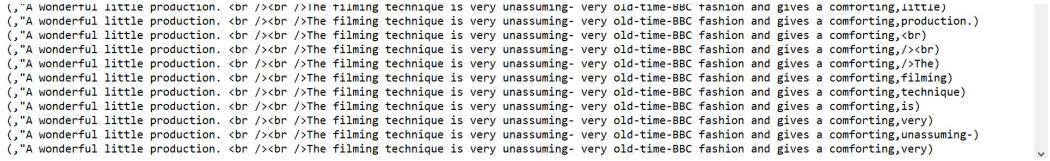


Figure 4.5 Result of FOREACH statement on Tokenization

4.1.5 LOAD Statement for Sentiment Dictionary:

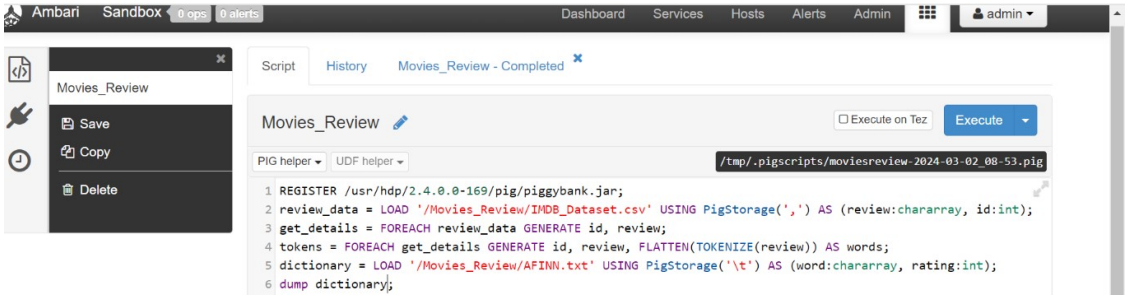


Figure 4.6 Loading Sentiment Dictionary

It will load a dictionary from the file '/Movies_Review/AFINN.txt' into a relation called 'dictionary', where each word is associated with a sentiment rating, serving the purpose of sentiment analysis or sentiment scoring.

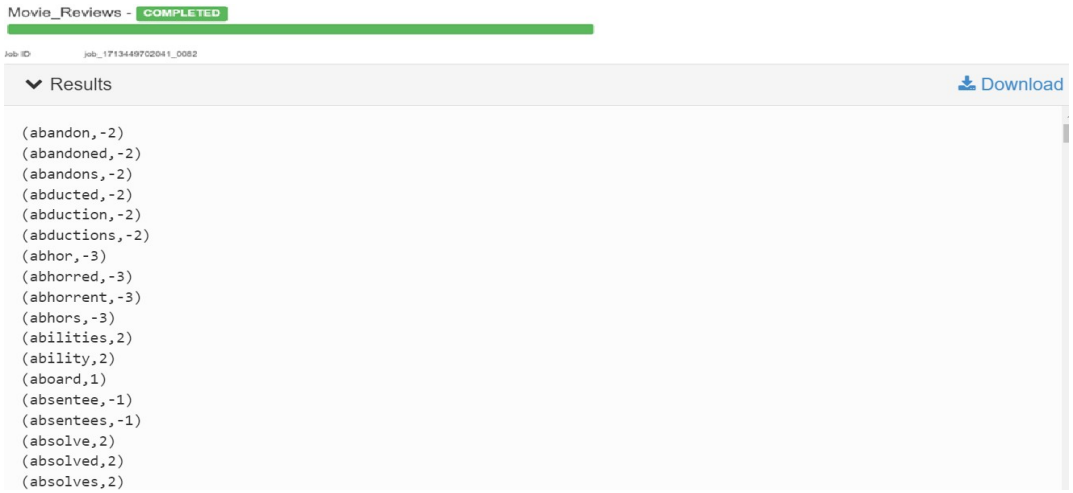


Figure 4.7 Sentiment Ratings for words in dictionary

4.1.6 JOIN Statement for Joining Tokens and Dictionary:

It will perform a left outer join between the 'tokens' and 'dictionary' relations on the 'words' and 'word' columns respectively, creating a new relation called 'word_ratings', which combines the tokenized words from the 'tokens' relation with their corresponding sentiment ratings from the 'dictionary' relation.

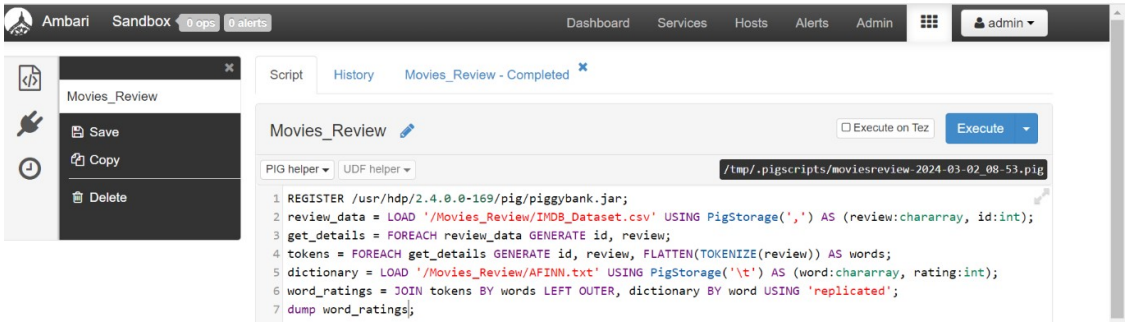


Figure 4.8 Applying JOIN statement

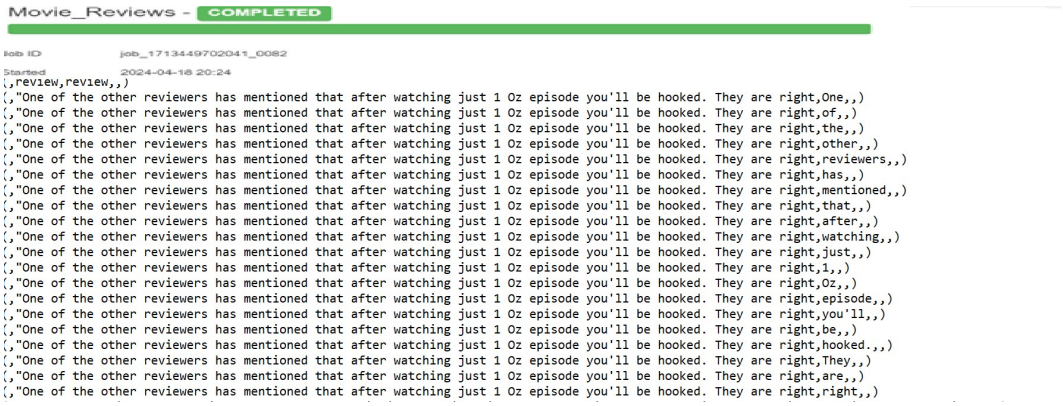


Figure 4.9 Result of JOIN statement

4.1.7 FOREACH Statement for Generating Ratings:

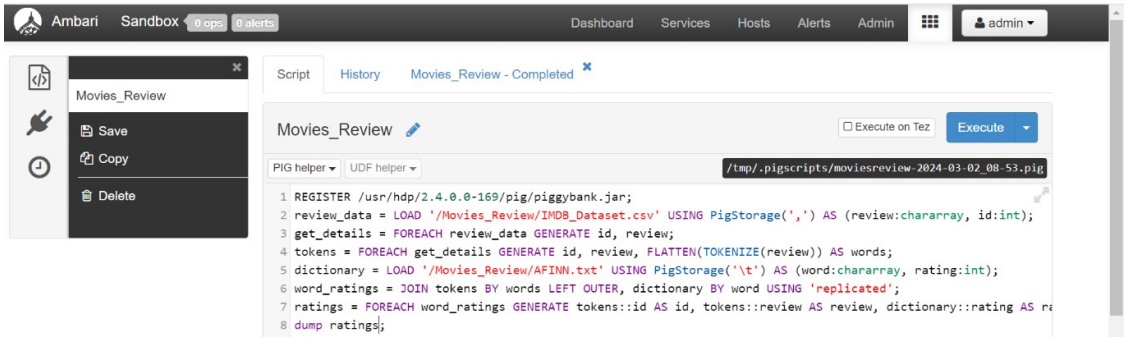


Figure 4.10 Applying FOREACH statement

It will extract the 'id' and 'review' columns from the 'word_ratings' relation and assigning the 'rating' column from the 'dictionary' relation to the 'rating' column in 'ratings'.



Figure 4.11 Result of FOREACH statement

4.1.8 GROUP Statement for Grouping Ratings:



Figure 4.12 Applying GROUP statement

Here it groups the tuples in the 'ratings' relation based on the 'id' and 'review' columns, creating a new relation called 'group_words'.

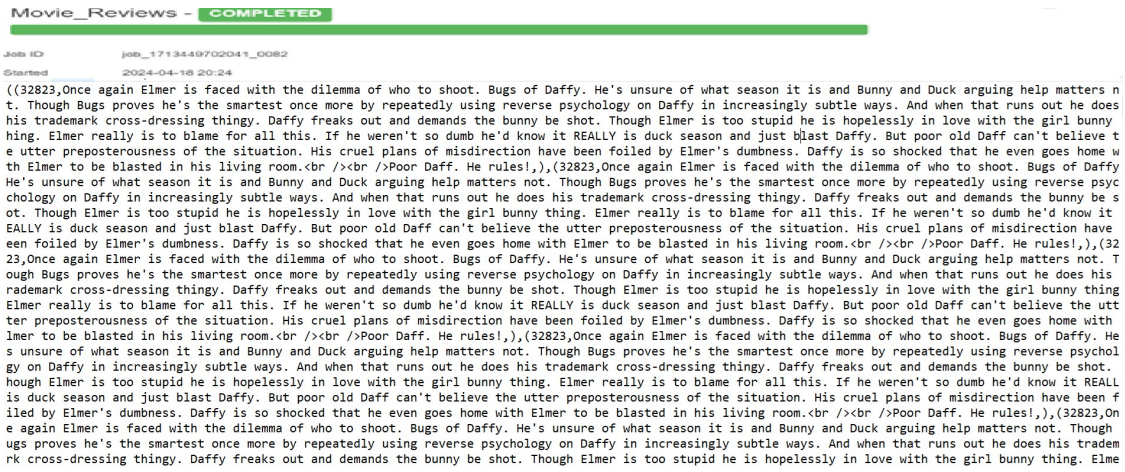


Figure 4.13 Result of GROUP statement

4.1.9 FOREACH Statement for Calculating Average Ratings:

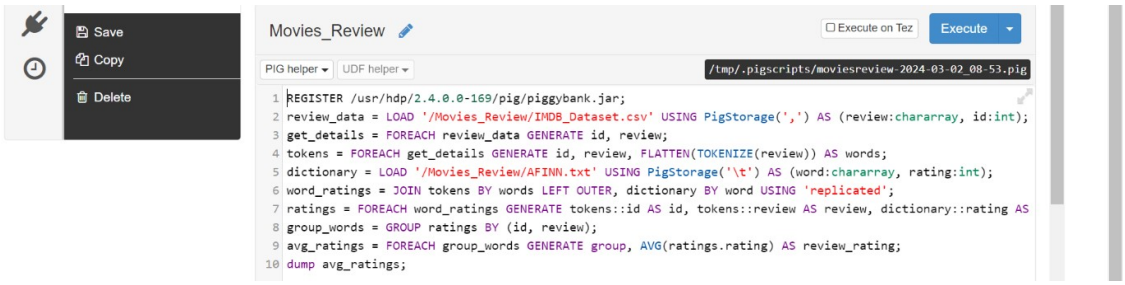


Figure 4.14 Calculating Average Ratings

Then it calculates the average rating for each group of 'id' and 'review' in the 'group_words' relation. The result is stored in the 'avg_ratings' relation, which includes the 'group' column (containing the 'id' and 'review' values) and the calculated average rating as 'review_rating'.

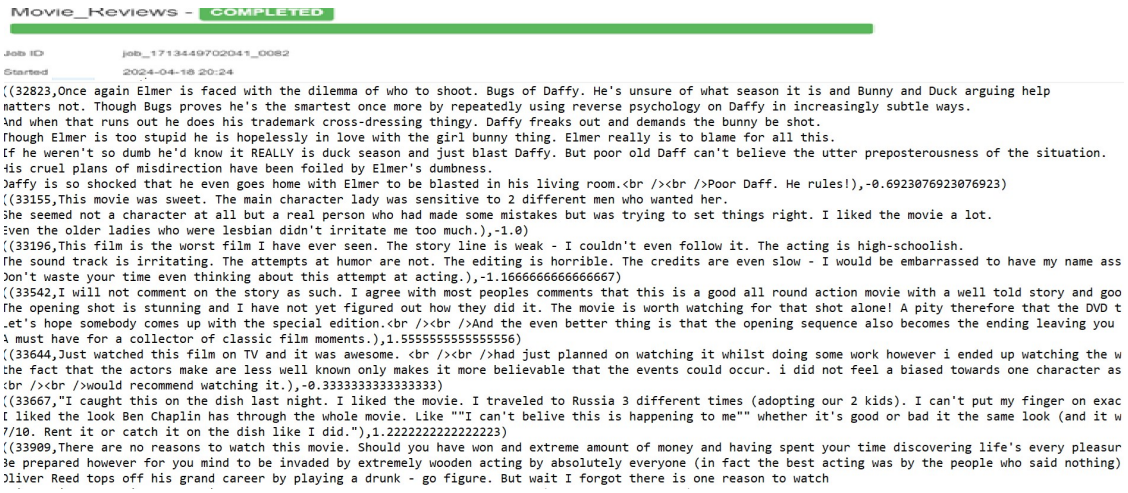


Figure 4.15 Calculated Average Ratings

4.1.10 FILTER Statements for Positive and Negative Reviews:

Positive Reviews:

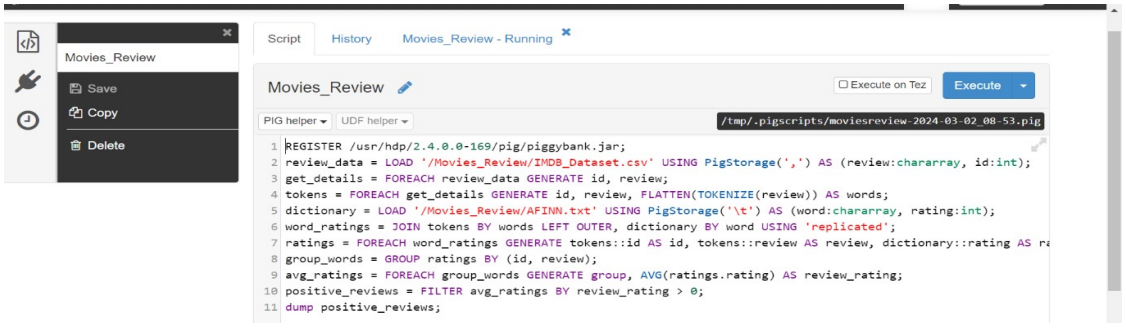


Figure 4.16 Applying Filter on Positive Reviews

Then it will filters the 'avg_ratings' relation to keep only the tuples where the 'review_rating' is greater than 0, indicating positive reviews.



Figure 4.17 Filtered Positive Reviews

Negative Reviews:

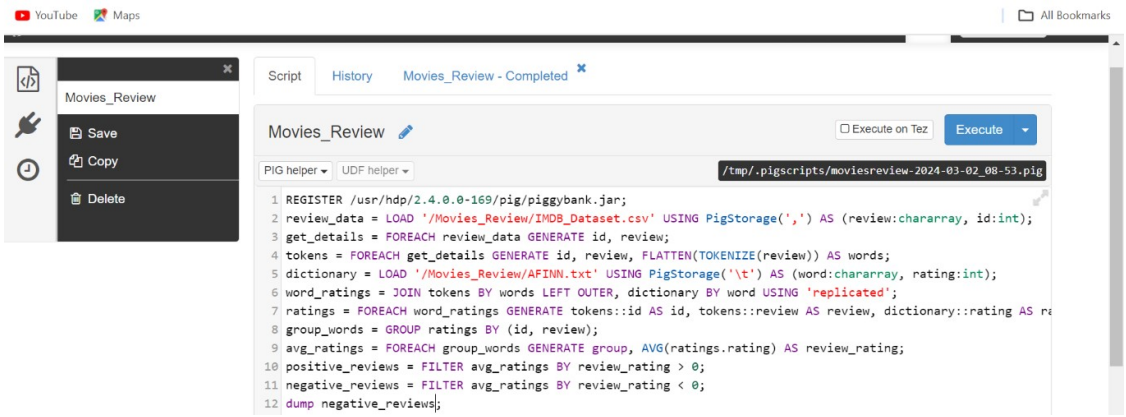


Figure 4.18 Applying Filter on Negative Reviews

Similarly, it will filters the 'avg_ratings' relation to keep only the tuples where the 'review_rating' is less than 0, indicating negative reviews.



Figure 4.19 Filtered Negative Reviews

4.1.11 STORE Statement for Storing the Results:

Finally, it stores the 'positive_reviews' and 'negative_reviews' relation into the HDFS at the specified path, 'hdfs:///user/admin//positive_reviewss', and 'hdfs: /// user / admin // negative reviewss' respectively, using a comma as the delimiter.

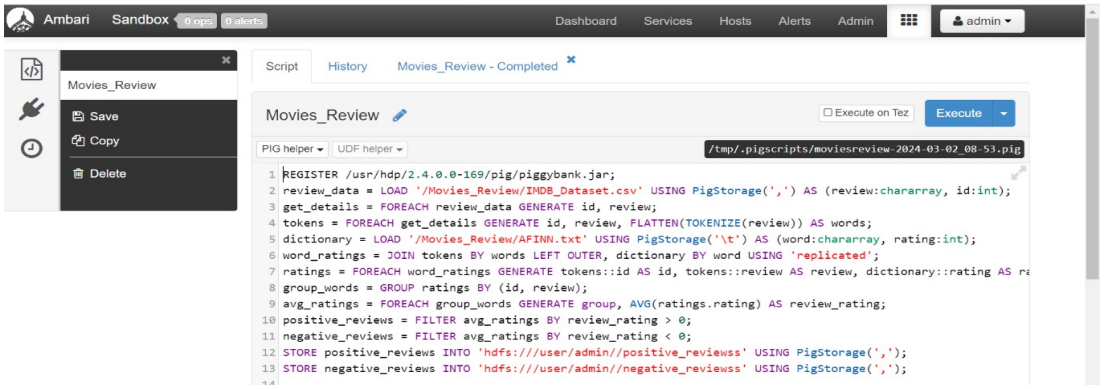


Figure 4.20 Applying STORE statement

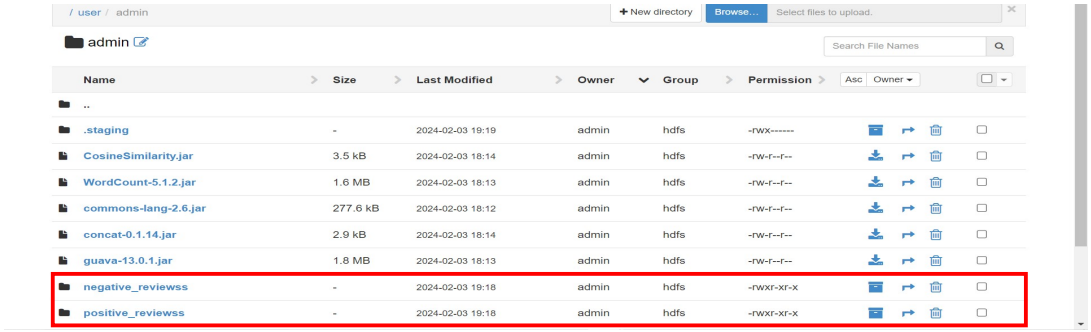


Figure 4.21 Storing the reviews

4.2 ANALYSIS OF MOVIE RECOMMENDATION SYSTEM ON APACHE PIG

4.2.1 LOAD Statement for Movie Recommendation Data:

data = LOAD '/Movie_Recommendation/movies.csv' USING PigStorage(',') AS (…): First it loads the data from the 'movies.csv' file using the PigStorage function. It specifies that the file is comma-separated and assigns names to the columns based on the provided schema.

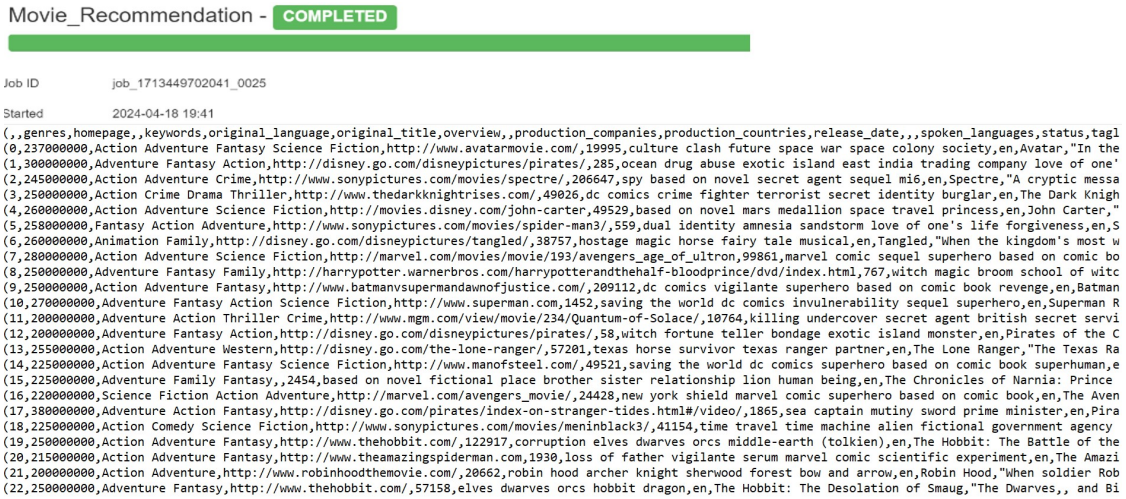


Figure 4.22 Movie metadata

4.2.2 FOREACH Statement for Getting Details:

each_movies = FOREACH data GENERATE index, genres, keywords, original_title, overview, release_date, vote_average;

It will select specific columns from the loaded data and create a new relation called 'each_movies' with only those columns.

4.2.3 Extract Only Comedy movies:

filterComedy = FILTER each_movies BY LOWER(genres) MATCHES '.*(comedy).*' OR LOWER(original_title) MATCHES '.*(comedy).*';

It will filter the 'each_movies' relation to include only rows where the 'genres' column or the 'original_title' column contains the word 'comedy' (case-insensitive).

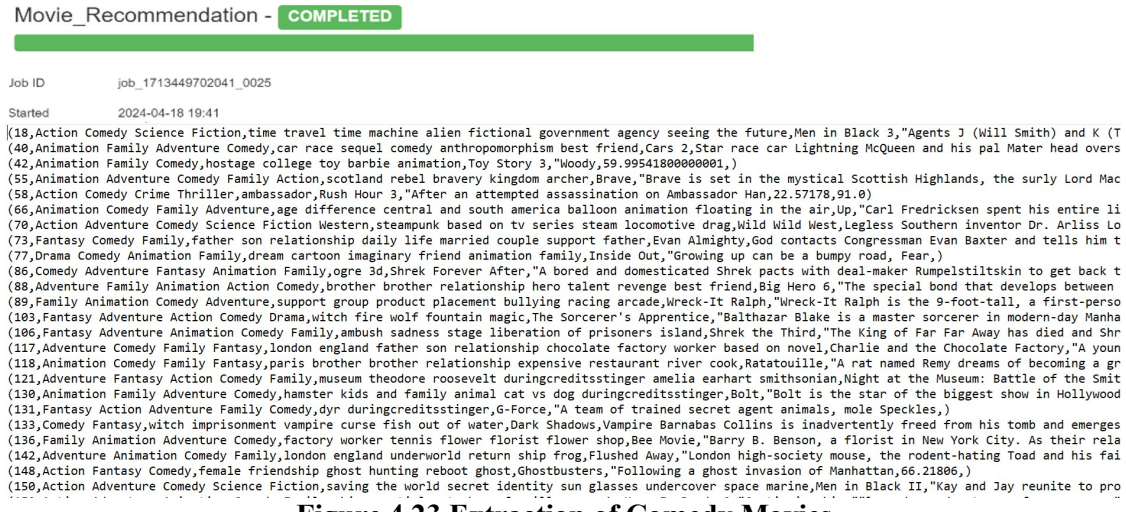


Figure 4.23 Extraction of Comedy Movies

comedyMovies = FOREACH filterComedy GENERATE original_title, genres, vote_average;
It will select specific columns from the filtered 'filterComedy' relation and creates a new relation called 'comedyMovies' with only those columns.



Figure 4.24 FOREACH statement on Comedy Movies

orderedComedyMovies = ORDER comedyMovies BY vote_average DESC;
Here it will orders the 'comedyMovies' relation based on the 'vote_average' column in descending order.

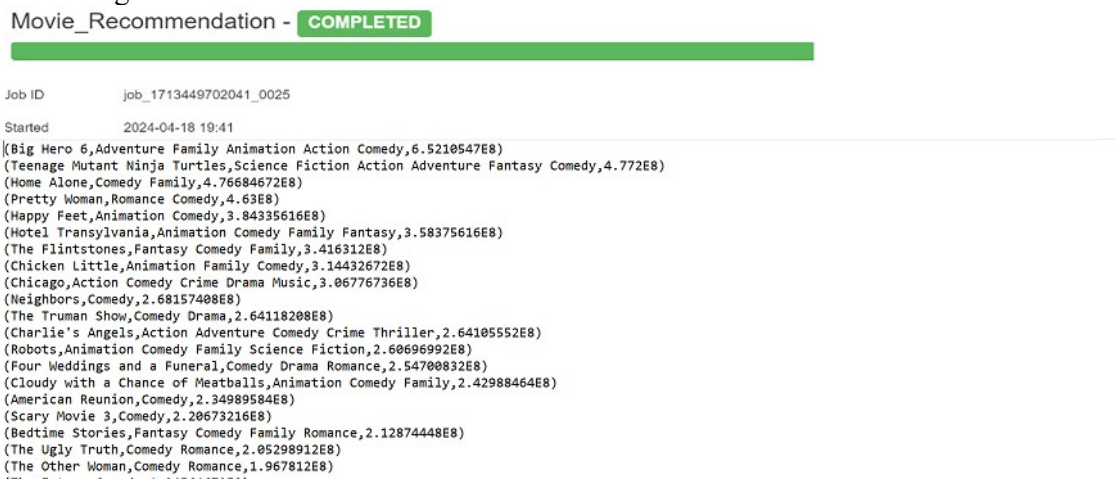


Figure 4.25 Sorting of Movies in Descending Order

topComedyMovies = LIMIT orderedComedyMovies 10;

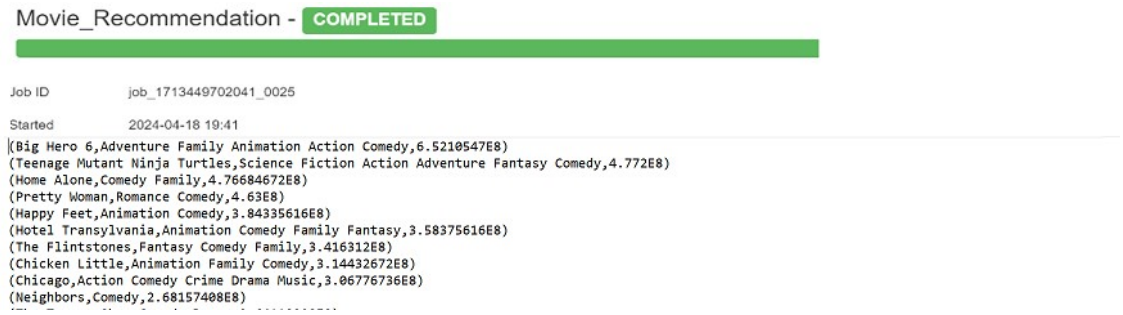


Figure 4.26 Top 10 movies using LIMIT function

It will limit the 'orderedComedyMovies' relation to the top 10 rows.
topComedyMoviesWithTitle = FOREACH topComedyMovies GENERATE original_title, genres, vote_average;
Here it selects specific columns from the 'topComedyMovies' relation and creates a new relation called 'topComedyMoviesWithTitle' with only those columns.

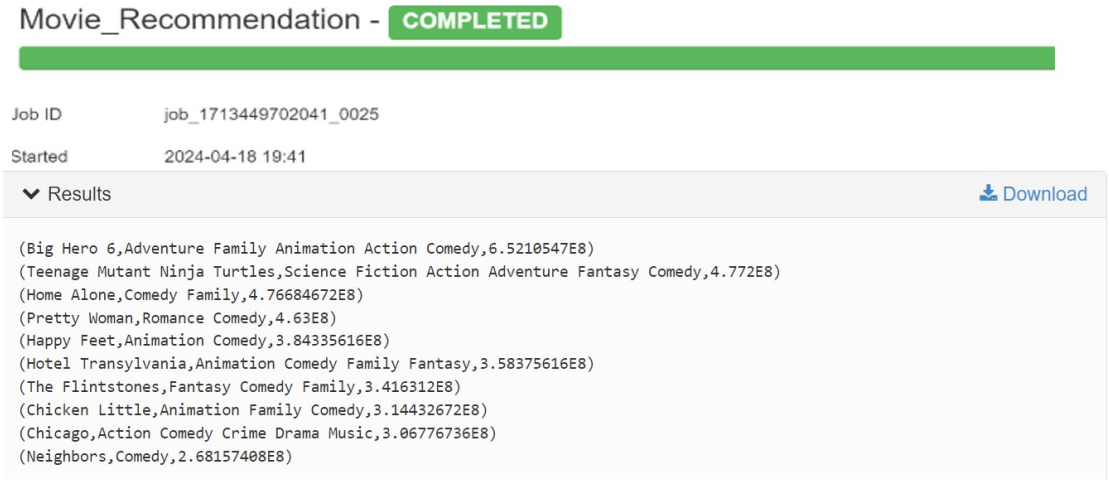


Figure 4.27 Result of SORT function

STORE topComedyMoviesWithTitle INTO 'hdfs:///user/admin/topComedyMoviesWithTitle' USING PigStorage(',');

In this command it will stores the 'topComedyMoviesWithTitle' relation into the HDFS directory '/user/admin/topComedyMoviesWithTitle' using the PigStorage function with comma as the delimiter. The remaining parts of the script follow a similar structure, performing similar operations, but for different genres such as adventures, action, thriller/mystery, drama, horror, and crime movies. Each section filters, orders, limits, and stores the top 10 movies in their respective genres.

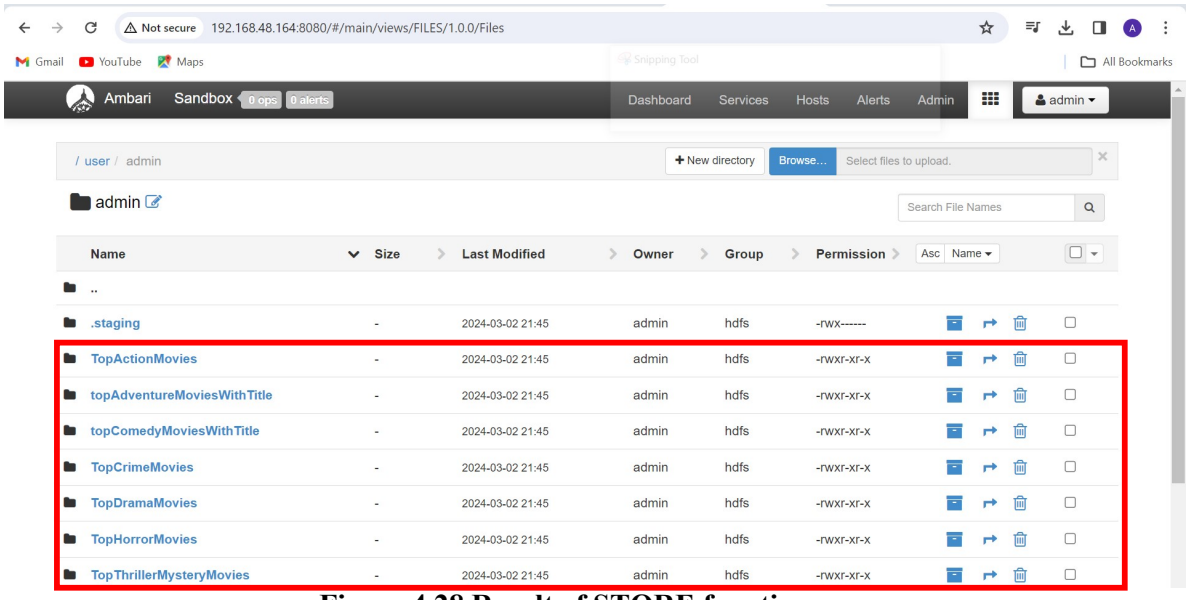


Figure 4.28 Result of STORE function

4.2.4 Extract only Adventures movies:

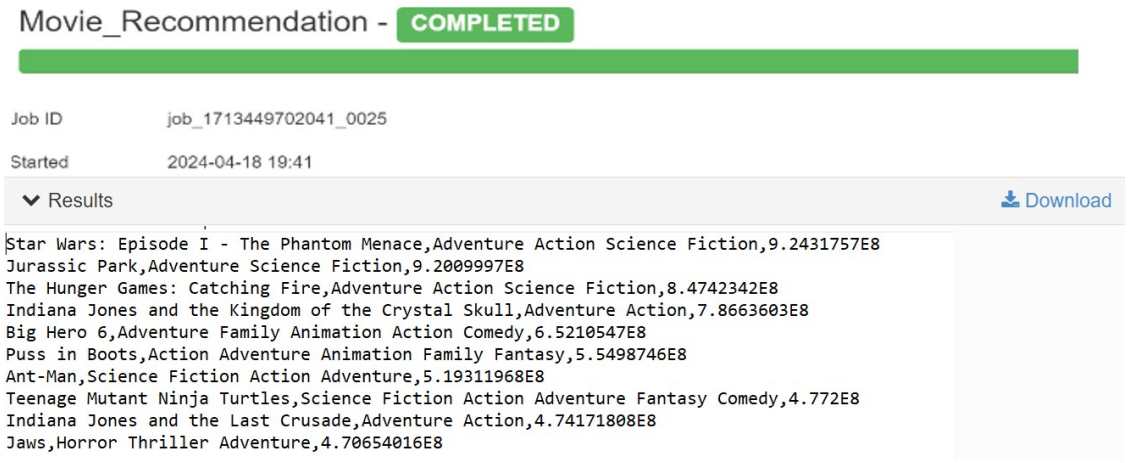


Figure 4.29 Extraction of Adventure movies

4.2.5 Extract only crime movies:



Figure 4.30 Extraction of Crime movies

4.2.6 Extract only Action movies:

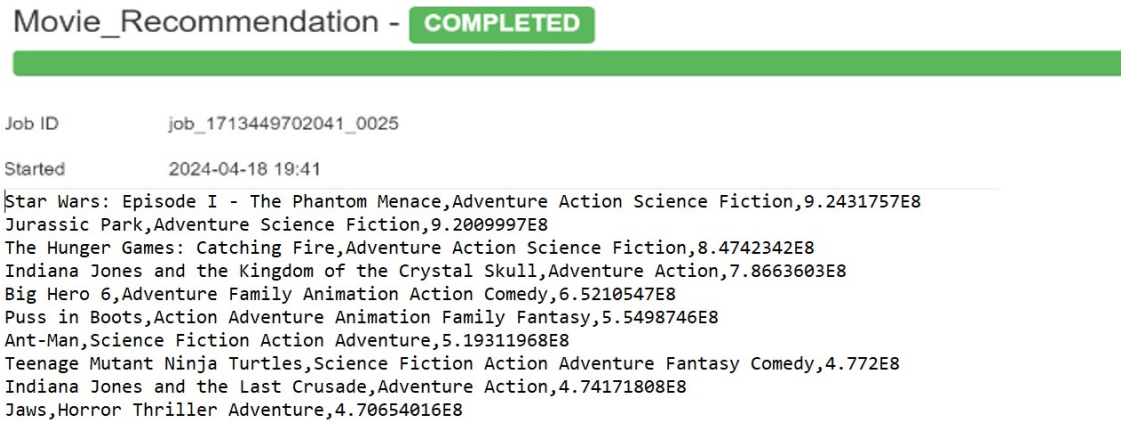


Figure 4.31 Extraction of Action movies

4.2.7 Extract only dramatic movies:

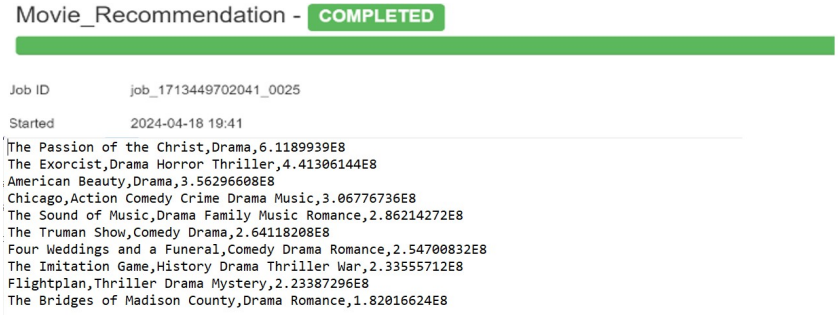


Figure 4.32 Extraction of Dramatic movies

4.2.8 Extract only Horror movies:

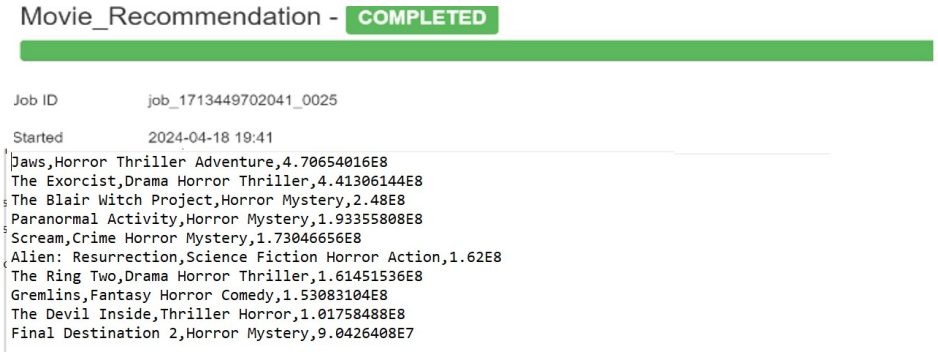


Figure 4.33 Extraction of Horror movies

4.2.9 Extract only Thriller movies:

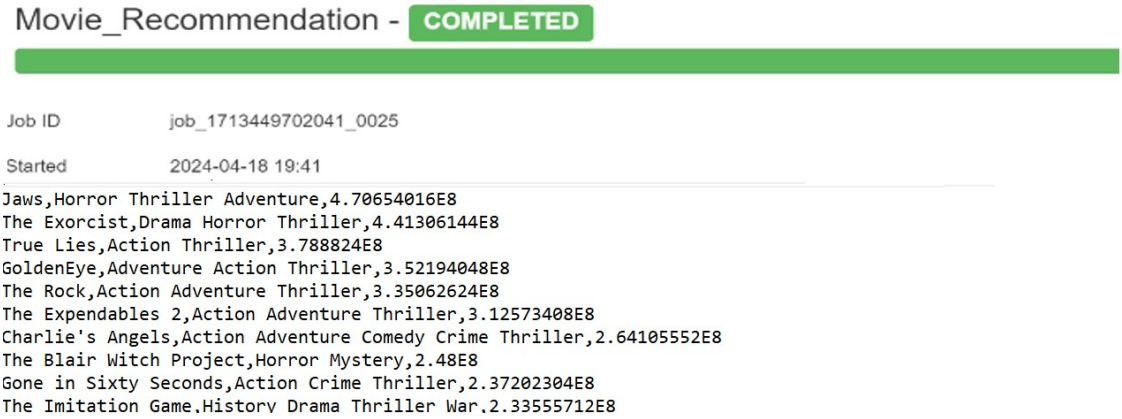


Figure 4.34 Extraction of Thriller movies

4.3 MOVIE RECOMMENDATION SYSTEM ON MACHINE LEARNING

The movie recommendation system based on content analysis has demonstrated promising results in delivering personalized movie recommendations to users. By leveraging various input data, such as the user's historical movie ratings and their viewing history, the system effectively processes and analyzes patterns and preferences specific to each user.

The system begins by examining the characteristics of movies, such as genre, plot, cast, and director. These movie attributes are transformed into feature vectors, which represent the quantitative representation of each movie's attributes. This transformation allows for easier comparison and analysis of the movies based on their content.

```
In [13]: print(feature_vectors)

  (0, 2432)     0.17272411194153
  (0, 7755)     0.1128035714854756
  (0, 13024)    0.1942362060108871
  (0, 10229)    0.16058685400095302
  (0, 8756)     0.22709015857011816
  (0, 14608)    0.15150672398763912
  (0, 16668)    0.19843263965100372
  (0, 14064)    0.20596090415084142
  (0, 13319)    0.2177470539412484
  (0, 17290)    0.20197912553916567
  (0, 17007)    0.23643326319898797
  (0, 13349)    0.15021264094167086
  (0, 11503)    0.27211310056983656
  (0, 11192)    0.09049319826481456
  (0, 16998)    0.1282126322850579
  (0, 15261)    0.07095833561276566
  (0, 4945)     0.24025852494110758
  (0, 14271)    0.21392179219912877
  (0, 3225)     0.24960162956997736
  (0, 16587)    0.12549432354918996
  (0, 14378)    0.33962752210959823
```

Figure 4.35 Calculating Feature Vectors

In the second step, the system calculates the cosine similarity scores between the feature vectors of each movie. The cosine similarity score measures the similarity between two feature vectors by calculating the cosine of the angle between them. A higher cosine similarity score indicates a stronger similarity between the movies.

```
In [15]: print(similarity)

[[1.         0.07219487 0.037733   ... 0.         0.         0.        ]
 [0.07219487 1.         0.03281499 ... 0.03575545 0.         0.        ]
 [0.037733   0.03281499 1.         ... 0.         0.05389661 0.        ]
 ...
 [0.         0.03575545 0.         ... 1.         0.         0.02651502]
 [0.         0.         0.05389661 ... 0.         1.         0.        ]
 [0.         0.         0.         ... 0.02651502 0.         1.        ]]
```

Figure 4.36 Calculating Similarities

The third step, model requests the user to enter the title of the movie which he would like to watch.

```
HYY!! WELCOME TO RECOMMENDATION SYSTEM
Enter the name of the Movie you want to watch: The Avengers
```

Figure 4.37 Prompt to enter Movie Title

Based on the input title, the system calculates the cosine similarity scores between the user's entered movie and all other movies in the dataset. It then displays these scores, along with the corresponding indices or identifiers of the movies. This step allows users to gain insights into the degree of similarity between the input movie and other films, facilitating a better understanding of the relationships and connections among different movies.

```
print(sorted_similar_movies)
```

```
[(16, 0.9999999999999999), (7, 0.7860896884964055), (85, 0.4301532854607663), (26, 0.40469694076000009), (79, 0.39336533630612
58), (126, 0.312803363203424), (511, 0.2892396085894662), (174, 0.28862963963238886), (64, 0.27464767891998554), (182, 0.2727
6411277152907), (129, 0.26928516313639983), (203, 0.25660973492915823), (33, 0.2505154038738609), (788, 0.24366990820486284),
(46, 0.24006691472148445), (169, 0.23702484619175357), (38, 0.2338952010106937), (4759, 0.23084821219774054), (68, 0.22704403
782296803), (31, 0.21487810271500662), (14, 0.1837336996264891), (782, 0.1825529075668662), (870, 0.1821588304668644), (101,
0.18018869067098506), (94, 0.16311290114551133), (9, 0.16210160555183875), (1294, 0.15229231204591692), (1192, 0.151759952646
9107), (241, 0.15100480467438165), (4401, 0.1480526961044298), (2197, 0.14650796705575336), (1720, 0.14569980465005258), (71
3, 0.1452783854259603), (1740, 0.14444382118424323), (122, 0.14409292201469673), (242, 0.1411718728209748), (700, 0.138691786
13924196), (421, 0.1368353792924146), (2390, 0.1322595220403441), (1382, 0.13098072544431874), (1001, 0.12944550202615981),
(163, 0.1288465026382687), (1177, 0.1272425593788525), (615, 0.12654048024367776), (2235, 0.12609039014244472), (545, 0.12378
867296576604), (4129, 0.12228540013864365), (1445, 0.12193133980142785), (271, 0.12140111037132327), (232, 0.1207447485198007
6), (1569, 0.12035659926302612), (78, 0.12014258310331971), (668, 0.11850432643557361), (254, 0.11606677472916561), (30, 0.11
350883208096145), (2714, 0.1128632945861024), (20, 0.11215733913329524), (310, 0.11168549404492704), (356, 0.1101851826656085
5), (1230, 0.10985219336743686), (2086, 0.1095184231484171), (1277, 0.10854190298956724), (4033, 0.10612239382703968), (4440,
0.1060206385570333), (1578, 0.10567800985452946), (3286, 0.10562012273253656), (183, 0.10377001356824804), (954, 0.1006739140
2525636), (4685, 0.09858773208659986), (1651, 0.09844531813706475), (805, 0.09511435088009323), (792, 0.09337750567835211),
(774, 0.09310441321456729), (1335, 0.09235732956258072), (778, 0.09193829719770913), (439, 0.09187374873136556), (205, 0.0915
916443111956), (3397, 0.09074175752917837), (80, 0.0904665829737873), (882, 0.09036493641073211), (1997, 0.0902835548092687
2), (2931, 0.09022929970851917), (1398, 0.09021175031061483), (2880, 0.09007583630142246), (1168, 0.08871950829962877), (215,
```

Figure 4.38 Sorted Similar Movies

Then system sorts the movies based on their cosine similarity scores in descending order. By sorting the movies, the system identifies the most similar movies and ranks them accordingly. The higher a movie's similarity score, the more relevant it is in relation to the user's input or preferences.

```
print(similarity_score)
```

```
[(0, 0.03426340578061641), (1, 0.018757441743698592), (2, 0.03907993864609115), (3, 0.0066022243575397788), (4, 0.057215392028
04143), (5, 0.014194201691251133), (6, 0.02358797178187124), (7, 0.7860896884964055), (8, 0.008045023221038158), (9, 0.162101
60555183875), (10, 0.07691564861356003), (11, 0.01217213686353538), (12, 0.013990444052447457), (13, 0.012631427434045384),
(14, 0.1837336996264891), (15, 0.04563629684495581), (16, 0.9999999999999999), (17, 0.013383483902995971), (18, 0.02529414696
558761), (19, 0.01188236224021718), (20, 0.11215733913329524), (21, 0.027050077660405823), (22, 0.007034788718083841), (23,
0.036653962075193565), (24, 0.01292652624563084), (25, 0.010727286350311038), (26, 0.4046969407600009), (27, 0.03121225816313
765), (28, 0.06887119248461429), (29, 0.025483413379835257), (30, 0.11350883208096145), (31, 0.21487810271500662), (32, 0.029
868095335032138), (33, 0.2505154038738609), (34, 0.0), (35, 0.036079460398325214), (36, 0.05009193086600237), (37, 0.00818989
0844817246), (38, 0.2338952010106937), (39, 0.032268010883379605), (40, 0.008252843612503852), (41, 0.08372421650873615), (4
2, 0.0), (43, 0.026910814957550926), (44, 0.006373587850243624), (45, 0.024016228413213332), (46, 0.24006691472148445), (47,
0.07624326610444224), (48, 0.03913748248706997), (49, 0.020497347421914745), (50, 0.013246547557517922), (51, 0.0281256367332
4002), (52, 0.03095212438235573), (53, 0.02291820224070657), (54, 0.007062117885792904), (55, 0.014414813493096578), (56, 0.07
723929324352055), (57, 0.008662387371797685), (58, 0.05793346052092418), (59, 0.0323599564922284), (60, 0.03966950422705946),
(61, 0.03416366267861242), (62, 0.013566646668548385), (63, 0.02983429961672506), (64, 0.27464767891998554), (65, 0.006029278
836837788), (66, 0.0073647540809259666), (67, 0.029266346030766055), (68, 0.22704403782296803), (69, 0.02240800226598572), (7
0, 0.07292475904015482), (71, 0.03840807152716896), (72, 0.033244687778485105), (73, 0.0), (74, 0.02659274258979185), (75, 0.
027652946836421284), (76, 0.03454845505224777), (77, 0.0), (78, 0.12014258310331971), (79, 0.3933653363061258), (80, 0.090466
5829737873), (81, 0.039896818751218664), (82, 0.027914510922315176), (83, 0.03528302106688945), (84, 0.03100114959160097), (8
5, 0.4301532854607663), (86, 0.006648679839665979), (87, 0.027887327121866333), (88, 0.047267094046577945), (89, 0.0066790429
```

Figure 4.39 Calculating Similarity score based on searched movie

In the last step the system recommends the most relevant movies to the user by selecting a certain number of movies from the sorted list. The number of recommendations can be adjusted based on system settings. The recommended movies are those with the highest cosine similarity scores, indicating a strong similarity in terms of genre, plot, cast, and director.

```
MOVIE SUGGESTED FOR YOU ARE :

1 . The Avengers
2 . Avengers: Age of Ultron
3 . Captain America: The Winter Soldier
4 . Captain America: Civil War
5 . Iron Man 2
6 . Thor: The Dark World
7 . X-Men
8 . The Incredible Hulk
9 . X-Men: Apocalypse
10 . Ant-Man
11 . Thor
12 . X2
13 . X-Men: The Last Stand
14 . Deadpool
15 . X-Men: Days of Future Past
16 . Captain America: The First Avenger
17 . The Amazing Spider-Man 2
18 . The Image Revolution
19 . Iron Man
20 . Iron Man 3
21 . Man of Steel
```

Figure 4.40 Movie Recommendations

4.3.1 Result Analysis:

The output of the recommendation system is shown in the figure below. When a user searches for a specific movie, the system calculates the similarity between the searched movie and other movies in its database, considering factors like genre, director, actors, and plot keywords. It then generates a ranked list of recommended movies that are most similar to the searched movie, accompanied by relevant details such as title, genre, release year, and a synopsis. The system also takes into account the user's preferences and viewing history to further personalize the recommendations. The aim is to provide users with tailored suggestions that match their tastes and allow them to discover new movies aligned with their interests.

4.4 REVIEW PREDICTION SYSTEM ON MACHINE LEARNING

The result of sentiment analysis accurately predicts the sentiment expressed in user reviews of movies. The model successfully identifies and classifies the sentiment as positive, negative, or neutral. It basically demonstrates the model's ability to understand and interpret the emotions conveyed in the reviews.

4.4.1 User Input for Movie Review:

Here is how the algorithm takes first input from the user for movie review. It asks the user to enter a review for a movie watched. The user can write reviews according to their choices and preferences in the given box. They are able to compose reviews of any length, from a few phrases to an in-depth analysis of the film.

```
In [*]: if __name__ == "__main__":
            main()

        Welcome to the Movie Review Sentiment Predictor!
        Please enter your review about the movie:
```

Figure 4.41 Input User Review

The algorithm could carry out input validation, depending on how it is implemented, to make sure the user has typed a review before moving forward. It could verify that the input is not empty or satisfies specific length specifications. If the input is incorrect, the user can receive the necessary feedback, which will encourage them to submit a legitimate review.

4.4.2 Positive Sentiment Prediction:

Once the user submits a movie review, the sentiment analysis algorithm processes the input using the trained model. It maintains a dictionary of commonly occurring words of English language and give scores of all the words. After this it predicts the overall sentiment as either negative or positive. Here in the output shown below, it takes the review from the user and after processing all the words it predicts the sentiment as positive.

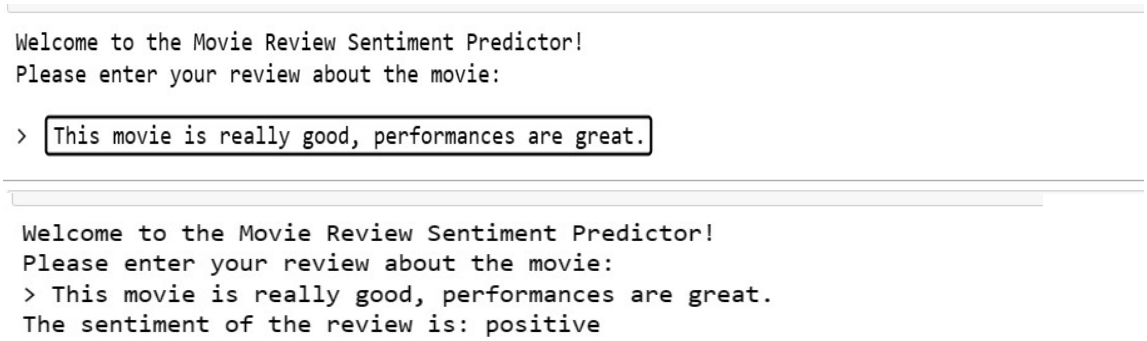


Figure 4.42 Positive Review Prediction

4.4.3 Negative Sentiment Prediction:

Now, as we have written a negative review here, the algorithm calculates the scores of all the important words and then it gives the overall sentiment as negative in the output.

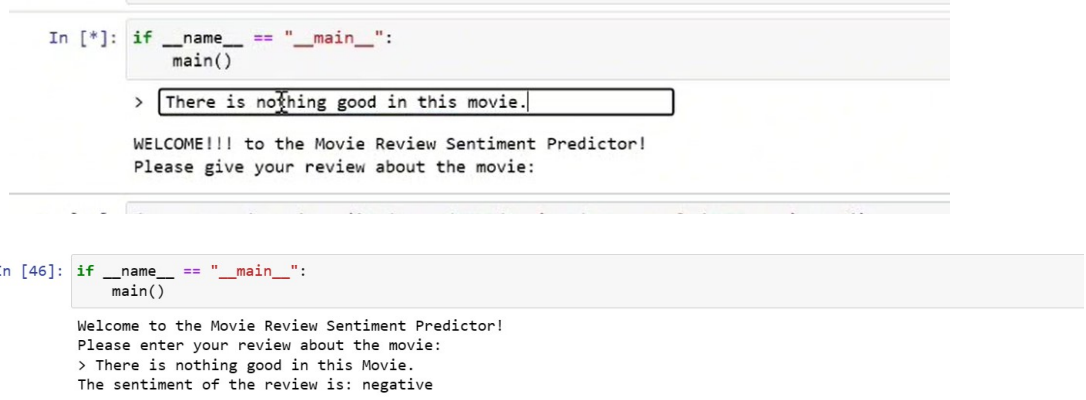


Figure 4.43 Negative Review Prediction

4.4.4 Adding Reviews to the Database:

After predicting the sentiment of the user's review, the code adds the review along with its sentiment prediction to the movie reviews database. These both reviews, along with its sentiment prediction, are successfully added to the database. This inclusion allows for comprehensive analysis and the identification of areas for improvement. By saving the user reviews in the same file as the existing reviews, the database becomes more extensive and diverse, enabling better insights and analysis.

```
Updated dataset:
                                                review sentiment
0                    There is nothing good in this Movie.  negative
1    This movie is really good, performances are gr...  positive
2    One of the other reviewers has mentioned that ...  positive
3    A wonderful little production. The filming tec...  positive
4    I thought this was a wonderful way to spend ti...  positive
..                                                 ...       ...
497  "American Nightmare" is officially tied, in my...  negative
498  First off, I have to say that I loved the book...  negative
499  This movie was extremely boring. I only laughe...  negative
500  I was disgusted by this movie. No it wasn't be...  negative
501  Such a joyous world has been created for us in...  positive

[502 rows x 2 columns]
```

Figure 4.44 Addition of Reviews in Database

4.4.5 Comparison of Models:

A. Logistic Regression

By comparing the predicted sentiment labels with the ground truth labels, a confusion matrix will be used to assess the accuracy of the sentiment analysis component and reveal information about the algorithm's performance. The formula for calculating accuracy is in equation 1.9.

$$Accuracy = \frac{(TP+TN)}{(TP+TN+FP+FN)} \quad (4.1)$$

By putting values in the above equation, the value of accuracy is yielded as 0.89 or 89% which shows a moderate level of accuracy in the system.

It is used to measure the percentage of accurately predicted positive cases among all the instances that are projected to be positive. It is calculated by using equation 1.10 for the above confusion matrix and resulted in 0.889 or 89% for the sentiment analysis review system.

$$Precision = \frac{TP}{(TP+FP)} \quad (4.2)$$

Perfomance Evaluation through Confusion Matrix: We have used the confusion matrix to access the accuracy and performance of the sentiment analysis algorithm by evaluating the distribution of prediction across different classes. In the figure below it identifies the number of correct and incorrect prediction effectively.

The confusion matrix is a table that shows how well a model's predictions align with the true labels across different sentiment categories. It enables the analysis of accuracy, precision, recall, and other performance metrics for each sentiment class. By visualizing the confusion matrix through a plot, patterns and discrepancies in the model's performance across various sentiment categories can be identified. This visualization helps gain insights into how well the model is predicting sentiments and highlights areas where improvements can be made.

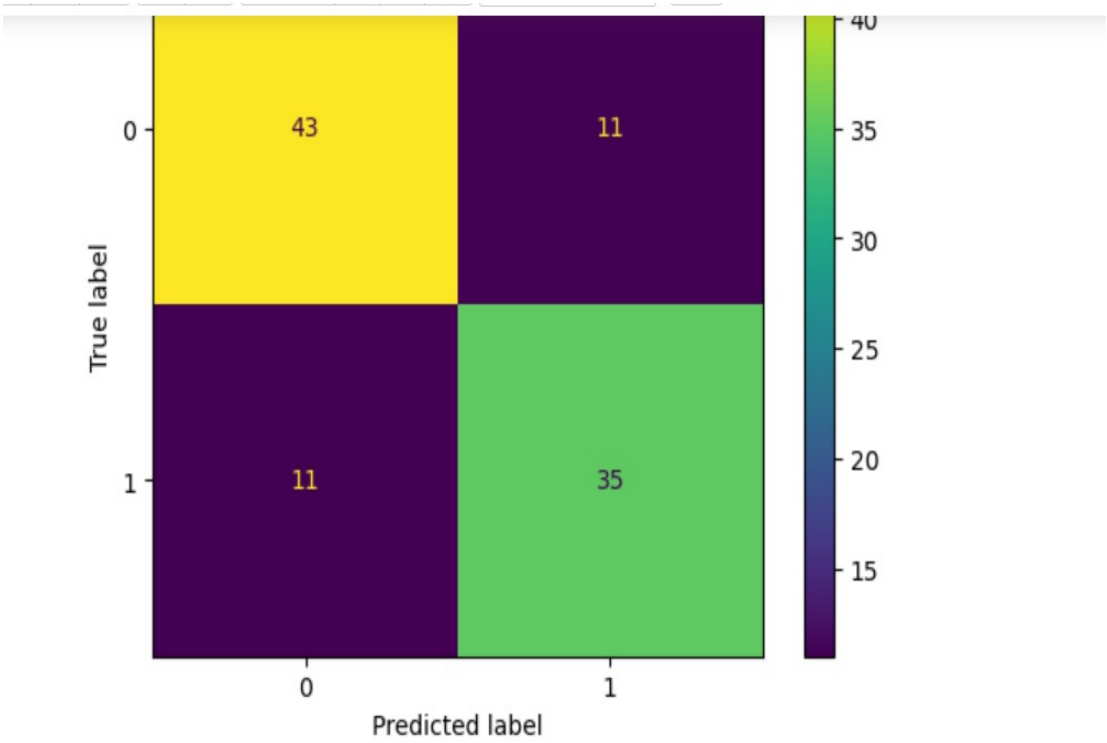


Figure 4.45 Confusion Matrix

B. Random Forest

The calculated accuracy with the help of Random Forest (RF) model is yielded as 0.86 or 86%. While the precision is also calculated as 86%.

C. Naïve Bayes

The accuracy with the help of Naïve Bayes model is yielded as 0.85 or 85% and precision is also calculated as 85%.

As accuracy and precision of Logistic Regression is greater than the other two models, it is being used as the primary model in sentiment analysis review system. Fig. 2.59 shows the comparison graph of the three models with their calculated accuracies.

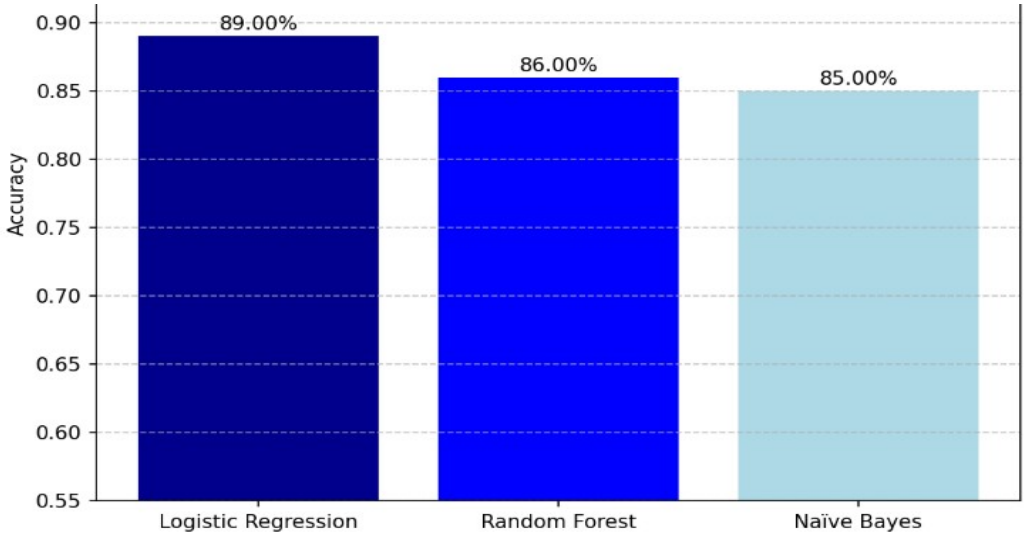


Figure 4.46 Comparison of Models

4.5 ACTOR-BASED RECOMMENDATION SYSTEM:

The recommendation system, using deep learning techniques like Convolutional Neural Networks (CNNs) for face recognition, effectively yields results in the form of movie recommendations. These recommendations are based on the user's input or preferences and aim to suggest movies with similar actors. The system presents a curated list of recommended movies that share common actors with the user's selected movie or preferred actor. The CNN-based face recognition methodology plays a vital role in accurately identifying and extracting relevant features from the faces of actors. By analyzing these features, the system determines the similarities between different actors and subsequently suggests movies that involve those actors. This approach enhances the system's ability to provide tailored recommendations that align with the user's preferences, improving the overall user experience.

4.5.1 Movie Input:

First, the system asks the user to input their favorite movie or specify the movie they want to watch. The user is prompted to enter the title of their favorite movie or the name of the movie they intend to watch. To make sure the user entered an accurate movie title, the system could carry out input validation. It might determine whether the input matches an already-existing movie or see if the movie is already in the dataset.

4.5.2 Generating Face Encodings:

In the second step, utilizing embedding techniques, the system will generate and display the face encodings of the actors in the user's favorite movie. In order to provide numerical encodings or embeddings for later processing and analysis, this stage attempts to extract face characteristics from the actors in the film.

```
In [*]: movie_title = input("Enter the movie title: ")
        Enter the movie title: The Avengers
```

```
print(Face_encoding)
```

```
[-0.07348645  0.12712842  0.06938212  0.02227501 -0.0529418  -0.08385946
 -0.11480134 -0.09105008  0.14995013 -0.02919006  0.26233697  0.0039524
 -0.20984094  0.04494866 -0.02030469  0.11319412 -0.22424707  0.00212715
 -0.07932249 -0.07922951  0.03656622  0.08524913  0.07121463  0.04756653
 -0.11662953 -0.291031   -0.08599591 -0.09445021  0.16594464 -0.0904222
 -0.02822261 -0.03653787 -0.10776613 -0.10200138  0.02462663  0.0419908
 -0.10776136 -0.11175432  0.21714491  0.08131914 -0.15595548  0.00391102
  0.01681023  0.35831106  0.20420595  0.08448698  0.05711294 -0.08228662
  0.08600908 -0.26876903  0.07241362  0.11427528  0.10797165  0.0854819
  0.12666138 -0.2284245   0.01850226  0.19389074 -0.16228312  0.10406172
  0.09035937  0.01609887  0.10004359 -0.0211318   0.27415243  0.10665333
 -0.13176568 -0.05387063  0.06119443 -0.12767154 -0.09536985  0.06090394
```

Figure 4.47 Face Encoding

4.5.3 Object Detection and Bounding Boxes:

After generating the face encodings in the previous step, the recommendation system proceeds to the next stage, where it applies object detection algorithms to draw bounding boxes around the faces of the actors. The system examines the face encodings and uses the trained object identification model to pinpoint the exact positions of the performers' faces on the movie poster. It creates bounding boxes that firmly encompass each actor's face areas. This step is crucial for precisely localizing and highlighting the facial regions of the main character within the movie poster.

4.5.4 Face Identification:

Additionally, the system displays the name of the actor associated with each bounding box, providing identification and context. The system shows the movie poster or any pertinent picture with the actors' faces surrounded by bounding boxes after object detection and identification is complete. The relevant bounding boxes are clearly identified and contextualized by the actor names that are displayed next to or below them.

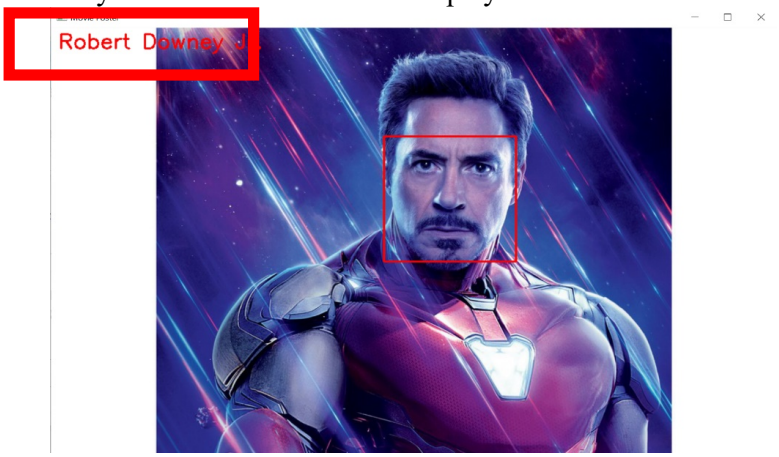


Figure 4.48 Face Detection and Encoding

4.5.5 Similarity Comparison and Resemblance Detection:

Now, based on the similarity, the recommendation system compares the face encodings of the actor from the user's favorite movie with the face encodings of all other actors in the dataset. By calculating the similarity scores or distances between the face encodings, the system identifies actors who have a close resemblance to the main actor.

```
In [12]: matching_results = find_target_face()
         recommend_matching_results(matching_results)

         Recommended movie For you are:
         The Avengers
         Iron Man
         Zodiac
```

Figure 4.49 Movie Recommendations

4.5.6 Result Evaluation:

By incorporating deep learning techniques such as face recognition, this recommendation system leverages the visual cues from movie posters and the faces of actors to provide tailored recommendations. It enhances the user experience by suggesting movies with similar actors or heroes, expanding their movie-watching options. This approach not only considers the user's favorite movie but also captures the essence of the actors' faces, resulting in more accurate and relevant recommendations. Overall, this recommendation system combines computer vision and similarity analysis to offer a comprehensive and visually informed movie recommendation process.

4.6 USER FRIENDLY INTERFACE:

The front end is made up of html and CSS and a few lines of code from JavaScript. The architecture of the website is shown in Fig. 4.50.

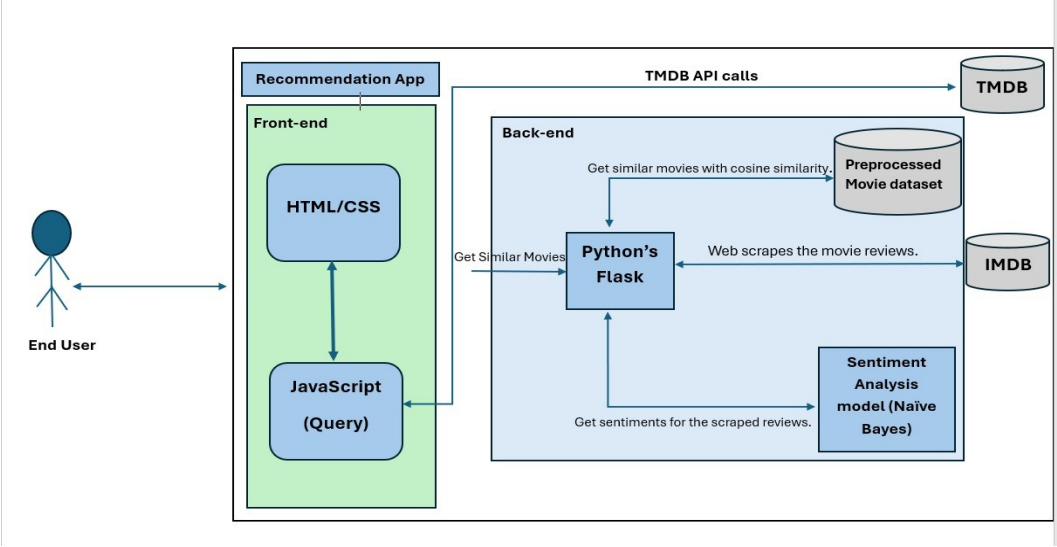


Figure 4.50 Architecture of Front-end

This is how the front or home page of the website looks like. The navigation bar consists of six links to the other pages of the webpages. From the search button, we can connect with the server webpage from where we can watch and have recommendations of user's favorite movies. Fig. 4.51 shows the front page of the website.

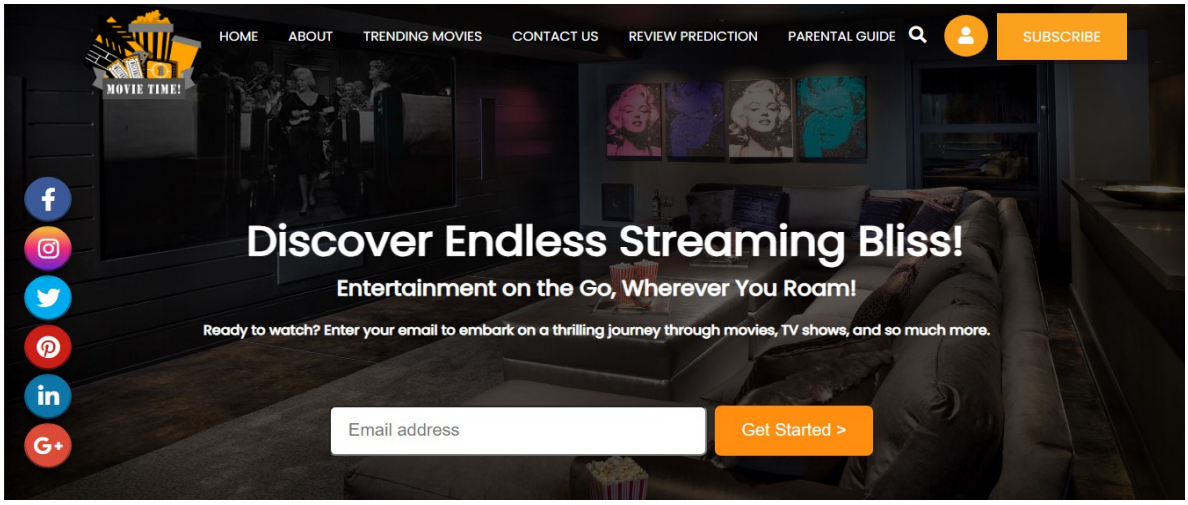


Figure 4.51 Front page of website

The recommendation server has an input field where the users can enter the movies they want to watch and by clicking the enter button, they can get the recommendations. Fig. 4.52 shows the recommendation page.

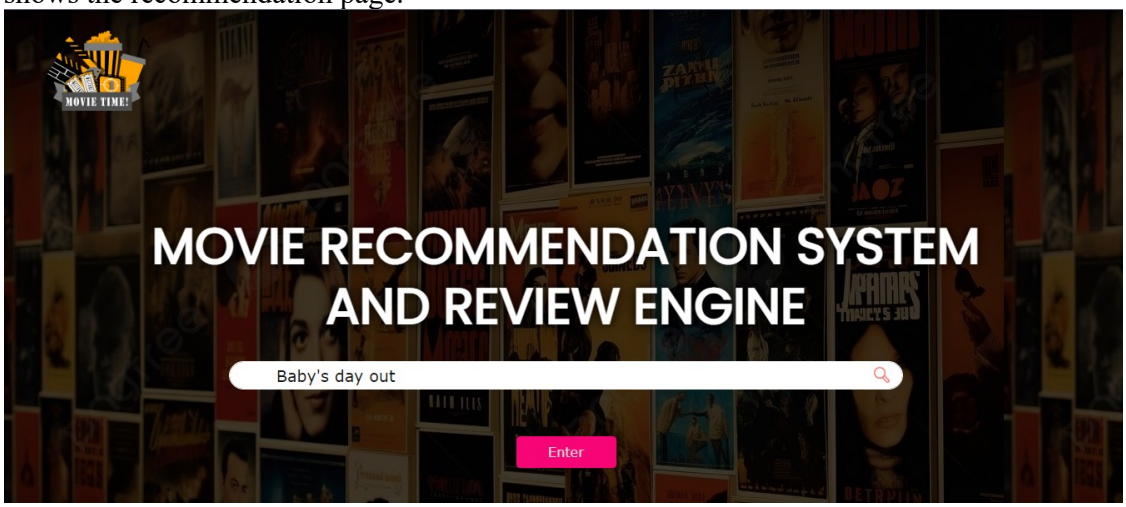


Figure 4.52 Recommendation Server

The movies with their information regarding the movie metadata including the title, runtime, overview, rating, genre, release date, and status. Below this, the main cast of the movie is also displayed. By clicking on any cast member, the information related to their birthday, place of birth and biography is displayed. Then user reviews with their sentiments are

displayed on the table. Users can get to know about the number of good and bad sentiments and can analyze if they want to watch that particular movie or not.



Figure 4.53 Movie Information



Figure 4.54 Cast and Crew Information

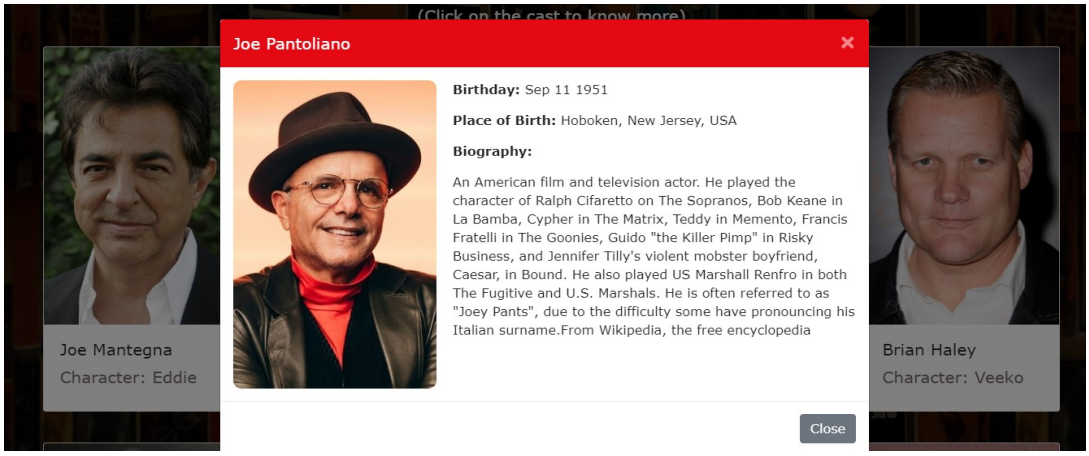


Figure 4.55 Actor's Information



Figure 4.56 Reviews and sentiments Table

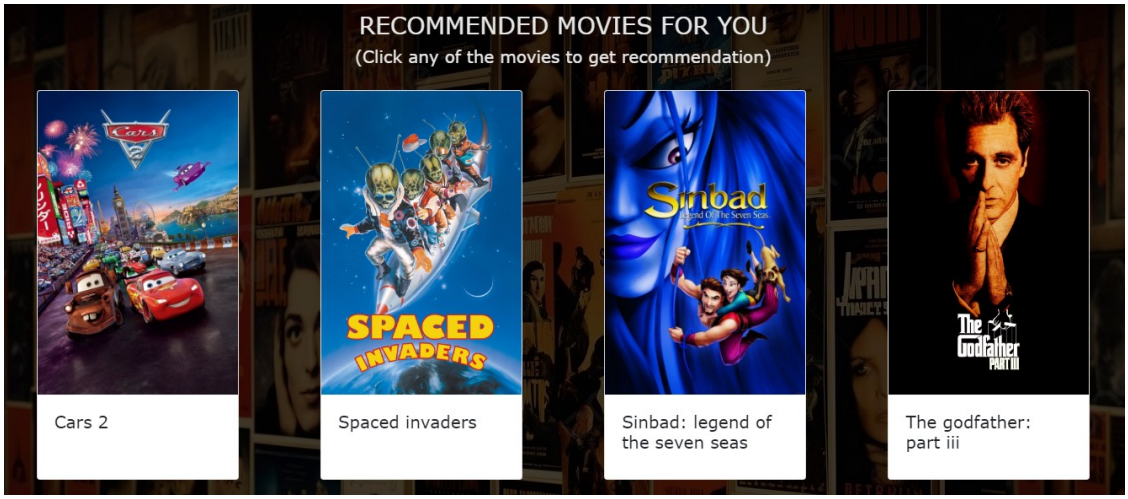


Figure 4.57 Recommended Movies

Other than this, by clicking the login button, the users are prompted to the login page which enables them to either log in or register into the website so that they can search for their respective movies. Fig. 4.58 shows the log-in page. It contains the email ID and password to enter the website. Fig. 4.59 shows the register page. The information of first name, last name, email address and password need to be entered to get registered through it.

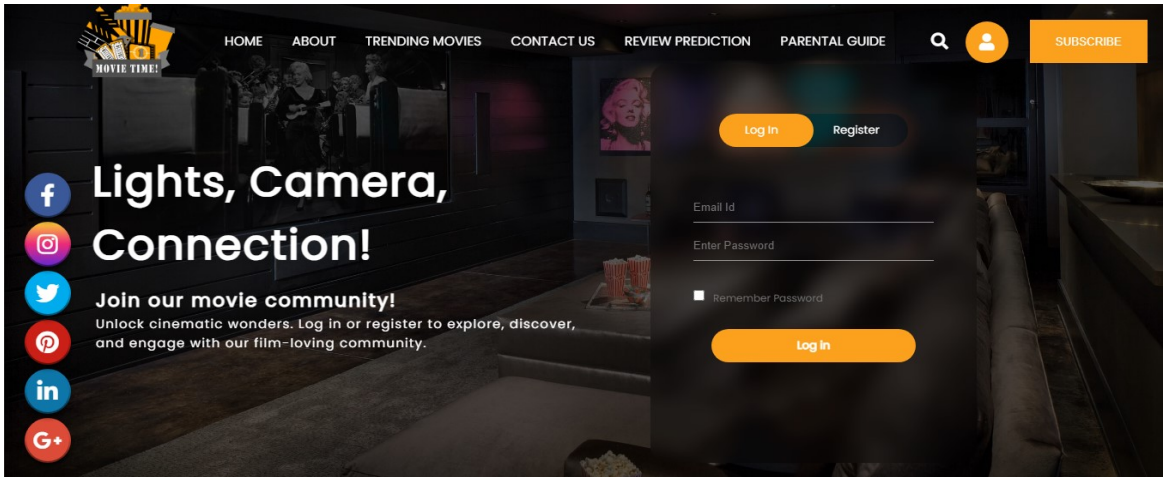


Figure 4.58 Log-In Page

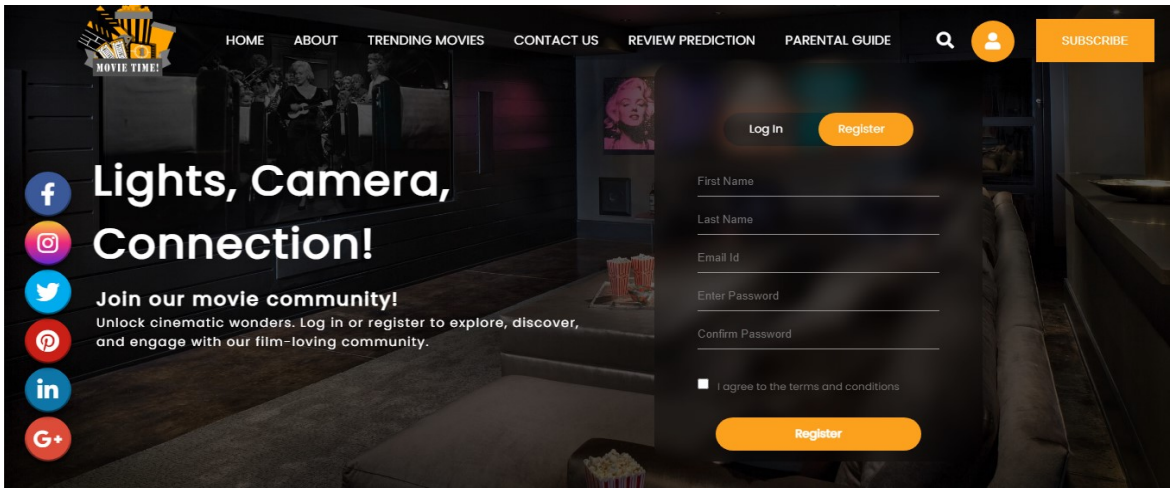


Figure 4.59 Register Page

The about us page of the website consists of two columns. The objective and value proposition of the website are briefly introduced in the left column. Two buttons are positioned in the right column to provide visitors with quick access to the suggestion and review pages. These buttons act as unambiguous navigational indicators, directing visitors to the precise areas containing well-curated suggestions and in-depth reviews. Furthermore,

social media icons have been included on the page so that readers can easily interact with the social media platforms that are linked to the website. Fig. 4.60 shows the about us page.

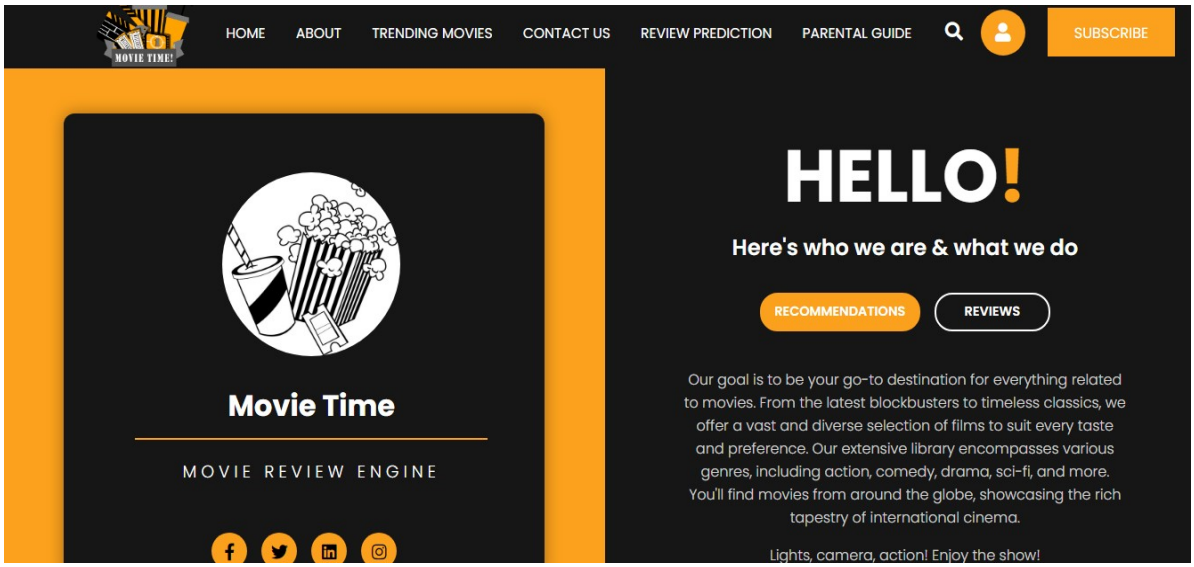


Figure 4.60 About Us Page

The Trending movies page first contains a scrolling bar of different movies on it. Beneath it there are three different categories including popular movies, applauded TV series, and upcoming movies. These consist of different trending movies of specific times. A "Watch Trailer" button appears next to every film, enabling viewers to have a sneak peek and decide whether or not to watch it. Fig. 4.61 shows the page of trending movies.

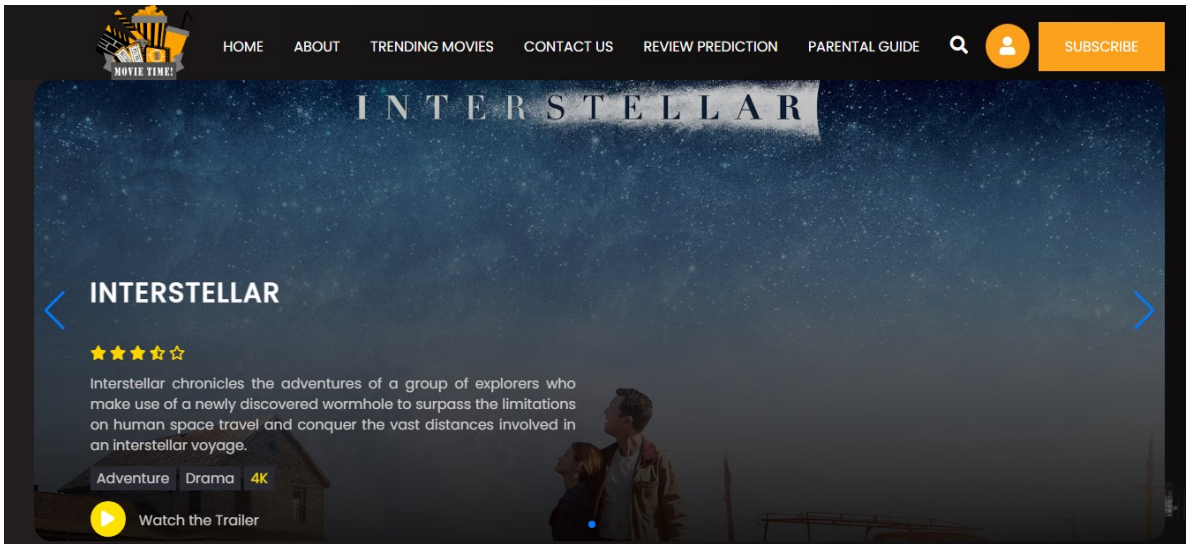


Figure 4.61 Trending Movies

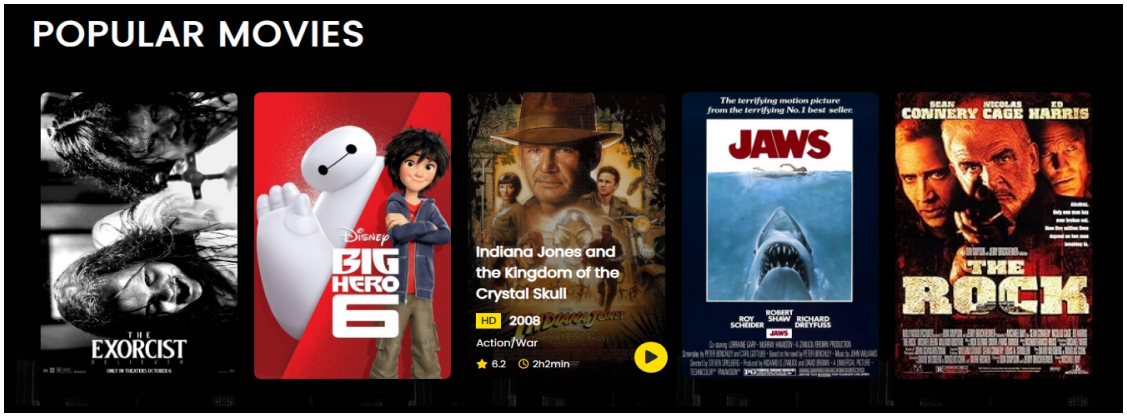


Figure 4.62 Popular Movies

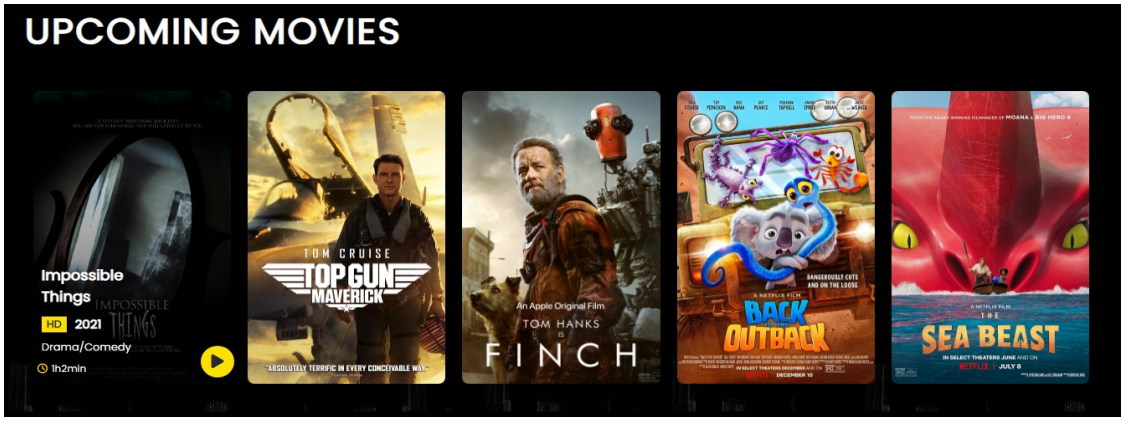


Figure 4.63 Upcoming Movies

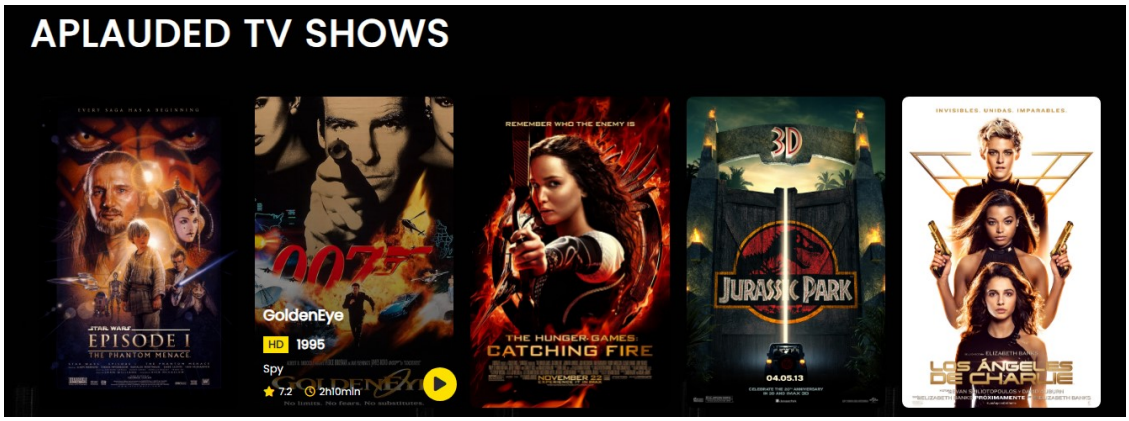


Figure 4.64 Applauded TV Shows

The Contact us page has an easy-to-use, although basic, contact form. Regardless of the queries, suggestions, or remarks, the user-friendly interface ensures a seamless and pleasant experience. For contacting to the website's admin, or having any kind of queries, the users

need to enter the username, email address, phone number and the message they want to write. Fig. 4.65 shows the contact us page of the website.

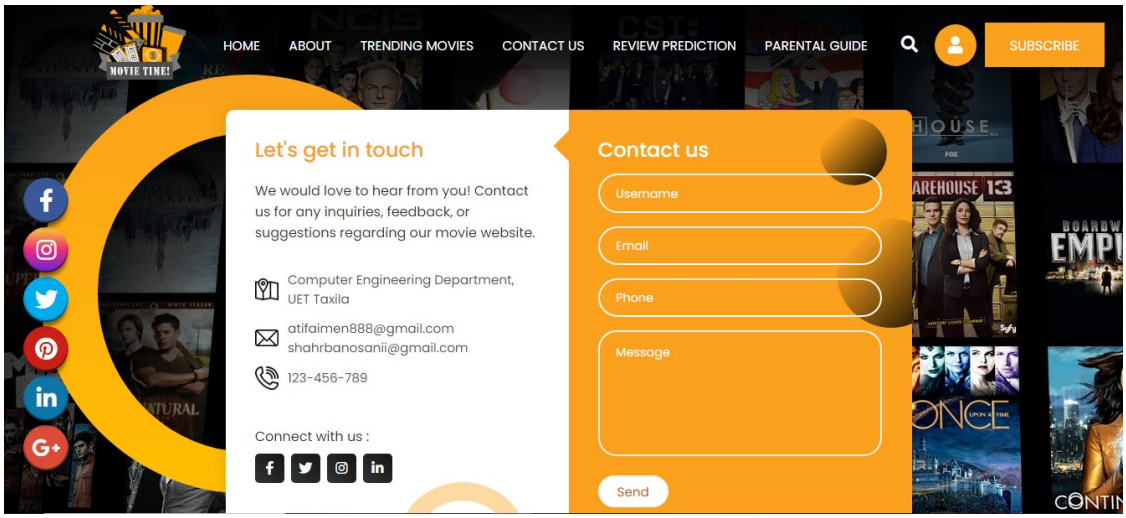


Figure 4.65 Contact Us Page

Review Prediction page consists of two things, a star-rating widget, and a movie review form. Users can submit their movie reviews using the movie review form. It has an input field, which is usually a text field that may hold several lines of text. Users can express their ideas, comments, and criticism on a film. There is a "Predict" button accessible beneath the input box. Fig. 4.66 shows the reviews prediction page of the website.

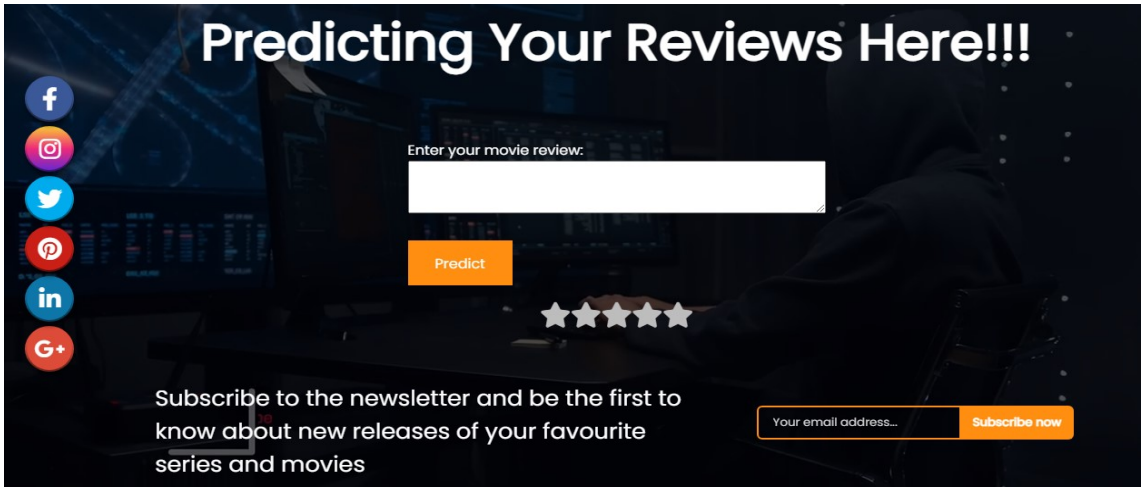


Figure 4.66 Review Prediction Page

The parental guidance page consists of the information related to the learning purpose of whether or not the films, TV series, and other media are appropriate for which age groups. Every piece of material undergoes a thorough evaluation process before being assigned a

category based on suitability, which considers elements like language, violence, and mature themes. The platform is simple for parents to access and offers a wealth of options for age-appropriate media. The information is divided into age categories so that parents can quickly locate the best advice for the particular requirements of their kids. A button of "Know more about it is present" at the end of the page. If the users want to know more about it, they can click on it and access the official IMDB page related to the parent guidance. Fig. 4.67 shows the parental guidance page.

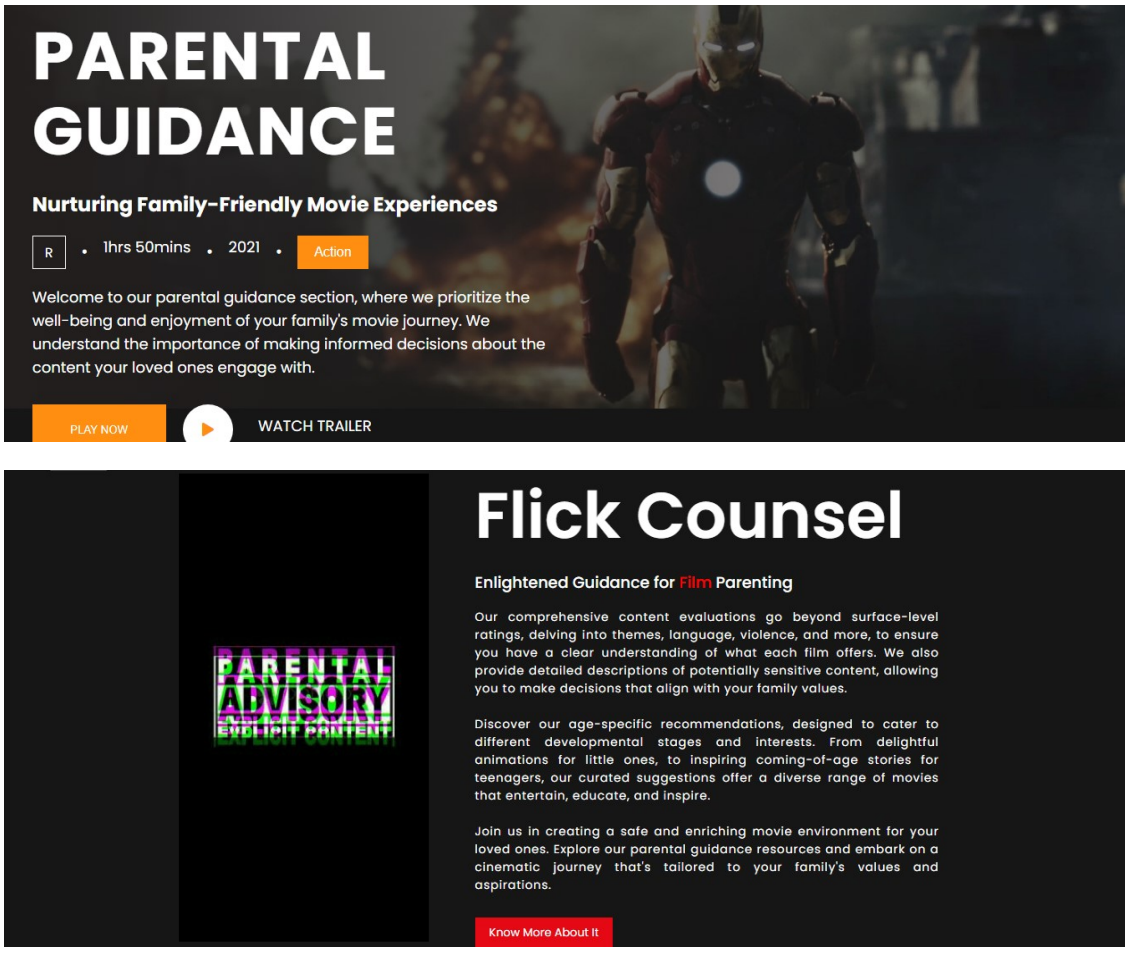


Figure 4.67 Parental Guidance Page

CONCLUSION & FUTURE WORK

To sum up, our interest has been directed to the development and implementation of a "Big Data-based IMDb Movie Review Engine" that combines a sentiment analysis-based movie review engine with a content-based movie recommendation system. The outcomes of the research show how big data approaches may be used to access enormous amounts of movie-related data to improve user experience and offer insightful analysis of sentiment and movie preferences. Users have found the cosine similarity technique-based content-based movie recommendation system useful for receiving individualized movie suggestions. The technology provides customized suggestions that suit the likes and preferences of users by comparing movies based on their content aspects and identifying similarities between them. This improves the movie-finding experience and helps viewers locate films that interest them. This thesis has demonstrated the potential of big data techniques to improve movie-watching experience. Combining a recommendation system based on content with a movie review engine that analyzes sentiments gives users a comprehensive platform for watching, selecting, and understanding movies. Film research and entertainment both benefit from big data analytics. The study's conclusions support this. Additionally, they contribute to the domains of sentiment analysis and recommendation systems for movies. In the future, studies might concentrate on growing the dataset and improving the algorithms to improve the sentiment analysis engine's and recommendation system's performance and accuracy. Furthermore, adding user reviews and social media information can offer more in-depth understanding of movie tastes and attitudes. Through constant development and improvement of these technologies, we can provide consumers a more customized and enriched movie-watching experience, which will eventually increase their happiness and level of involvement with the IMDb Movie Review Engine.

REFERENCES

[1] Sarker, Kamal Uddin, et al. "A Ranking Learning Model by K-Means Clustering Technique for Web Scraped Movie Data." *Computers* 11.11 (2022): 158.

[2] Rajarajeswari, S., et al. "Movie recommendation system." *Emerging Research in Computing, Information, Communication and Applications: ERCICA 2018, Volume 1*. Springer Singapore, 2019.

[3] Peng, Yuanzhe. "A survey on modern recommendation system based on big data." *arXiv preprint arXiv:2206.02631* (2022).

[4] Wei, Chutian, et al. "Fully content-based IMDb movie recommendation engine with Pearson similarity." *International Conference on Green Communication, Network, and Internet of Things (GCNIoT 2021)*. Vol. 12085. SPIE, 2021.

[5] Verma, Jai Prakash, Bankim Patel, and Atul Patel. "Big data analysis: recommendation system with Hadoop framework." *2015 IEEE International Conference on Computational Intelligence & Communication Technology*. IEEE, 2015.

[6] Hongyan Liu, Jun He, Tingting Wang, Wenting Song, Xiaoyang Du, "Combining user preferences and user opinions for accurate recommendation", Elsevier Journal- Electronic Commerce Research and Applications 12 (2013) 14–23.

[7] Budiprasetyo, Gunawan, et al. "Big data system of research data in the informatics department based on software enhancement." *International Journal CISIMA (14)* (2021): 347-362.

[8] Wang, Yibo, Mingming Wang, and Wei Xu. "A sentiment-enhanced hybrid recommender system for movie recommendation: a big data analytics framework." *Wireless Communications and Mobile Computing* 2018 (2018).

[9] C. Basu, H. Hirsh, and W. Cohen, "Recommendation as classifcation: Using social and content-based information in recommendation," in Proceedings of the 1998 15th National Conference on Artifcial Intelligence, AAAI, pp. 714–720, July 1998.

[10] C. Ono, M. Kurokawa, Y. Motomura, and H. Asoh, "A contextaware movie preference model using a bayesian network for recommendation and promotion," in Proceedings of 11th International Conference on User Modeling, pp. 247–257, 2007

[11] N. Entezari, E. E. Papalexakis, H. Wang, S. Rao, and S. K. Prasad, "Tensor-based complementary product recommendation," in 2021 IEEE International Conference on Big Data (Big Data). IEEE, 2021, pp. 409–415.

[12] Y. Cheng and X. Bu, “Research on key technologies of personalized education resource recommendation system based on big data environment,” in Journal of Physics: Conference Series, vol. 1437, no. 1. IOP Publishing, 2020, p. 012024.

[13] Gavua, Ebenezer Komla, Seth Okyere-Dankwa, and Gignarta Kasaye Soka. "Big Data Sentiment Analysis based on PLSA and its Application."

[14] B. Pang, L. Lee, and S. Vaithyanathan, “Tumbs up?: sentiment classifcation using machine learning techniques,” in Proceedings of the ACL-02 Conference on Empirical Methods in Natural Language Processing—Volume 10 (EMNLP ’02), pp. 79– 86, Association for Computational Linguistics, Stroudsburg, Pa, USA, July 2002.

[15] P. D. Turney, “Tumbs up or thumbs down?” in Proceedings of the 40th Annual Meeting on Association for Computational Linguistics, pp. 417–424, Philadelphia, Pennsylvania, July 2002.

[16] S. Panigrahi, R. K. Lenka, and A. Stitipragyan, “A Hybrid Distributed Collaborative Filtering Recommender Engine Using Apache Spark,” in Proceedings of the 7th International Conference on Ambient Systems, Networks and Technologies, ANT 2016 and the 6th International Conference on Sustainable Energy Information Technology, SEIT 2016, pp. 1000–1006, Spain, May 2016.

[17] Singh, Ramni Harbir, et al. "Movie recommendation system using cosine similarity and KNN." *International Journal of Engineering and Advanced Technology* 9.5 (2020): 556-559.

[18] Lüthe, Marvin. *Calculate Similarity - the Most Relevant Metrics in a Nutshell*. 2021.

[19] Dahir, Ubaid Mohamed, and Faisal Kevin Alkindy. "Utilizing Machine Learning for Sentiment Analysis of IMDB Movie Review Data." *vol* 71 (2023): 18-26.

[20] Wang, Jie, and Zihao Li. "Research on face recognition based on CNN." *IOP Conference Series: Earth and Environmental Science*. Vol. 170. No. 3. IOP Publishing, 2018.

[21] Tommandru, Suresh, and Domnic Sandanam. "An Automated Framework for Patient Identification and Verification Using Deep Learning." *Revue d'Intelligence Artificielle* 34.6 (2020).

[22] Reddy Boyapally, Shashank. "Facial Recognition and Attendance System Using Dlib and Face_Recognition Libraries." *Available at SSRN 3804334* (2021).

[23] Abubakar, Haisal Dauda, Mahmood Umar, and Muhammad Abdullahi Bakale. "Sentiment classification: Review of text vectorization methods: Bag of words, Tf-Idf, Word2vec and Doc2vec." *SLU Journal of Science and Technology* 4.1 & 2 (2022): 27-33.

[24] Peng, Yuanzhe. "A survey on modern recommendation system based on big data." *arXiv preprint arXiv:2206.02631* (2022).

[25] Movie Recommendation System S. Rajarajeswari, Sharat Naik, Shagun Srikant, M. K. Sai Prakash and Prarthana Uday

[26] Skrzypczyński, Mikołaj, and Piotr Muryjas. "Analysis of data processing efficiency with use of Apache Hive and Apache Pig in Hadoop environment." *Journal of Computer Sciences Institute* 30 (2024): 1-8.

[27] Singh, Ranjeet Kumar. "Developing a big data analytics platform using Apache Hadoop Ecosystem for delivering big data services in libraries." *Digital Library Perspectives* (2024).

[28] Gupta, Urvashi, and Rohit Sharma. "Apache Hadoop framework for big data analytics using AI." *Artificial Intelligence and Blockchain in Industry 4.0*. CRC Press, 2024. 130-140.

[29] Chen, Luyang, Fan Yang, and Heqing Yang. "Image-based product recommendation system with convolutional neural networks." (2017).

[30] Saxena, Aastha, et al. "Sentiment Analysis of Stocks Based on News Headlines Using NLP." *International Conference on Artificial Intelligence Techniques for Electrical Engineering Systems (AITEES 2022)*. Atlantis Press, 2022.

[31] Awan, Mazhar Javed, et al. "A recommendation engine for predicting movie ratings using a big data approach." *Electronics* 10.10 (2021): 1215.

APPENDIX

APACHE PIG SCRIPTS FOR ANALYSIS OF MOVIE REVIEW DATA:

```
REGISTER /usr/hdp/2.4.0.0-169/pig/piggybank.jar;
review_data = LOAD '/Movies_Review/IMDB_Dataset.csv' USING PigStorage(',') AS
(review:chararray, id:int);
get_details = FOREACH review_data GENERATE id, review;
tokens = FOREACH get_details GENERATE id, review, FLATTEN(TOKENIZE(review)) AS
words;
dictionary = LOAD '/Movies_Review/AFINN.txt' USING PigStorage('\t') AS
(word:chararray, rating:int);
word_ratings = JOIN tokens BY words LEFT OUTER, dictionary BY word USING
'replicated';
ratings = FOREACH word_ratings GENERATE tokens::id AS id, tokens::review AS
review, dictionary::rating AS rating;
group_words = GROUP ratings BY (id, review);
avg_ratings = FOREACH group_words GENERATE group, AVG(ratings.rating) AS
review_rating;
positive_reviews = FILTER avg_ratings BY review_rating > 0;
negative_reviews = FILTER avg_ratings BY review_rating < 0;
STORE positive_reviews INTO 'hdfs:///user/admin//positive_reviewss' USING
PigStorage(',');
STORE negative_reviews INTO 'hdfs:///user/admin//negative_reviewss' USING
PigStorage(',');
```

APACHE PIG SCRIPTS FOR ANALYSIS OF MOVIE METADATA:

```
REGISTER /usr/hdp/2.4.0.0-169/pig/piggybank.jar;
data = LOAD '/Movie_Recommendation/movies.csv' USING PigStorage(',') AS (

index:int, budget:long, genres:chararray, homepage:chararray, id:int,
keywords:chararray, original_language:chararray, original_title
:chararray,overview:chararray, popularity:float, production_companies:chararray,
production_countries:chararray, release_date:chararray,
revenue:long, runtime:int, spoken_languages:chararray, status:chararray,
tagline:chararray, title:chararray, vote_average:float,
vote_count:int, cast:chararray, crew:chararray, director:chararray

);
each_movies = FOREACH data GENERATE index, genres, keywords, original_title,
overview, release_date, vote_average;
```

```
-- Comedy movies
filterComedy = FILTER each_movies BY LOWER(genres) MATCHES '.*(comedy).*' OR
LOWER(original_title) MATCHES '.*(comedy).*';
comedyMovies = FOREACH filterComedy GENERATE original_title, genres,
vote_average;
orderedComedyMovies = ORDER comedyMovies BY vote_average DESC;
topComedyMovies = LIMIT orderedComedyMovies 10;
topComedyMoviesWithTitle = FOREACH topComedyMovies GENERATE original_title,
genres, vote_average;
STORE topComedyMoviesWithTitle INTO 'hdfs:///user/admin/topComedyMoviesWithTitle'
USING PigStorage(',');

-- Adventures movies
filterAdventure = FILTER data BY LOWER(genres) MATCHES '.*(adventure).*' OR
LOWER(original_title) MATCHES '.*(adventure).*';
adventureMovies = FOREACH filterAdventure GENERATE original_title, genres,
vote_average;
orderedAdventureMovies = ORDER adventureMovies BY vote_average DESC;
topAdventureMovies = LIMIT orderedAdventureMovies 10;
topAdventureMoviesWithTitle = FOREACH topAdventureMovies GENERATE original_title,
genres, vote_average;
STORE topAdventureMoviesWithTitle INTO
'hdfs:///user/admin/topAdventureMoviesWithTitle' USING PigStorage(',');

-- Action movies
filterAction = FILTER data BY LOWER(genres) MATCHES '.*(action).*' OR
LOWER(original_title) MATCHES '.*(action).*';
actionMovies = FOREACH filterAction GENERATE original_title, genres,
vote_average;
orderedActionMovies = ORDER actionMovies BY vote_average DESC;
topActionMovies = LIMIT orderedActionMovies 10;
topActionMoviesWithTitle = FOREACH topActionMovies GENERATE original_title,
genres, vote_average;
STORE topActionMoviesWithTitle INTO 'hdfs:///user/admin/TopActionMovies' USING
PigStorage(',');

--thriller/mystery movies
ThrillerMystery = FILTER data BY LOWER(genres) MATCHES '.*(thriller|mystery).*'
OR LOWER(original_title) MATCHES '.*(thriller|mystery).*';
thrillerMysteryMovies = FOREACH ThrillerMystery GENERATE original_title, genres,
vote_average;
orderedThrillerMysteryMovies = ORDER thrillerMysteryMovies BY vote_average DESC;
topThrillerMysteryMovies = LIMIT orderedThrillerMysteryMovies 10;
```

```
topThrillerMysteryMoviesWithTitle = FOREACH topThrillerMysteryMovies GENERATE
original_title, genres, vote_average;
STORE topThrillerMysteryMoviesWithTitle INTO
'hdfs:///user/admin/TopThrillerMysteryMovies' USING PigStorage(',');

--Dramatic movies
DramaticMovies = FILTER data BY LOWER(genres) MATCHES '.*(drama).*' OR
LOWER(original_title) MATCHES '.*(drama).*';
dramaMovies = FOREACH DramaticMovies GENERATE original_title, genres,
vote_average;
orderedDramaMovies = ORDER dramaMovies BY vote_average DESC;
topDramaMovies = LIMIT orderedDramaMovies 10;
topDramaMoviesWithTitle = FOREACH topDramaMovies GENERATE original_title, genres,
vote_average;
STORE topDramaMoviesWithTitle INTO 'hdfs:///user/admin/TopDramaMovies' USING
PigStorage(',');

-- Horror movies
Horror_movies = FILTER data BY LOWER(genres) MATCHES '.*(horror).*' OR
LOWER(original_title) MATCHES '.*(horror).*';
horrorMovies = FOREACH Horror_movies GENERATE original_title, genres,
vote_average;
orderedHorrorMovies = ORDER horrorMovies BY vote_average DESC;
topHorrorMovies = LIMIT orderedHorrorMovies 10;
topHorrorMoviesWithTitle = FOREACH topHorrorMovies GENERATE original_title,
genres, vote_average;
STORE topHorrorMoviesWithTitle INTO 'hdfs:///user/admin/TopHorrorMovies' USING
PigStorage(',');

-- Crime movies
Crime_movies = FILTER data BY LOWER(genres) MATCHES '.*(crime).*' OR
LOWER(original_title) MATCHES '.*(crime).*';
crimeMovies = FOREACH Crime_movies GENERATE original_title, genres, vote_average;
orderedCrimeMovies = ORDER crimeMovies BY vote_average DESC;
topCrimeMovies = LIMIT orderedCrimeMovies 10;
topCrimeMoviesWithTitle = FOREACH topCrimeMovies GENERATE original_title, genres,
vote_average;
STORE topCrimeMoviesWithTitle INTO 'hdfs:///user/admin/TopCrimeMovies' USING
PigStorage(',');
```

CODE FOR MOVIE RECOMMENDATION:

```
import pandas as pd
```

```
movies=pd.read_csv('dataset.csv')
movies.head(10)
movies.describe()
movies.info()
movies.isnull().sum()
movies.columns
movies=movies[['id', 'title', 'overview', 'genre']]
movies
movies['tags'] = movies['overview']+movies['genre']
movies
new_data  = movies.drop(columns=['overview', 'genre'])
new_data
from sklearn.feature_extraction.text import CountVectorizer
cv=CountVectorizer(max_features=10000, stop_words='english')
cv
vector=cv.fit_transform(new_data['tags'].values.astype('U')).toarray()
vector.shape
from sklearn.metrics.pairwise import cosine_similarity
similarity=cosine_similarity(vector)
similarity
new_data[new_data['title']=="The Godfather"].index[0]
distance = sorted(list(enumerate(similarity[2])), reverse=True, key=lambda
vector:vector[1])
for i in distance[0:5]:
    print(new_data.iloc[i[0]].title)
def recommand(movies):
    index=new_data[new_data['title']==movies].index[0]
    distance = sorted(list(enumerate(similarity[index])), reverse=True,
key=lambda vector:vector[1])
    for i in distance[0:5]:
        print(new_data.iloc[i[0]].title)
recommand("Iron Man")
import pickle
pickle.dump(new_data, open('movies_list.pkl', 'wb'))
pickle.dump(similarity, open('similarity.pkl', 'wb'))
pickle.load(open('movies_list.pkl', 'rb'))
```

CODE FOR SENTIMENT ANALYSIS:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn import naive_bayes
```

```
from sklearn.metrics import roc_auc_score,accuracy_score
import pickle as pk
import streamlit as st
import nltk
from nltk.corpus import stopwords

dataset = pd.read_csv('/IMDB_movie.csv')
dataset

dataset.dropna(inplace=True)
dataset
def clean_review(review):
    str = ' '.join(word for word in review.split() if word.lower() not in
stopwords.words('english'))
    return str
dataset['review'] = dataset['review'].apply(clean_review)

#wordcloud

from wordcloud import WordCloud
#draw Wordcloud
reviews = ' '.join(word for word in dataset['review'][dataset['sentiment'] ==
'negative'].astype(str))
wordcloud = WordCloud(height = 600,width =1000, max_font_size = 100)
plt.figure(figsize = (15,12))
plt.imshow(wordcloud.generate(reviews), interpolation='bilinear')
plt.axis('off')
plt.show()

reviews = ' '.join(word for word in dataset['review'][dataset['sentiment']==
'positive'].astype(str))
wordcloud = WordCloud(height = 600, width = 1000, max_font_size =100)
plt.figure(figsize=(15,12))
plt.imshow(wordcloud.generate(reviews), interpolation='bilinear')
plt.axis('off')
plt.show()

from sklearn.feature_extraction.text import TfidfVectorizer
cv = TfidfVectorizer(max_features=2500)

reviews = cv.fit_transform(dataset['review']).toarray()
dataset['sentiment'] =
dataset['sentiment'].replace(['positive','negative'],[1,0])
dataset['sentiment'].value_counts()
```

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
from sklearn.model_selection import train_test_split
reviews_train, reviews_test, sent_train, sent_test = train_test_split(reviews,
dataset['sentiment'], test_size=0.2)
model.fit(reviews_train,sent_train)
predict = model.predict(reviews_test)

from sklearn.metrics import confusion_matrix,ConfusionMatrixDisplay
cm = confusion_matrix(sent_test, predict, labels=model.classes_)
display = ConfusionMatrixDisplay(confusion_matrix = cm,
display_labels=model.classes_)
display.plot()
plt.show()

import pickle as pk
pk.dump(model,open('model.pkl','wb'))
pk.dump(cv,open('scaler.pkl','wb'))
model = pk.load(open('model.pkl','rb'))
scaler = pk.load(open('scaler.pkl','rb'))
def predict_review_sentiment(user_review):
    review_scale = scaler.transform([user_review]).toarray()
    result = model.predict(review_scale)
    return 'positive' if result[0] == 1 else 'negative'

def main():
    print("Welcome to the Movie Review Sentiment Predictor!")
    print("Please enter your review about the movie:")

    user_review = input("> ")

    sentiment = predict_review_sentiment(user_review)
    print(f"The sentiment of the review is: {sentiment}")
    # Load the existing dataset or create an empty DataFrame if the file doesn't
exist
    try:
        dataset = pd.read_csv('C:/Users/PMLS/Project/Data_sample/IMDB_movie.csv')
    except FileNotFoundError:
        dataset = pd.DataFrame(columns=['review', 'sentiment'])

    # Create a new DataFrame with the user review and sentiment
    new_entry = pd.DataFrame({'review': [user_review], 'sentiment': [sentiment]})

    # Concatenate the new entry with the existing dataset
```

```
    dataset = pd.concat([new_entry, dataset], ignore_index=True)

    # Save the updated dataset
    dataset.to_csv('IMDB_movie.csv', index=False)

    # Print the updated dataset
    print("Updated dataset:")
    print(dataset)

if __name__ == "__main__":
    main()
dataset = pd.read_csv('/IMDB_movie.csv')
dataset

for i in range(5):
    print("Review: ", [i])
    print(dataset['review'].iloc[i], "\n")
    print("Sentiment: ", dataset['sentiment'].iloc[i], "\n\n")
```

CODE FOR ACTOR-BASED RECOMMENDATION SYSTEM

```
import face_recognition as fr
import cv2 as cv
from tkinter import Tk
from tkinter.filedialog import askopenfilename
import os
Tk().withdraw
load_image = askopenfilename()
target_image = fr.load_image_file(load_image)
target_encoding = fr.face_encodings(target_image)
print(target_encoding)
def encode_faces(folder):
    list_people_encoding = []
    for filename in os.listdir(folder):
        known_image = fr.load_image_file(f'{folder}{filename}')
        known_encoding = fr.face_encodings(known_image)[0]
        list_people_encoding.append((known_encoding,filename))
    return list_people_encoding
def find_target_face():
    face_location = fr.face_locations(target_image)
    matching_results = []
    for person in
encode_faces('C:/Users/PMLS/Project/Data_sample/face_recog/images/'):
        encoded_face = person[0]
```

```
        filename = person[1]
        is_target_face = fr.compare_faces(encoded_face, target_encoding,
tolerance=0.55)
        print(f'{is_target_face}{filename}')
        if face_location:
            face_number = 0
            for location in face_location:
                if is_target_face[face_number]:
                    label=filename
                    create_frame(location, label)

                face_number +=1
        if is_target_face[0]:
            matching_results.append(filename)

    return matching_results

def recommend_matching_results(matching_results):
    if matching_results:
        print("Recommended matches:")
        for result in matching_results:
            print(result)
    else:
        print("No matches found.")

def create_frame(location, label):
    top, right, bottom, left = location

    cv.rectangle(target_image, (left,top), (right,bottom), (255,0,0), 2)
    cv.rectangle(target_image, (left,bottom + 20), (right, bottom), (255,0,0),
cv.FILLED)
    cv.putText(target_image, label, (left + 3, bottom + 14),
cv.FONT_HERSHEY_DUPLEX, 0.4, (255,255,255), 1)

def render_image():
    rgb_img = cv.cvtColor(target_image, cv.COLOR_BGR2RGB)
    cv.imshow('Face Recognition', rgb_img)
    cv.waitKey(0)

matching_results = find_target_face()
recommend_matching_results(matching_results)
render_image()
```

CODE FOR FRONTEND RECOMMENDATION SERVER

- **HOME.html**

```
<!DOCTYPE html>
<html>
<head>
  <title>Movie Recommendation System and Review Engine</title>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">

  <!-- Google Fonts -->
  <link href='https://fonts.googleapis.com/css?family=Poppins' rel='stylesheet'>

  <!-- Font Awesome -->
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/4.7.0/css/font-awesome.min.css">
  <link rel="stylesheet" type="text/css" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/4.7.0/css/font-awesome.css">

  <!-- Bootstrap -->
  <link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/4.0.0/css/bootstrap.min.css" integrity="sha384-Gn5384xqQ1aoWXA+058RXPxPg6fy4IWvTNh0E263XmFcJlSAwiGgFAW/dAiS6JXm" crossorigin="anonymous">

  <!-- Auto Complete -->
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/@tarekraafat/autocomplete.js@7.2.0/dist/css/autoComplete.min.css">
  <link rel= "stylesheet" type= "text/css" href= "{{ url_for('static',filename='style.css') }}">
  <script type="text/javascript">
    var films = {{suggestions|tojson}};
  </script>

    <style>
*{
margin: 0;
padding: 0;
}
```

```
.header{
min-height: 5vh;
width: 100%;
}
nav {
display: flex;
padding: 1% 6%;
justify-content: space-between;
align-items: center;
}
.nav-links{
flex: 1;
text-align: center;
}
.nav-links ul li{
list-style: none;
display: inline-block;
padding: 8px 15px;
position: relative;
}
.nav-links ul li a{
color: #fff;
text-decoration: none;
font-size: 16px;
font-family: Poppins;
}
.nav-links ul li::after{
content: '';
width: 0%;
height: 2px;
background: #f70577;
display: block;
margin: auto;
transition: 0.5s;
}
.nav-links ul li:hover::after{
width: 100%;
}
.footer{
    font-size: 15px;
    color: #777;
    text-align: center;
}
```

```
.wrapper{
    position: absolute;
    bottom: 45px;
    left: 2%;
}
.wrapper ul li{
    list-style-type: none;
    padding: 2px;
    border-radius: 50%;
    margin: 5px 10px;
}
.wrapper ul li .fa {
    font-size: 27px;
    line-height: 50px;
    transition: .5s;
}
.wrapper ul li .fa:hover {
    color: #ffffff;
}
.wrapper ul li a {
    position: relative;
    display: block;
    width: 50px;
    height: 50px;
    border-radius: 50%;
    color: white;
    text-align: center;
    transition: 0.5s;
    transform: translate(0,0px);
    box-shadow: 0px 7px 5px rgba(0, 0, 0, 0.5);
}
.wrapper ul li a:hover {
    transform: rotate(0deg) skew(0deg) translate(0, -10px);
}
.wrapper ul li:nth-child(1){
    background: #3b5998;
}
.wrapper ul li:nth-child(2){
    background-image: linear-gradient(#f9ce34,  #ee2a7b , #6228d7);
}
.wrapper ul li:nth-child(3){
    background: #00aced;
}
```

```
.wrapper ul li:nth-child(4){
    background: #cb2017;
}
.wrapper ul li:nth-child(5){
    background: #0e76A8;
}
.wrapper ul li:nth-child(6){
    background: #dd4b39;
}
</style>
</head>

<body id="content" style="font-family: Verdana, sans-serif">
    <div class="ml-container" style="display: block;">
        <svg data-toggle="tooltip"
              data-placement="right" width="80" height="80" viewBox="0 0 250 250"
            style="fill:#f70577; color:#fff; position: fixed;z-index:100; top: 0;
border: 0; right: 0;" aria-hidden="true">
            <path d="M0,0 L115,115 L130,115 L142,142 L250,250 L250,0 Z"></path>
            <path
              d="M128.3,109.0 C113.8,99.7 119.0,89.6 119.0,89.6 C122.0,82.7
120.5,78.6 120.5,78.6 C119.2,72.0 123.4,76.3 123.4,76.3 C127.3,80.9 125.5,87.3
125.5,87.3 C122.9,97.6 130.6,101.9 134.4,103.2"
              fill="currentColor" style="transform-origin: 130px 106px;"
class="octo-arm"></path>
            <path
              d="M115.0,115.0 C114.9,115.1 118.7,116.5 119.8,115.4 L133.7,101.6
C136.9,99.2 139.9,98.4 142.2,98.6 C133.8,88.0 127.5,74.4 143.8,58.0 C148.5,53.4
154.0,51.2 159.7,51.0 C160.3,49.4 163.2,43.6 171.4,40.1 C171.4,40.1 176.1,42.5
178.8,56.2 C183.1,58.6 187.2,61.8 190.9,65.4 C194.5,69.0 197.7,73.2 200.1,77.6
C213.8,80.2 216.3,84.9 216.3,84.9 C212.7,93.1 206.9,96.0 205.4,96.6 C205.1,102.4
203.0,107.8 198.3,112.5 C181.9,128.9 168.3,122.5 157.7,114.1 C157.9,116.9
156.7,120.9 152.7,124.9 L141.0,136.5 C139.8,137.7 141.6,141.9 141.8,141.8 Z"
              fill="currentColor" class="octo-body"></path>
              </svg>
            </a><br><br><br><br><br>
    <center><b>
    <h1 style="font-family: 'Poppins'; color: #fff; font-size: 60px"> MOVIE
RECOMMENDATION SYSTEM <br> AND REVIEW ENGINE</h1></b></center>
    <div class="form-group shadow-textarea" style="margin-top: 30px;text-align:
center;color: white;">
        <input type="text" name="movie" class="movie form-control"
id="autoComplete" autocomplete="off" placeholder="Enter the Movie Name"
```

```
style="background-color: #ffffff;border-color:#ffffff;width: 60%;color: #181818"
required="required" />
        <br>
    </div>
    <div class="form-group" style="text-align: center;">
      <button class="btn btn-primary btn-block movie-button" style="background-
color: #f70577;text-align: center;border-color: #f70577;width:120px;"
disabled="true" >Enter</button><br><br>
    </div>
    </div>

  <div id="loader" class="text-center">
  </div>

  <div class="fail">
    <center><h2>INVALID MOVIE NAME!!!</h2><br>
    <h3>Sorry! The movie you requested is not in our database.
    Please check the spelling or try with other movies!</h3></center>
  </div>

    <div class="results">
    <center>
      <h2 id="name" class="text-uppercase"></h2>
    </center>
    </div>
<div class="footer">
    <div class="wrapper">
      <ul>
            <li><a href="https://www.facebook.com/"><i class="fa fa-
facebook"></i></a></li>
            <li><a href="https://www.instagram.com/"><i class="fa fa-
instagram"></i></a></li>
            <li><a href="https://www.twitter.com/"><i class="fa fa-
twitter"></i></a></li>
            <li><a href="https://www.pinterest.com/"><i class="fa fa-
pinterest"></i></a></li>
            <li><a href="https://www.linkedin.com/"><i class="fa fa-
linkedin"></i></a></li>
            <li><a href="https://www.google.com/"><i class="fa fa-google-
plus"></i></a></li>
</ul>
  </div>
</div>
```

```
    <script
src="https://cdn.jsdelivr.net/npm/@tarekraafat/autocomplete.js@7.2.0/dist/js/auto
Complete.min.js"></script>
  <script type="text/javascript" src="{{url_for('static',
filename='autocomplete.js')}}"></script>

  <script
src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js"></script>
  <script type="text/javascript" src="{{url_for('static',
filename='recommend.js')}}"></script>

</body>
</html>
```

- **INDEX.html**

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Movie Recommendation System and Review Engine</title>

<!-- Google Fonts -->
  <link href='https://fonts.googleapis.com/css?family=Poppins' rel='stylesheet'>

<!-- Font Awesome -->
  <link rel="stylesheet"
href="https://pro.fontawesome.com/releases/v5.10.0/css/all.css"
integrity="sha384-
AYmEC3Yw5cVb3ZcuHtOA93w35dYTsvhLPVnYs9eStHfGJvOvKxVfELGroGkvsg+p"
crossorigin="anonymous" />

<style>
*{
    padding: 0;
    margin: 0;
    box-sizing: border-box;
}
body{
    width: 100%;
```

```
    height: 100%;
    color: white;
    background: black;
    font-family: Poppins;
}
.nav-links{
flex: 1;
text-align: center;
}
.nav-links ul li{
list-style: none;
display: inline-block;
padding: 15px 15px;
position: relative;
}
.nav-links ul li a{
color: #fff;
text-decoration: none;
font-size: 15.5px;
font-family: Poppins;
}
.nav-links ul li::after{
content: '';
width: 0%;
height: 2px;
background: #FBA11D;
display: block;
margin: auto;
transition: 0.5s;
}
.nav-links ul li:hover::after{
width: 100%;
}

/*---------global------*/
.container {
  max-width: 100%;
  margin: auto;
}

a {
  text-decoration: none;
  transition: 0.5s;
```

```
}

img {
  width: 100%;
  height: 100%;
}

ul {
  list-style: none;
}

.flex {
  display: flex;
}

.flex1 {
  display: flex;
  justify-content: space-between;
}

i {
  cursor: pointer;
  transition: 0.5s;
}

.logo img {
  width: 140px;
  height: 100%;
  margin-top: 50px;
  margin-left: 50px;
}

header {
  height: 12vh;
  background: rgba(0, 0, 0, 0.2);
}

header .navbar {
  align-items: center;
  transition: 0.5s;
  padding: 40px;
}
```

```css
.subscribe i:nth-child(1) {
  margin-top: 17px;
}

.subscribe i {
  font-size: 20px;
  margin-right: 20px;
}

#palybtn {
  background: #FBA11D;
  width: 50px;
  height: 50px;
  color: #fff;
  line-height: 50px;
  text-align: center;
  border-radius: 50%;
  margin-right: 10px;
}

.button4 {
  background: #FBA11D;
  border: none;
  outline: none;
  color: #fff;
  font-size: 17px;
  padding: 15px 30px;
  cursor: pointer;
}

.subscribe button {
  margin-left: 15px;
}

header.sticky .headerbg {
  position: relative;
}

header.sticky {
  z-index: 10;
  position: fixed;
  top: 0;
  width: 100%;
```

```
  background: #000;
  transition: 0.5s;
  height: 10vh;
  box-shadow: 2px 2px 2px 1px rgba(0, 0, 0, 0.1);
  transition: 0.5s;
}

header.sticky .subscribe {
  display: none;
}

@media only screen and (max-width:768px) {
  header nav ul {
    position: absolute;
    top: 100px;
    left: 0;
    width: 100%;
    background: #000;
    overflow: hidden;
    transition: max-height 0.5s;
    z-index: 100;
  }

  header nav ul li {
    display: block;
    margin: 50px;
    margin-top: 10px;
    margin-bottom: 10px;
  }

  header .navbar span {
    color: white;
    display: block;
    cursor: pointer;
  }

  .subscribe {
    display: none;
  }
}

.container::before{
    content: "";
```

```
    background: url(https://st.hzcdn.com/simgs/pictures/home-theaters/project-
photos-digitech-custom-audio-and-video-img~d2619875088115c6_14-6156-1-
717857f.jpg) no-repeat center center/cover;
    position: absolute;
    height: 100%;
    width: 100%;
    opacity: 0.4;
    z-index: -1;
}
.container .navbar {
    height: 100%;
    display: flex;
    float: left;
}
.container .navbar .right{
    display: flex;
    height: 100px;
    float: right;
    position: absolute;
    right: 50px;
}
.container .navbar .right button a{
    text-decoration: none;
    color: white;
}
.container .title{
    padding: 50px 45px;
}
.container .title .content{
    width: 950px;
    margin: auto;
    padding: 75px 0;
    text-align: center;
}
.container .title .content h1{
    width: 970px;
    margin: auto;
    font-size: 3.1rem;
    text-align: center;
}
.container .title .content h2{
    width: 650px;
    margin: auto;
```

```
    font-size: 1.6rem;
    text-align: center;
}
.container .title .content form h3{
    font-size: 1rem;
    margin: 18px 0;
    text-align: center;
}
.container .title .content form .email{
    width: 1000px;
    margin: auto;
    padding-right: 50px;
}
.container .title .content form .email input{
    height: 60px;
    width: 430px;
    margin: 10px;
    padding: 20px;
    font-size: 20px;
    outline: none;
    border-radius: 7px;
}
.container .title .content form .email button{
    margin: auto;
    height: 60px;
    width: 180px;
    align-items: center;
    justify-content: center;
    font-size: 20px;
    padding: 7px 17px;
    background: #FF8E11;
    border-radius: 7px;
    cursor: pointer;
    outline: none;
    border: none;
    margin-left: -5px;
}
.container .title .content form .email button:hover{
    background-color: #EA6400;
}
.container .title .content form .email button a{
    text-decoration: none;
    color: white;
```

```
}
.footer{
    font-size: 15px;
    color: #777;
    text-align: center;
}
.wrapper{
    position: absolute;
    bottom: 45px;
    left: 1%;
}
.wrapper ul li{
    list-style-type: none;
    padding: 2px;
    border-radius: 50%;
    margin: 5px 10px;
}
.wrapper ul li .fab {
    font-size: 27px;
    line-height: 50px;
    transition: .5s;
}
.wrapper ul li .fa:hover {
    color: #ffffff;
}
.wrapper ul li a {
    position: relative;
    display: block;
    width: 50px;
    height: 50px;
    border-radius: 50%;
    color: white;
    text-align: center;
    transition: 0.5s;
    transform: translate(0,0px);
    box-shadow: 0px 7px 5px rgba(0, 0, 0, 0.5);
}
.wrapper ul li a:hover {
    transform: rotate(0deg) skew(0deg) translate(0, -10px);
}
.wrapper ul li:nth-child(1){
    background: #3b5998;
}
```

```
.wrapper ul li:nth-child(2){
    background-image: linear-gradient(#f9ce34,  #ee2a7b , #6228d7);
}
.wrapper ul li:nth-child(3){
    background: #00aced;
}
.wrapper ul li:nth-child(4){
    background: #cb2017;
}
.wrapper ul li:nth-child(5){
    background: #0e76A8;
}
.wrapper ul li:nth-child(6){
    background: #dd4b39;
}
</style>
</head>
<body>
<div class="container ">
<section class="home">
    <div class="headerbg ">
      <header>
          <div class="navbar flex1">
            <div class="logo">
              <img src="https://seeklogo.com/images/M/movie-time-cinema-logo-
8B5BE91828-seeklogo.com.png" alt="">
            </div>
            <div class="nav-links">
<ul>
            <li><a href="index.html" target="_self">HOME</a></li>
            <li><a href="about.html">ABOUT</a></li>
            <li><a href="trending.html" target="_self">TRENDING MOVIES</a></li>
            <li><a href="contact.html" target="_self">CONTACT US</a></li>
            <li><a href="movie.html" target="_self">REVIEW PREDICTION</a></li>
            <li><a href="guide.html" target="_self">PARENTAL GUIDE</a></li>
</ul>
</div>
            <div class="subscribe flex">
              <a href ="http://127.0.0.1:5000/"><i class="fas fa-search"
style="color: white; margin-top: 1rem;"></i></a>
              <a href ="login.html"><i id="palybtn" class="fas fa-user"
style="color: white; margin-top: 0.1rem;"></i></a>
              <a href="movie.html" class="button4">SUBSCRIBE</a>
```

```
            </div>
          </div>
        </div>
</header>
<div class="title">
            <div class="content">
                <br><br><h1><b>Discover Endless Streaming Bliss!</b></h1>
                <h2>Entertainment on the Go, Wherever You Roam!</h2>
                <form action="#">
                    <h3>Ready to watch? Enter your email to embark on a thrilling
journey through movies, TV shows, and so much more.</h3>
                    <div class="email">
                    <br><br>
                        <input type="email" name="email" placeholder="Email
address">
                        <button><a href="http://127.0.0.1:5000/">Get Started
></a></button>
                    </div>
                </form>
            </div>
</div>
</div>
</section>
<div class="footer">
    <div class="wrapper">
      <ul>
            <li><a href="https://www.facebook.com/"><i class="fab fa-facebook-
f"></i></a></li>
            <li><a href="https://www.instagram.com/"><i class="fab fa-
instagram"></i></a></li>
            <li><a href="https://www.twitter.com/"><i class="fab fa-
twitter"></i></a></li>
            <li><a href="https://www.pinterest.com/"><i class="fab fa-
pinterest"></i></a></li>
            <li><a href="https://www.linkedin.com/"><i class="fab fa-linkedin-
in"></i></a></li>
            <li><a href="https://www.google.com/"><i class="fab fa-google-plus-
g"></i></a></li>
</ul>
  </div>
</div>
</body>
</html>
```

- **LOGIN / REGISTER**

```
<!DOCTYPE html>
<html>
<head>
<title>Login Signup</title>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-
fit=no">
<!-- Google Fonts -->
  <link href='https://fonts.googleapis.com/css?family=Poppins' rel='stylesheet'>

 <!-- Font Awesome -->
<link rel="stylesheet"
href="https://pro.fontawesome.com/releases/v5.10.0/css/all.css"
integrity="sha384-
AYmEC3Yw5cVb3ZcuHtOA93w35dYTsvhLPVnYs9eStHfGJvOvKxVfELGroGkvsg+p"
crossorigin="anonymous" />
<style>
*{
    padding: 0;
    margin: 0;
    box-sizing: border-box;
}
body{
    width: 100%;
    height: 100%;
    color: white;
    background: black;
    font-family: Poppins;
}
.nav-links{
flex: 1;
text-align: center;
}
.nav-links ul li{
list-style: none;
display: inline-block;
padding: 35px 15px;
position: relative;
}
.nav-links ul li a{
```

```
color: #fff;
text-decoration: none;
font-size: 15.5px;
font-family: Poppins;
}
.nav-links ul li::after{
content: '';
width: 0%;
height: 2px;
background: #FBA11D;
display: block;
margin: auto;
transition: 0.5s;
}
.nav-links ul li:hover::after{
width: 100%;
}

/*---------global------*/
.container {
  max-width: 95%;
  margin: auto;
}

a {
  text-decoration: none;
  transition: 0.5s;
}

img {
  width: 100%;
  height: 100%;
}

ul {
  list-style: none;
}

.flex {
  display: flex;
}

.flex1 {
```

```
  display: flex;
  justify-content: space-between;
}

i {
  cursor: pointer;
  transition: 0.5s;
}

/*---------global------*/
/*---------header------*/

.logo img {
  width: 120px;
  height: 100%;
  margin-top: 1px;
  margin-left: 50px;
}

.headerbg {
  height: 100vh;
  width: 100%;
}

header {
  height: 12vh;
  background: rgba(0, 0, 0, 0.2);
}

header .navbar {
  align-items: center;
  transition: 0.5s;
}

.subscribe i:nth-child(1) {
  margin-top: 17px;
}

.subscribe i {
  font-size: 20px;
  margin-right: 20px;
}
```

```css
#palybtn {
  background: #FBA11D;
  width: 50px;
  height: 50px;
  color: #fff;
  line-height: 50px;
  text-align: center;
  border-radius: 50%;
  margin-right: 10px;
}

button {
  background: #FBA11D;
  border: none;
  outline: none;
  color: white;
  padding: 0px 30px;
  cursor: pointer;
}

.subscribe button {
  margin-left: 15px;
}

header.sticky .headerbg {
  position: relative;
}

header.sticky {
  z-index: 10;
  position: fixed;
  top: 0;
  width: 100%;
  background: #000;
  transition: 0.5s;
  height: 10vh;
  box-shadow: 2px 2px 2px 1px rgba(0, 0, 0, 0.1);
  transition: 0.5s;
}

header.sticky .subscribe {
  display: none;
}
```

```
@media only screen and (max-width:768px) {
  header nav ul {
    position: absolute;
    top: 100px;
    left: 0;
    width: 100%;
    background: #000;
    overflow: hidden;
    transition: max-height 0.5s;
    z-index: 100;
  }

  header nav ul li {
    display: block;
    margin: 50px;
    margin-top: 10px;
    margin-bottom: 10px;
  }

  header .navbar span {
    color: white;
    display: block;
    cursor: pointer;
  }

  .home {
    background-repeat: no-repeat;
    background-size: cover;
    background-position: center;
    height: 70vh;
  }

  .subscribe {
    display: none;
  }
}
.full-page
{
        height: 100%;
    width: 100%;
    background-image: linear-
gradient(rgba(0,0,0,0.6),rgba(0,0,0,0.6)),url(https://st.hzcdn.com/simgs/pictures
```

```
/home-theaters/project-photos-digitech-custom-audio-and-video-
img~d2619875088115c6_14-6156-1-717857f.jpg);
    background-position: center;
    background-size: cover;
    position: absolute;
        font-family: Poppins;
}
#login-form
{
    display: block;
}
.form-box
{
        width:380px;
    height:480px;
    position:absolute;
        backdrop-filter: blur(10px);
    background:rgba(0,0,0,0.3);
    padding:25px;
        top: -80px;
        overflow: hidden;
        left: 670px;
        transform: translate(0%,-5%);
        border-radius: 20px;
}
.button-box
{
    width:220px;
    margin:35px auto;
    position:relative;
    box-shadow: 0 0 20px 9px #ff61241f;
    border-radius: 30px;
        font-family: Poppins;
}
.toggle-btn
{
    padding:10px 30px;
    cursor:pointer;
        font-family: Poppins;
    background:transparent;
    border:0;
    outline: none;
    position: relative;
```

```
        color: #fff;
}
#btn
{
    top: 0;
    left:0;
    position: absolute;
    width: 110px;
    height: 100%;
    background: #FBA11D;
    border-radius: 30px;
        font-family: Poppins;
    transition: .5s;
}
.input-group-login
{
    top: 150px;
    position:absolute;
    width:280px;
    transition:.5s;
        font-family: Poppins;
}
.input-group-register
{
        top: 120px;
    position:absolute;
    width:280px;
    transition:.5s;
        font-family: Poppins;
}
.input-field
{
    width: 100%;
    padding:10px 0;
    margin:5px 0;
    border-left:0;
    border-top:0;
    border-right:0;
    border-bottom: 1px solid #999;
    outline:none;
    background: transparent;
}
.submit-btn
```

```
{
    width: 85%;
    padding: 10px 30px;
    cursor: pointer;
    display: block;
    margin: auto;
    background: #FBA11D;
        color: #fff;
    border: 0;
        font-family: Poppins;
    outline: none;
    border-radius: 30px;
}
.check-box
{
    margin: 30px 10px 34px 0;
}
span
{
    color:#777;
    font-size:12px;
    bottom:68px;
    position:absolute;
}
#login
{
    left:50px;
}
#login input
{
    color:white;
    font-size:15;
}
#register
{
    left:450px;
}
#register input
{
    color:white;
    font-size: 15;
}
.content{
```

```
    width: 1200px;
    height: auto;
    margin: auto;
    color: #fff;
    position: relative;
}

.content .par{
    padding-left: 24px;
    padding-bottom: 25px;
    font-family: Poppins;
    letter-spacing: 1.2px;
    font-size: 15px;
}

.content h1{
    font-family: Poppins;
    font-size: 50px;
    padding-left: 20px;
    margin-top: 9%;
    letter-spacing: 2px;
}
.content h3{
    font-family: Poppins;
    font-size: 22px;
    padding-left: 24px;
    margin-top: 1%;
    letter-spacing: 2px;
}
.content .cn{
    width: 160px;
    height: 40px;
    background: #ff7200;
    border: none;
    margin-bottom: 10px;
    margin-left: 20px;
    font-size: 18px;
    border-radius: 10px;
    cursor: pointer;
    transition: .4s ease;

}
```

```css
.content .cn a{
    text-decoration: none;
    color: #000;
    transition: .3s ease;
}

.cn:hover{
    background-color: #fff;
}
.wrapper{
    position: absolute;
    bottom: 45px;
    left: 1%;
}
.wrapper ul li{
    list-style-type: none;
    padding: 2px;
    border-radius: 50%;
    margin: 5px 10px;
}
.wrapper ul li .fab {
    font-size: 27px;
    line-height: 50px;
    transition: .5s;
}
.wrapper ul li .fa:hover {
    color: #ffffff;
}
.wrapper ul li a {
    position: relative;
    display: block;
    width: 50px;
    height: 50px;
    border-radius: 50%;
    color: white;
    text-align: center;
    transition: 0.5s;
    transform: translate(0,0px);
    box-shadow: 0px 7px 5px rgba(0, 0, 0, 0.5);
}
.wrapper ul li a:hover {
    transform: rotate(0deg) skew(0deg) translate(0, -10px);
}
```

```
.wrapper ul li:nth-child(1){
    background: #3b5998;
}
.wrapper ul li:nth-child(2){
    background-image: linear-gradient(#f9ce34,  #ee2a7b , #6228d7);
}
.wrapper ul li:nth-child(3){
    background: #00aced;
}
.wrapper ul li:nth-child(4){
    background: #cb2017;
}
.wrapper ul li:nth-child(5){
    background: #0e76A8;
}
.wrapper ul li:nth-child(6){
    background: #dd4b39;
}
</style>
</head>
<body>
<div class="full-page">
<section class="home">
    <div class="headerbg ">
      <header>
        <div class="container ">
          <div class="navbar flex1">
            <div class="logo">
              <img src="https://seeklogo.com/images/M/movie-time-cinema-logo-
8B5BE91828-seeklogo.com.png" alt="">
            </div>

            <div class="nav-links">
<ul>
            <li><a href="index.html" target="_self">HOME</a></li>
            <li><a href="about.html">ABOUT</a></li>
            <li><a href="trending.html" target="_self">TRENDING MOVIES</a></li>
            <li><a href="contact.html" target="_self">CONTACT US</a></li>
            <li><a href="movie.html" target="_self">REVIEW PREDICTION</a></li>
            <li><a href="guide.html" target="_self">PARENTAL GUIDE</a></li>
</ul>
</div>
```

```
            <div class="subscribe flex">
              <a href ="http://127.0.0.1:5000/"><i class="fas fa-search"
style="color: white; margin-top: 1rem;"></i></a>
              <a href ="login.html"><i id="palybtn" class="fas fa-user"
style="color: white; margin-top: 0.1rem;"></i></a>
              <button>SUBSCRIBE</button>
            </div>
          </div>
        </div>
      </header>
<div class="content">
            <h1>Lights, Camera, <br> Connection!</h1>
            <h3>Join our movie community!</h3>
<p class="par">Unlock cinematic wonders. Log in or register to explore, discover,
<br> and engage with our film-loving community.</p>
<div id='login-form'class='login-page'>
<div class="form-box">
                <div class='button-box'>
                    <div id='btn'></div>
                    <button type='button'onclick='login()'class='toggle-btn'>Log
In</button>
                    <button type='button'onclick='register()'class='toggle-
btn'>Register</button>
                    </div>
<form id='login' class='input-group-login'>
                    <input type='text'class='input-field'placeholder='Email Id'
required >
            <input type='password'class='input-field'placeholder='Enter Password'
required>
            <input type='checkbox'class='check-box'><span>Remember
Password</span>
            <button type='submit' class='submit-btn'><b>Log in</b></button>
         </form>
         <form id='register' class='input-group-register'>
             <input type='text'class='input-field'placeholder='First Name'
required>
             <input type='text'class='input-field'placeholder='Last Name '
required>
             <input type='email'class='input-field'placeholder='Email Id'
required>
             <input type='password'class='input-field'placeholder='Enter
Password' required>
```

```
            <input type='password'class='input-field'placeholder='Confirm
Password'  required>
            <input type='checkbox'class='check-box'><span>I agree to the terms
and conditions</span>
                    <button type='submit'class='submit-
btn'><b>Register</b></button>
            </form>
           </div>
        </div>
    </div>
</div>
</section>

<div class="footer">
    <div class="wrapper">
      <ul>
            <li><a href="https://www.facebook.com/"><i class="fab fa-facebook-
f"></i></a></li>
            <li><a href="https://www.instagram.com/"><i class="fab fa-
instagram"></i></a></li>
            <li><a href="https://www.twitter.com/"><i class="fab fa-
twitter"></i></a></li>
            <li><a href="https://www.pinterest.com/"><i class="fab fa-
pinterest"></i></a></li>
            <li><a href="https://www.linkedin.com/"><i class="fab fa-linkedin-
in"></i></a></li>
            <li><a href="https://www.google.com/"><i class="fab fa-google-plus-
g"></i></a></li>
</ul>
  </div>
</div>

<script>
        var x=document.getElementById('login');
        var y=document.getElementById('register');
        var z=document.getElementById('btn');
        function register()
        {
            x.style.left='-400px';
            y.style.left='50px';
            z.style.left='110px';
        }
        function login()
```

```
        {
            x.style.left='50px';
            y.style.left='450px';
            z.style.left='0px';
        }
    </script>
</body>
</html>
```

- **ABOUT**

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <title>Movie Recommendation System and Review Engine</title>
    <meta name="viewport" content="width=device-width, initial-scale=1">
  <!-- Google Fonts -->
  <link href='https://fonts.googleapis.com/css?family=Poppins' rel='stylesheet'>
  <!-- Font Awesome -->
<link rel="stylesheet"
href="https://pro.fontawesome.com/releases/v5.10.0/css/all.css"
integrity="sha384-
AYmEC3Yw5cVb3ZcuHtOA93w35dYTsvhLPVnYs9eStHfGJvOvKxVfELGroGkvsg+p"
crossorigin="anonymous" />
<style>

@import
url('https://fonts.googleapis.com/css2?family=Poppins:wght@200;300;400;500;600;70
0;800&display=swap');

*{
    padding: 0;
    margin: 0;
    box-sizing: border-box;
}
body{
    width: 100%;
    height: 100%;
    color: white;
    background: #161616;
    font-family: Poppins;
```

```
}
.nav-links{
flex: 1;
text-align: center;
}
.nav-links ul li{
list-style: none;
display: inline-block;
padding: 15px 15px;
position: relative;
margin-bottom: 105px;
}
.nav-links ul li a{
color: #fff;
text-decoration: none;
font-size: 15.5px;
font-family: Poppins;
}
.nav-links ul li::after{
content: '';
width: 0%;
height: 2px;
background: #FBA11D;
display: block;
margin: auto;
transition: 0.5s;
}
.nav-links ul li:hover::after{
width: 100%;
}

/*---------global------*/
.container {
  max-width: 95%;
  margin: auto;
}

a {
  text-decoration: none;
  transition: 0.5s;
}

img {
```

```
  width: 100%;
  height: 100%;
}

ul {
  list-style: none;
}

.flex {
  display: flex;
}

.flex1 {
  display: flex;
  justify-content: space-between;
}

i {
  cursor: pointer;
  transition: 0.5s;
}

/*---------global------*/
/*---------header------*/

.logo img {
  width: 108px;
  height: 100%;
  margin-bottom: 110px;
  margin-left: 70px;
}

.headerbg {
  height: 100vh;
  width: 100%;
}

header {
  height: 14vh;
}

header .navbar {
  align-items: center;
```

```
  transition: 0.5s;
}

.subscribe i:nth-child(1) {
  margin-top: 17px;
}

.subscribe i {
  font-size: 20px;
  margin-right: 20px;
}

#palybtn {
  background: #FBA11D;
  width: 50px;
  height: 50px;
  color: #fff;
  line-height: 50px;
  text-align: center;
  border-radius: 50%;
  margin-right: 10px;
  margin-bottom: 110px;
}

.button {
  background: #FBA11D;
  border: none;
  outline: none;
  height: 53px;
  color: white;
  padding: 15px 30px;
  cursor: pointer;
  font-family: Poppins;
}

.subscribe .button {
  margin-left: 15px;
}

header.sticky .headerbg {
  position: relative;
}
```

```css
header.sticky {
  z-index: 10;
  position: fixed;
  top: 0;
  width: 100%;
  background: #000;
  transition: 0.5s;
  height: 10vh;
  box-shadow: 2px 2px 2px 1px rgba(0, 0, 0, 0.1);
  transition: 0.5s;
}

header.sticky .subscribe {
  display: none;
}

@media only screen and (max-width:768px) {
  header nav ul {
    position: absolute;
    top: 100px;
    left: 0;
    width: 100%;
    background: #000;
    overflow: hidden;
    transition: max-height 0.5s;
    z-index: 100;
  }

  header nav ul li {
    display: block;
    margin: 50px;
    margin-top: 10px;
    margin-bottom: 10px;
  }

  header .navbar span {
    color: white;
    display: block;
    cursor: pointer;
  }

  .home {
    background-repeat: no-repeat;
```

```
    background-size: cover;
    background-position: center;
    height: 70vh;
  }

  .subscribe {
    display: none;
  }
}
.footer{
    font-size: 15px;
    color: #777;
    text-align: center;
}
.wrapper{
    position: absolute;
    bottom: 45px;
    left: 1%;
}
.wrapper ul li{
    list-style-type: none;
    padding: 2px;
    border-radius: 50%;
    margin: 5px 10px;
}
.wrapper ul li .fab {
    font-size: 27px;
    line-height: 50px;
    transition: .5s;
}
.wrapper ul li .fa:hover {
    color: #ffffff;
}
.wrapper ul li a {
    position: relative;
    display: block;
    width: 50px;
    height: 50px;
    border-radius: 50%;
    color: white;
    text-align: center;
    transition: 0.5s;
    transform: translate(0,0px);
```

```
    box-shadow: 0px 7px 5px rgba(0, 0, 0, 0.5);
}
.wrapper ul li a:hover {
    transform: rotate(0deg) skew(0deg) translate(0, -10px);
}
.wrapper ul li:nth-child(1){
    background: #3b5998;
}
.wrapper ul li:nth-child(2){
    background-image: linear-gradient(#f9ce34,  #ee2a7b , #6228d7);
}
.wrapper ul li:nth-child(3){
    background: #00aced;
}
.wrapper ul li:nth-child(4){
    background: #cb2017;
}
.wrapper ul li:nth-child(5){
    background: #0e76A8;
}
.wrapper ul li:nth-child(6){
    background: #dd4b39;
}
.about-drapper{
    height: 100vh;
}
.about-left{
    background-color: #FBA11D;
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: center;
    height: 100vh;
    color:#fff
}
.about-left-content > div{
    background: #161616;
    padding: 4rem 4rem 2.5rem 5rem;
    text-align: center;

    border-radius: 12px 12px 0 0;
}
.about-left-content{
```

```
    box-shadow: 0px 0px 18px -1px rgba(0, 0, 0, 0.39);
    -webkit-box-shadow: 0px 0px 18px -1px rgba(0, 0, 0, 0.39);
    -moz-box-shadow: 0px 0px 18px -1px rgba(0, 0, 0, 0.39);
    border-radius: 12px;
    width:80%;
}
.about-img img{
    display: block;
    width: 200px;
}
.about-img{
    width: 200px;
    height: 200px;
    overflow: hidden;
    border-radius: 50%;
    transition: all 0.5s ease-in-out;
}
.shadow{
    margin-left: auto;
    margin-right: auto;
    border-radius: 50%;
    width: 200px;
    height: 200px;
}
.about-left-content h2{
    font-size: 2rem;
    margin: 2.2rem 0 0.6rem 0;
    line-height: 1.2;
    padding-bottom: 1rem;
    border-bottom: 2px solid #FBA11D;
}
.about-left-content h3{
    text-transform: uppercase;
    font-weight: 300;
    letter-spacing: 5px;
    margin-top: 1.2rem;
}
.icons{
    background: #161616;
    display: flex;
    justify-content: center;
    padding: 0.8rem 0;
    border-radius: 0 0 12px 12px;
```

```
}
.icons li{
    list-style-type: none;
    background: #FBA11D;
    color: #000;
    width: 40px;
    height: 40px;
    margin: 0 0.5rem;
    border-radius: 50%;
    display: flex;
    justify-content: center;
    align-items: center;
    font-size: 1.2rem;
    cursor: pointer;
    transition: all 0.5s ease-in-out;
}
.icons li:hover{
    background: #edffec;
    color: #000;
}
.about-right{
    background: #161616;
    height: 100vh;
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: center;
    padding: 0 5rem;
    text-align: center;
    color:#fff
}
.about-right h1{
    font-size: 5rem;
    text-transform: uppercase;
}
.about-right h1 span{
    color: #B721FF;
}
.about-right h2{
    font-weight: 600;
}
.about-btns{
    display: flex;
```

```
    margin: 2rem 0;
}
.btn{
    border: none;
    font-size: 0.9rem;
    text-transform: uppercase;
    border: 2px solid #fff;
    border-radius: 20px;
    padding: 0.55rem 0;
    width: 130px;
    font-weight: 600;
    background: transparent;
    margin: 0 0.5rem;
    cursor: pointer;
    color:#fff
}
.btn.btn-pink{
    background: #FBA11D;
    color: #fff;
    width: 180px;
    border-color: #FBA11D;
    font-family: Poppins;
    transition: all 0.5s ease-in-out;
}
.btn.btn-pink:hover{
    background: transparent;
    border-color: #fff;
    color: #fff;
}
.btn.btn-white{
    transition: all 0.5s ease-in-out;
    font-family: Poppins;
}
.btn.btn-white:hover{
    background: #FBA11D;
    border-color: #FBA11D;
    color: #fff;
}
.about-para p{
    font-weight: 300;
    padding: 0.5rem;
    opacity: 0.8;
}
```

```
@media screen and (min-width: 992px){
    .about-drapper{
        display: grid;
        grid-template-columns: repeat(2, 2fr);
    }

    .about-left{
        position: relative;
    }
    .about-left-content{
        position: absolute;
        width:80%

    }
}
.credit{
    text-align: center;
    color: #fff;
    font-family: 'Trebuchet MS', 'Lucida Sans Unicode', 'Lucida Grande', 'Lucida
Sans', Arial, sans-serif;
}
.credit a{
    text-decoration: none;
    color:#B721FF;
    font-weight: bold;
}
  </style>
  </head>
  <body>
<section class="home">
    <div class="headerbg ">
      <header>
        <div class="container ">
          <div class="navbar flex1">
            <div class="logo">
              <img src="https://seeklogo.com/images/M/movie-time-cinema-logo-
8B5BE91828-seeklogo.com.png" alt="">
            </div>

            <div class="nav-links">
<ul>
            <li><a href="index.html" target="_self">HOME</a></li>
```

```
            <li><a href="about.html">ABOUT</a></li>
            <li><a href="trending.html" target="_self">TRENDING MOVIES</a></li>
            <li><a href="contact.html" target="_self">CONTACT US</a></li>
            <li><a href="movie.html" target="_self">REVIEW PREDICTION</a></li>
            <li><a href="guide.html" target="_self">PARENTAL GUIDE</a></li>
</ul>
</div>
            <div class="subscribe flex">
              <a href ="http://127.0.0.1:5000/"><i class="fas fa-search"
style="color: white; margin-top: 1rem;"></i></a>
              <a href ="login.html"><i id="palybtn" class="fas fa-user"
style="color: white; margin-top: 0.1rem;"></i></a>
              <a href="movie.html" class="button">SUBSCRIBE</a>
            </div>
          </div>
        </div>
</header>
<div class = "about-drapper">
      <div class = "about-left">
        <div class = "about-left-content">
          <div>
            <div class = "shadow">
              <div class = "about-img">
                <img src =
"https://images.freeimages.com/variants/EKGu1egY2XJxzeTuxm48oyR4/f4a36f6589a0e50e
702740b15352bc00e4bfaf6f58bd4db850e167794d05993d?fmt=webp&w=500" alt = "about
image">
              </div>
            </div>

            <h2>Movie Time</h2>
            <h3>Movie Review Engine</h3>
          </div>

          <ul class = "icons">
            <li><i class = "fab fa-facebook-f"></i></li>
            <li><i class = "fab fa-twitter"></i></li>
            <li><i class = "fab fa-linkedin"></i></li>
            <li><i class = "fab fa-instagram"></i></li>
          </ul>
        </div>

      </div>
```

```
      <div class = "about-right">
        <h1>Hello<span style="color: #FBA11D;">!</span></h1>
        <h2>Here's who we are & what we do</h2>
        <div class = "about-btns">
          <button type = "button" class = "btn btn-pink">Recommendations</button>
          <button type = "button" class = "btn btn-white">Reviews</button>
        </div>

        <div class = "about-para">
          <p>Our goal is to be your go-to destination for everything related to
movies. From the latest blockbusters to timeless classics, we offer a vast and
diverse selection of films to suit every taste and preference. Our extensive
library encompasses various genres, including action, comedy, drama, sci-fi, and
more. You'll find movies from around the globe, showcasing the rich tapestry of
international cinema.</p>
          <p>Lights, camera, action! Enjoy the show!</p>
        </div>
        </div>
    </div>
</section>
  </body>
</html>
```

- **CONTACT US**

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Contact Form</title>
<!-- Google Fonts -->
  <link href='https://fonts.googleapis.com/css?family=Poppins' rel='stylesheet'>

 <!-- Font Awesome -->
<link rel="stylesheet"
href="https://pro.fontawesome.com/releases/v5.10.0/css/all.css"
integrity="sha384-
AYmEC3Yw5cVb3ZcuHtOA93w35dYTsvhLPVnYs9eStHfGJvOvKxVfELGroGkvsg+p"
crossorigin="anonymous" />

<script
```

```
      src="https://kit.fontawesome.com/64d58efce2.js"
crossorigin="anonymous"></script>
<style>
@import
url("https://fonts.googleapis.com/css2?family=Poppins:wght@100;200;300;400;500;60
0;700;800&display=swap");

* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
body,
input,
textarea {
  font-family: Poppins;
}
body{
    width: 100%;
    height: 100%;
    color: white;
    background-image: url(https://wallpaper-mania.com/wp-
content/uploads/2018/09/High_resolution_wallpaper_background_ID_77701605548.jpg);
}

.nav-links{
flex: 1;
text-align: center;
}
.nav-links ul li{
list-style: none;
display: inline-block;
padding: 35px 15px;
position: relative;
margin-bottom: 105px;
}
.nav-links ul li a{
color: #fff;
text-decoration: none;
font-size: 15.5px;
font-family: Poppins;
}
.nav-links ul li::after{
```

```
content: '';
width: 0%;
height: 2px;
background: #FBA11D;
display: block;
margin: auto;
transition: 0.5s;
}
.nav-links ul li:hover::after{
width: 100%;
}

/*---------global------*/
.container {
  max-width: 95%;
  margin: auto;
}

a {
  text-decoration: none;
  transition: 0.5s;
}

img {
  width: 100%;
  height: 100%;
}

ul {
  list-style: none;
}

.flex {
  display: flex;
}

.flex1 {
  display: flex;
  justify-content: space-between;
}

i {
  cursor: pointer;
```

```
  transition: 0.5s;
}

/*---------global------*/
/*---------header------*/

.logo img {
  width: 118px;
  height: 100%;
  margin-bottom: 110px;
  margin-left: 70px;
}

header {
  height: 13vh;
  background: rgba(0, 0, 0, 0.2);
}

header .navbar {
  align-items: center;
  transition: 0.5s;
}

.subscribe i:nth-child(1) {
  margin-top: 17px;
}

.subscribe i {
  font-size: 20px;
  margin-right: 20px;
}

#palybtn {
  background: #FBA11D;
  width: 50px;
  height: 50px;
  color: #fff;
  line-height: 50px;
  text-align: center;
  border-radius: 50%;
  margin-right: 10px;
  margin-bottom: 110px;
}
```

```css
.button {
  background: #FBA11D;
  border: none;
  outline: none;
  height: 53px;
  color: white;
  padding: 15px 30px;
  cursor: pointer;
  font-family: Poppins;
}

.subscribe button {
  margin-left: 15px;
}

header.sticky .headerbg {
  position: relative;
}

header.sticky {
  z-index: 10;
  position: fixed;
  top: 0;
  width: 100%;
  background: #000;
  transition: 0.5s;
  height: 10vh;
  box-shadow: 2px 2px 2px 1px rgba(0, 0, 0, 0.1);
  transition: 0.5s;
}

header.sticky .subscribe {
  display: none;
}

@media only screen and (max-width:768px) {
  header nav ul {
    position: absolute;
    top: 100px;
    left: 0;
    width: 100%;
    background: #000;
```

```
    overflow: hidden;
    transition: max-height 0.5s;
    z-index: 100;
  }

  header nav ul li {
    display: block;
    margin: 50px;
    margin-top: 10px;
    margin-bottom: 10px;
  }

  header .navbar span {
    color: white;
    display: block;
    cursor: pointer;
  }

  .home {
    background-repeat: no-repeat;
    background-size: cover;
    background-position: center;
    height: 70vh;
  }

  .subscribe {
    display: none;
  }
}

.container123 {
  position: relative;
  width: 100%;
  min-height: 100vh;
  overflow: hidden;
  display: flex;
  align-items: center;
  justify-content: center;
}

.form {
  width: 100%;
  max-width: 820px;
```

```
  background-color: #fff;
  border-radius: 10px;
  box-shadow: 0 0 20px 1px rgba(0, 0, 0, 0.1);
  z-index: 1000;
  overflow: hidden;
  display: grid;
  grid-template-columns: repeat(2, 1fr);
}

.contact-form {
  background-color: #FBA11D;
  position: relative;
}
.circle {
  border-radius: 50%;
  background: linear-gradient(135deg, transparent 20%, #000);
  position: absolute;
}

.circle.one {
  width: 130px;
  height: 130px;
  top: 130px;
  right: -40px;
}

.circle.two {
  width: 80px;
  height: 80px;
  top: 10px;
  right: 30px;
}

.contact-form:before {
  content: "";
  position: absolute;
  width: 26px;
  height: 26px;
  background-color: #FBA11D;
  transform: rotate(45deg);
  top: 30px;
  left: -13px;
}
```

```css
form {
  padding: 2.3rem 2.2rem;
  z-index: 10;
  overflow: hidden;
  position: relative;
}

.title {
  color: #fff;
  font-weight: 500;
  font-size: 1.5rem;
  line-height: 1;
  margin-bottom: 0.7rem;
}

.input-container {
  position: relative;
  margin: 1rem 0;
}

.input {
  width: 100%;
  outline: none;
  border: 2px solid #fafafa;
  background: none;
  padding: 0.6rem 1.2rem;
  color: #fff;
  font-weight: 500;
  font-size: 0.95rem;
  letter-spacing: 0.5px;
  border-radius: 25px;
  transition: 0.3s;
}

textarea.input {
  padding: 0.8rem 1.2rem;
  min-height: 150px;
  border-radius: 22px;
  resize: none;
  overflow-y: auto;
}
```

```css
.input-container label {
  position: absolute;
  top: 50%;
  left: 15px;
  transform: translateY(-50%);
  padding: 0 0.4rem;
  color: #fafafa;
  font-size: 0.9rem;
  font-weight: 400;
  pointer-events: none;
  z-index: 1000;
  transition: 0.5s;
}

.input-container.textarea label {
  top: 1rem;
  transform: translateY(0);
}

.btn {
  padding: 0.6rem 1.3rem;
  background-color: #fff;
  border: 2px solid #fafafa;
  font-size: 0.95rem;
  color: #CC5500;
  line-height: 1;
  border-radius: 25px;
  outline: none;
  cursor: pointer;
  transition: 0.3s;
  margin: 0;
}

.btn:hover {
  background-color: transparent;
  color: #fff;
}

.input-container span {
  position: absolute;
  top: 0;
  left: 25px;
  transform: translateY(-50%);
```

```
  font-size: 0.8rem;
  padding: 0 0.4rem;
  color: transparent;
  pointer-events: none;
  z-index: 500;
}

.input-container span:before,
.input-container span:after {
  content: "";
  position: absolute;
  width: 10%;
  opacity: 0;
  transition: 0.3s;
  height: 5px;
  background-color: #FBA11D;
  top: 50%;
  transform: translateY(-50%);
}

.input-container span:before {
  left: 50%;
}

.input-container span:after {
  right: 50%;
}

.input-container.focus label {
  top: 0;
  transform: translateY(-50%);
  left: 25px;
  font-size: 0.8rem;
}

.input-container.focus span:before,
.input-container.focus span:after {
  width: 50%;
  opacity: 1;
}

.contact-info {
  padding: 2.3rem 2.2rem;
```

```
  position: relative;
}

.contact-info .title {
  color: #FBA11D;
}

.text {
  color: #333;
  margin: 1.5rem 0 2rem 0;
}

.information {
  display: flex;
  color: #555;
  margin: 0.7rem 0;
  align-items: center;
  font-size: 0.95rem;
}

.icon {
  width: 28px;
  margin-right: 0.7rem;
}

.social-media {
  padding: 2rem 0 0 0;
}

.social-media p {
  color: #333;
}

.social-icons {
  display: flex;
  margin-top: 0.5rem;
}

.social-icons a {
  width: 35px;
  height: 35px;
  border-radius: 5px;
  background: linear-gradient(45deg, #333, #161616);
```

```
  color: #fff;
  text-align: center;
  line-height: 35px;
  margin-right: 0.5rem;
  transition: 0.3s;
}

.social-icons a:hover {
  transform: scale(1.05);
}

.contact-info:before {
  content: "";
  position: absolute;
  width: 110px;
  height: 100px;
  border: 22px solid #FBA11D;
  border-radius: 50%;
  bottom: -77px;
  right: 50px;
  opacity: 0.4;
}

.big-circle {
  position: absolute;
  width: 500px;
  height: 500px;
  border-radius: 50%;
  background: linear-gradient(to bottom, #FBA11D, #FFBF00);
  bottom: 48%;
  right: 45%;
  transform: translate(-40%, 38%);
}

.big-circle:after {
  content: "";
  position: absolute;
  width: 360px;
  height: 380px;
  background-image: url(https://wallpaper-mania.com/wp-
content/uploads/2018/09/High_resolution_wallpaper_background_ID_77701605548.jpg);
  border-radius: 50%;
  top: calc(50% - 180px);
```

```
  left: calc(50% - 180px);
}

@media (max-width: 850px) {
  .form {
    grid-template-columns: 1fr;
  }

  .contact-info:before {
    bottom: initial;
    top: -75px;
    right: 65px;
    transform: scale(0.95);
  }

  .contact-form:before {
    top: -13px;
    left: initial;
    right: 70px;
  }

  .big-circle {
    bottom: 75%;
    transform: scale(0.9) translate(-40%, 30%);
    right: 50%;
  }

  .text {
    margin: 1rem 0 1.5rem 0;
  }

  .social-media {
    padding: 1.5rem 0 0 0;
  }
}

@media (max-width: 480px) {
  .container {
    padding: 1.5rem;
  }

  .contact-info:before {
    display: none;
```

```
  }

  .square,
  .big-circle {
    display: none;
  }

  form,
  .contact-info {
    padding: 1.7rem 1.6rem;
  }

  .text,
  .information,
  .social-media p {
    font-size: 0.8rem;
  }

  .title {
    font-size: 1.15rem;
  }

  .social-icons a {
    width: 30px;
    height: 30px;
    line-height: 30px;
  }

  .icon {
    width: 23px;
  }

  .input {
    padding: 0.45rem 1.2rem;
  }

  .btn {
    padding: 0.45rem 1.2rem;
  }
}
.wrapper{
    position: absolute;
    bottom: 45px;
```

```
    left: 1%;
}
.wrapper ul li{
    list-style-type: none;
    padding: 2px;
    border-radius: 50%;
    margin: 5px 10px;
}
.wrapper ul li .fab {
    font-size: 27px;
    line-height: 50px;
    transition: .5s;
}
.wrapper ul li .fa:hover {
    color: #ffffff;
}
.wrapper ul li a {
    position: relative;
    display: block;
    width: 50px;
    height: 50px;
    border-radius: 50%;
    color: white;
    text-align: center;
    transition: 0.5s;
    transform: translate(0,0px);
    box-shadow: 0px 7px 5px rgba(0, 0, 0, 0.5);
}
.wrapper ul li a:hover {
    transform: rotate(0deg) skew(0deg) translate(0, -10px);
}
.wrapper ul li:nth-child(1){
    background: #3b5998;
}
.wrapper ul li:nth-child(2){
    background-image: linear-gradient(#f9ce34,  #ee2a7b , #6228d7);
}
.wrapper ul li:nth-child(3){
    background: #00aced;
}
.wrapper ul li:nth-child(4){
    background: #cb2017;
}
```

```
.wrapper ul li:nth-child(5){
    background: #0e76A8;
}
.wrapper ul li:nth-child(6){
    background: #dd4b39;
}
</style>
  </head>
  <body>
<section class="home">
    <div class="headerbg ">
      <header>
        <div class="container ">
          <div class="navbar flex1">
            <div class="logo">
              <img src="https://seeklogo.com/images/M/movie-time-cinema-logo-
8B5BE91828-seeklogo.com.png" alt="">
            </div>
            <div class="nav-links">
<ul>
            <li><a href="index.html" target="_self">HOME</a></li>
            <li><a href="about.html">ABOUT</a></li>
            <li><a href="trending.html" target="_self">TRENDING MOVIES</a></li>
            <li><a href="contact.html" target="_self">CONTACT US</a></li>
            <li><a href="movie.html" target="_self">REVIEW PREDICTION</a></li>
            <li><a href="guide.html" target="_self">PARENTAL GUIDE</a></li>
</ul>
</div>

            <div class="subscribe flex">
              <a href ="http://127.0.0.1:5000/"><i class="fas fa-search"
style="color: white; margin-top: 1rem;"></i></a>
              <a href ="login.html"><i id="palybtn" class="fas fa-user"
style="color: white; margin-top: 0.1rem;"></i></a>
              <a href="movie.html" class="button">SUBSCRIBE</a>
            </div>
          </div>
        </div>
      </header>
<div class="container123">
      <span class="big-circle"></span>
<div class="form">
<div class="contact-info">
```

```
  <h3 class="title">Let's get in touch</h3>
  <p class="text">
    We would love to hear from you! Contact us for any inquiries,
    feedback, or suggestions regarding our movie website.
  </p>

  <div class="info">
    <div class="information">
      <img src="img/location.png" class="icon" alt="" />
      <p>Computer Engineering Department, UET Taxila</p>
    </div>
    <div class="information">
      <img src="img/email.png" class="icon" alt="" />
      <p>atifaimen888@gmail.com <br> shahrbanosanii@gmail.com</p>
    </div>
    <div class="information">
      <img src="img/phone.png" class="icon" alt="" />
      <p>123-456-789</p>
    </div>
  </div>

  <div class="social-media">
    <p>Connect with us :</p>
    <div class="social-icons">
      <a href="#">
        <i class="fab fa-facebook-f"></i>
      </a>
      <a href="#">
        <i class="fab fa-twitter"></i>
      </a>
      <a href="#">
        <i class="fab fa-instagram"></i>
      </a>
      <a href="#">
        <i class="fab fa-linkedin-in"></i>
      </a>
    </div>
  </div>
</div>

<div class="contact-form">
  <span class="circle one"></span>
  <span class="circle two"></span>
```

```
          <form action="" autocomplete="off">
            <h3 class="title">Contact us</h3>
            <div class="input-container">
              <input type="text" name="name" class="input" />
              <label for="">Username</label>
              <span>Username</span>
            </div>
            <div class="input-container">
              <input type="email" name="email" class="input" />
              <label for="">Email</label>
              <span>Email</span>
            </div>
            <div class="input-container">
              <input type="tel" name="phone" class="input" />
              <label for="">Phone</label>
              <span>Phone</span>
            </div>
            <div class="input-container textarea">
              <textarea name="message" class="input"></textarea>
              <label for="">Message</label>
              <span>Message</span>
            </div>
            <input type="submit" value="Send" class="btn" />
          </form>
        </div>
      </div>
    </div>
</section>
<div class="footer">
    <div class="wrapper">
      <ul>
            <li><a href="https://www.facebook.com/"><i class="fab fa-facebook-
f"></i></a></li>
            <li><a href="https://www.instagram.com/"><i class="fab fa-
instagram"></i></a></li>
            <li><a href="https://www.twitter.com/"><i class="fab fa-
twitter"></i></a></li>
            <li><a href="https://www.pinterest.com/"><i class="fab fa-
pinterest"></i></a></li>
            <li><a href="https://www.linkedin.com/"><i class="fab fa-linkedin-
in"></i></a></li>
```

```
            <li><a href="https://www.google.com/"><i class="fab fa-google-plus-
g"></i></a></li>
</ul>
  </div>
</div>

    <script>
const inputs = document.querySelectorAll(".input");

function focusFunc() {
  let parent = this.parentNode;
  parent.classList.add("focus");
}

function blurFunc() {
  let parent = this.parentNode;
  if (this.value == "") {
    parent.classList.remove("focus");
  }
}

inputs.forEach((input) => {
  input.addEventListener("focus", focusFunc);
  input.addEventListener("blur", blurFunc);
});
</script>
  </body>
</html>
```

- **REVIEW PREDICTION**

```
<!DOCTYPE html>
<html>
<head>
<title>Review Predictor System</title>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-
fit=no">
<!-- Google Fonts -->
  <link href='https://fonts.googleapis.com/css?family=Poppins' rel='stylesheet'>

   <!-- Font Awesome -->
```

```
<link rel="stylesheet"
href="https://pro.fontawesome.com/releases/v5.10.0/css/all.css"
integrity="sha384-
AYmEC3Yw5cVb3ZcuHtOA93w35dYTsvhLPVnYs9eStHfGJvOvKxVfELGroGkvsg+p"
crossorigin="anonymous" />
<style>
*{
    padding: 0;
    margin: 0;
    box-sizing: border-box;
}
body{
    width: 100%;
    height: 100%;
    color: white;
    background-color: #000;
    font-family: Poppins;
}
.nav-links{
flex: 1;
text-align: center;
}
.nav-links ul li{
list-style: none;
display: inline-block;
padding: 35px 15px;
position: relative;
margin-bottom: 105px;
}
.nav-links ul li a{
color: #fff;
text-decoration: none;
font-size: 15.5px;
font-family: Poppins;
}
.nav-links ul li::after{
content: '';
width: 0%;
height: 2px;
background: #FBA11D;
display: block;
margin: auto;
transition: 0.5s;
```

```
}
.nav-links ul li:hover::after{
width: 100%;
}

/*---------global------*/
.container1 {
  max-width: 95%;
  margin: auto;
}

a {
  text-decoration: none;
  transition: 0.5s;
}

img {
  width: 100%;
  height: 100%;
}

ul {
  list-style: none;
}

.flex {
  display: flex;
}

.flex1 {
  display: flex;
  justify-content: space-between;
}

i {
  cursor: pointer;
  transition: 0.5s;
}

/*---------global------*/
/*---------header------*/
.logo img {
  width: 118px;
```

```
  height: 100%;
  margin-bottom: 110px;
  margin-left: 70px;
}

header {
  height: 12vh;
  background: rgba(0, 0, 0, 0.2);
}

header .navbar {
  align-items: center;
  transition: 0.5s;
}

.subscribe i:nth-child(1) {
  margin-top: 17px;
}

.subscribe i {
  font-size: 20px;
  margin-right: 20px;
}

#palybtn {
  background: #FBA11D;
  width: 50px;
  height: 50px;
  color: #fff;
  line-height: 50px;
  text-align: center;
  border-radius: 50%;
  margin-right: 10px;
  margin-bottom: 110px;
}

.button {
  background: #FF8E11;
  border: none;
  outline: none;
  height: 53px;
  color: white;
  font-size: 16px;
```

```
  padding: 15px 30px;
  cursor: pointer;
  font-family: Poppins;
}

.subscribe button {
  margin-left: 15px;
}

header.sticky .headerbg {
  position: relative;
}

header.sticky {
  z-index: 10;
  position: fixed;
  top: 0;
  width: 100%;
  background: #000;
  transition: 0.5s;
  height: 10vh;
  box-shadow: 2px 2px 2px 1px rgba(0, 0, 0, 0.1);
  transition: 0.5s;
}

header.sticky .subscribe {
  display: none;
}

.container {
    max-width: 600px;
    margin: 0 auto;
    padding: 70px;
}
form {
    margin-bottom: 20px;
}
#result {
    text-align: center;
}

textarea {
    width: 100%;
```

```
}
h2 {
    margin-top: 16px;
    font-weight: bold;
}

.star-widget {
    display: flex;
    justify-content: center;
    margin-top: 20px;
}

.star-widget input {
    display: none;
}

.star-widget label {
    font-size: 30px;
    color: #bdbdbd;
    cursor: pointer;
    transition: color .2s;
}

.star-widget label:hover,
.star-widget label:hover ~ label,
.star-widget input:checked ~ label {
    color: gold;
}

.back-video{
position: absolute;
right: 0;
bottom: 0;
top: 0;
left: 0;
z-index: -1;
opacity: 0.3;
}

@media (min-aspect-ratio: 16/9) {
.back-video{
width: 100%;
height: auto;
```

```
}
}

@media (max-aspect-ratio: 16/9) {
.back-video{
width: auto;
height: 100%;
}
}

@media only screen and (max-width:768px) {
  header nav ul {
    position: absolute;
    top: 100px;
    left: 0;
    width: 100%;
    background: #000;
    overflow: hidden;
    transition: max-height 0.5s;
    z-index: 100;
  }

  header nav ul li {
    display: block;
    margin: 50px;
    margin-top: 10px;
    margin-bottom: 10px;
  }

  header .navbar span {
    color: white;
    display: block;
    cursor: pointer;
  }

  .home {
    background-repeat: no-repeat;
    background-size: cover;
    background-position: center;
    height: 70vh;
  }

  .subscribe {
```

```
    display: none;
  }
}

.footer{
    font-size: 15px;
    color: #777;
    text-align: center;
}
.wrapper{
    position: absolute;
    bottom: 45px;
    left: 1%;
}
.wrapper ul li{
    list-style-type: none;
    padding: 2px;
    border-radius: 50%;
    margin: 5px 10px;
}
.wrapper ul li .fab {
    font-size: 27px;
    line-height: 50px;
    transition: .5s;
}
.wrapper ul li .fa:hover {
    color: #ffffff;
}
.wrapper ul li a {
    position: relative;
    display: block;
    width: 50px;
    height: 50px;
    border-radius: 50%;
    color: white;
    text-align: center;
    transition: 0.5s;
    transform: translate(0,0px);
    box-shadow: 0px 7px 5px rgba(0, 0, 0, 0.5);
}
.wrapper ul li a:hover {
    transform: rotate(0deg) skew(0deg) translate(0, -10px);
}
```

```css
.wrapper ul li:nth-child(1){
    background: #3b5998;
}
.wrapper ul li:nth-child(2){
    background-image: linear-gradient(#f9ce34,  #ee2a7b , #6228d7);
}
.wrapper ul li:nth-child(3){
    background: #00aced;
}
.wrapper ul li:nth-child(4){
    background: #cb2017;
}
.wrapper ul li:nth-child(5){
    background: #0e76A8;
}
.wrapper ul li:nth-child(6){
    background: #dd4b39;
}

/* ------------------- */
.subscribe123{
    width: 100%;
    height: 105px;
    display: flex;
    justify-content: space-around;
    align-items: center;
    background-size: cover;
    background-position: center;
    background-repeat: no-repeat;
    padding: 0 10%;
}

.subscribe123 .subs-content{
    width: 600px;
}
.subs-content h3{
    color: #fff;
    font-size: 1.6rem;
    font-weight: 500;
}
.subscribe .subs-btn{
    display: flex;
    justify-content: center;
```

```
    align-items: center;
}
.subs-btn .email{
    width: 225px;
    height: 40px;
    background-color: transparent;
    font-family: Poppins;
    color: #d9d9d9;
    padding: 0 1rem;
    border: 2px solid #FF8E11;
    border-top-left-radius: 6px;
    border-bottom-left-radius: 6px;
    position: relative;
}
.subs-btn .email:focus{
    border: 2px solid #FF8E11;
    outline: none;
}
.subs-btn .email::placeholder{
    color: var(--comment);
}

.subs-btn .input-btn{
    position: relative;
    left: -5px;
    width: 125px;
    height: 40px;
    background-color: #FF8E11;
    color: #fff;
    font-family: Poppins;
    font-weight: 600;
    border: none;
    outline: none;
    border-top-right-radius: 6px;
    border-bottom-right-radius: 6px;
    cursor: pointer;
    transition: all 0.25s ease-in-out;
    -webkit-transition: all 0.25s ease-in-out;
    -moz-transition: all 0.25s ease-in-out;
    -ms-transition: all 0.25s ease-in-out;
    -o-transition: all 0.25s ease-in-out;
}
```

```
.subs-btn .input-btn:hover{
    background-color: #d9d9d9;
    color: #000;
}
</style>
</head>
<body>
<section class="home">
    <div class="headerbg ">
      <header>
        <div class="container1 ">
          <div class="navbar flex1">
            <div class="logo">
              <img src="https://seeklogo.com/images/M/movie-time-cinema-logo-
8B5BE91828-seeklogo.com.png" alt="">
            </div>
            <div class="nav-links">
<ul>
            <li><a href="index.html" target="_self">HOME</a></li>
            <li><a href="about.html">ABOUT</a></li>
            <li><a href="trending.html" target="_self">TRENDING MOVIES</a></li>
            <li><a href="contact.html" target="_self">CONTACT US</a></li>
            <li><a href="movie.html" target="_self">REVIEW PREDICTION</a></li>
            <li><a href="guide.html" target="_self">PARENTAL GUIDE</a></li>
</ul>
</div>
            <div class="subscribe flex">
              <a href ="http://127.0.0.1:5000/"><i class="fas fa-search"
style="color: white; margin-top: 1rem;"></i></a>
              <a href ="login.html"><i id="palybtn" class="fas fa-user"
style="color: white; margin-top: 0.1rem;"></i></a>
              <a href="movie.html" class="button">SUBSCRIBE</a>
            </div>
          </div>
        </div>
      </header>
<center><b><br><br>
    <h1 style="font-family: 'Poppins'; color: #fff; font-size: 60px";>Predicting
Your Reviews Here!!!</h1></b></center>
<video autoplay loop muted plays-inline class="back-video">
<source src="img/video.mp4" type="video/mp4">
</video>
<div class="container">
```

```
        <form id="reviewForm">
            <label for="review">Enter your movie review:</label><br>
            <textarea id="review" name="review" rows="4"
cols="50"></textarea><br><br>
            <input type="submit" value="Predict" class="button">
        </form>
        <div id="result">
            <h3 id="sentimentResult"></h3>
        </div>
        <div class="star-widget">
            <input type="radio" name="rate" id="rate-5">
            <label for="rate-5" class="fas fa-star"></label>
            <input type="radio" name="rate" id="rate-4">
            <label for="rate-4" class="fas fa-star"></label>
            <input type="radio" name="rate" id="rate-3">
            <label for="rate-3" class="fas fa-star"></label>
            <input type="radio" name="rate" id="rate-2">
            <label for="rate-2" class="fas fa-star"></label>
            <input type="radio" name="rate" id="rate-1">
            <label for="rate-1" class="fas fa-star"></label>
        </div>
    </div>
<div class="subscribe123">
        <div class="subs-content">
            <h3>Subscribe to the newsletter and be the first to know about new
releases of your favourite series and movies</h3>
        </div>
        <form class="subs-btn" method="post" action="">
            <input type="email" class="email" placeholder="Your email
address...">
            <input type="submit" value="Subscribe now" class="input-btn">
        </div>
    </div>
</section>
<div class="footer">
    <div class="wrapper">
      <ul>
            <li><a href="https://www.facebook.com/"><i class="fab fa-facebook-
f"></i></a></li>
            <li><a href="https://www.instagram.com/"><i class="fab fa-
instagram"></i></a></li>
            <li><a href="https://www.twitter.com/"><i class="fab fa-
twitter"></i></a></li>
```

```
            <li><a href="https://www.pinterest.com/"><i class="fab fa-
pinterest"></i></a></li>
            <li><a href="https://www.linkedin.com/"><i class="fab fa-linkedin-
in"></i></a></li>
            <li><a href="https://www.google.com/"><i class="fab fa-google-plus-
g"></i></a></li>
</ul>
  </div>
</div>
<script>
// Define a variable to store the sentiment prediction
let sentiment;

document.getElementById('reviewForm').addEventListener('submit', function (event)
{
    event.preventDefault();
    const userReview = document.getElementById('review').value;

    if (!sentiment) {
        sentiment = predictReviewSentiment(userReview);
    }

    document.getElementById('sentimentResult').textContent = `The sentiment of
your review is: ${sentiment}`;

    const starIcons = document.querySelectorAll('.star-widget label');
    starIcons.forEach((icon, index) => {
       if (index < 2) {
          if (sentiment === 'negative') {
      icon.style.color = 'gold'; // Set the first two stars to yellow for
negative sentiment
    }      else {
      icon.style.color = 'gold'; // Set the first two stars to yellow for
positive sentiment
    }
  }        else {
           if (sentiment === 'positive') {
      icon.style.color = 'gold'; // Set the rest of the stars to yellow for
positive sentiment
    }
  }
    });
});
```

```
function predictReviewSentiment(userReview) {
    const sentiments = ['positive', 'negative'];
    const randomIndex = Math.floor(Math.random() * sentiments.length);
    return sentiments[randomIndex];
}

</script>
</body>
</html>
```

- **TRENDING MOVIES**

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Trending Movies</title>
    <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/swiper@9/swiper-
bundle.min.css" />
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.2.1/css/all.min.css" integrity="sha512-
MV7K8+y+gLIBoVD59lQIYicR65iaqukzvf/nwasF0nqhPay5w/9lJmVM2hMDcnK1OnMGCdVK+iQrJ7lzP
JQd1w==" crossorigin="anonymous" referrerpolicy="no-referrer" />
    <!-- Google Fonts -->
    <link href='https://fonts.googleapis.com/css?family=Poppins'
rel='stylesheet'>
    <!-- Font Awesome -->
    <link rel="stylesheet"
href="https://pro.fontawesome.com/releases/v5.10.0/css/all.css"
integrity="sha384-
AYmEC3Yw5cVb3ZcuHtOA93w35dYTsvhLPVnYs9eStHfGJvOvKxVfELGroGkvsg+p"
crossorigin="anonymous" />
<style>
*{
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: Poppins;
}
```

```
body{
background: url(https://www.desktopbackground.org/p/2010/12/24/131037_download-
black-wide-and-hd-wallpapers_1366x768_h.jpg);
}

section{
    width: 100%;
    height: 100vh;
}

.swiper {
    width: 95%;
    height: 85%;
    border-radius: 20px;
    overflow: hidden;
  }

.swiper::selection{
    user-select: none;
  }

.swiper-slide {
    text-align: center;
    font-size: 18px;
    background: #fff;
    display: flex;
    justify-content: center;
    align-items: center;
 }

.swiper-slide .image .black{
    width: 100%;
    height: 130%;
    position: absolute;
    background: linear-gradient(to bottom, rgba(0,0,0,0.0),rgba(0,0,0,0.9));
  }

.swiper-slide .image .black h1{
    font-size: 30px;
    text-align: left;
    position: relative;
    top: 50%;
    left: 5%;
```

```
    color: #fff;
  }

.swiper-slide .image .black .star{
    color: #ffd500;
    text-align: left;
    font-size: 16px;
    margin: 10px 0;
    position: relative;
    top: 50%;
    left: 5%;
  }

.swiper-slide .image .black p{
    width: 550px;
    text-align: justify;
    font-size: 16px;
    color: #cccccc;
    margin: 10px 0;
    position: relative;
    top: 50%;
    left: 5%;
    line-height: 23px;
    font-family: Poppins;
  }

.swiper-slide .image .black .genre{
    text-align: left;
    font-size: 16px;
    margin: 13px 0;
    position: relative;
    top: 50%;
    left: 5%;
  }

.swiper-slide .image .black .genre a.category{
    background: #282931;
    text-decoration: none;
    color: #cccccc;
    padding: 2px 7px;
  }

.swiper-slide .image .black .genre a.category span{
```

```
    color: #ffe100;
  }

.swiper-slide .image .black .watch{
    width: 700px;
    text-align: left;
    display: flex;
    font-size: 16px;
    margin: 10px 0;
    position: relative;
    top: 50%;
    left: 5%;
  }

.swiper-slide .image .black .watch i{
    width: 40px;
    height: 40px;
    line-height: 40px;
    font-size: 20px;
    text-align: center;
    background-color: #ffe100;
    border-radius: 50%;
    cursor: pointer;
  }

.swiper-slide .image .black .watch p{
    position: relative;
    left: 15px;
    font-family: Poppins;
  }

.swiper-slide img {
    display: block;
    width: 100%;
    height: 100%;
    object-fit: cover;
  }

.swiper {
    margin-left: auto;
    margin-right: auto;
  }
```

```
/*--------------*/

.second{
  width: 100%;
  height: auto;
  margin: 0 auto;
  padding-bottom: 25px;
  background: url(https://www.desktopbackground.org/p/2010/12/24/131037_download-
black-wide-and-hd-wallpapers_1366x768_h.jpg);
}

.second .latest{
  width: 95%;
  margin: 0 auto;
}

.second .latest h1{
  font-size: 45px;
  color: #fff;
  margin-bottom: 20px;
}

.second .latest .box{
  display: flex;
  align-items: center;
  justify-content: space-between;
}

.second .latest .box .card{
  width: 250px;
  height: 370px;
  margin: 10px;
  overflow: hidden;
  border-radius: 8px;
}

.second .latest .box .card .details{
  width: 250px;
  height: 370px;
  overflow: hidden;
  position: absolute;
  border-radius: 8px;
  transition: 0.3s;
```

```
  opacity: 0;
}

.second .latest .box .card:hover .details{
  opacity: 1;
  background: linear-gradient(to bottom, rgba(0,0,0,0.0), rgba(0,0,0,0.7));
}

.second .latest .box .card .details .left{
  position: absolute;
  bottom: 0;
  color: #fff;
  transform: translateX(-100px);
  transition: 0.4s;
  opacity: 0;
}

.second .latest .box .card:hover .details .left{
  opacity: 1;
  transform: translateX(0);
}

.second .latest .box .card .details .left p.name{
  width: 75%;
  font-size: 18px;
  font-weight: bold;
  margin-left: 10px;
  margin-bottom: 10px;
}

.second .latest .box .card .details .left .date_quality{
  display: flex;
  align-items: center;
}

.second .latest .box .card .details .left .date_quality p.quality{
  width: 30px;
  font-size: 13px;
  background: #ffe100;
  color: #282931;
  margin-left: 10px;
  margin-bottom: 8px;
  text-align: center;
```

```
  font-weight: bold;
}
.second .latest .box .card .details .left .date_quality p.date{
  color: #fff;
  font-size: 15px;
  font-weight: bold;
  text-align: left;
  margin-left: 10px;
  margin-bottom: 8px;
}

.second .latest .box .card .details .left p.category{
  margin-left: 10px;
  font-size: 13px;
  margin-bottom: 8px;
}

.second .latest .box .card .details .left .info{
  display: flex;
  align-items: center;
  margin-bottom: 10px;
}

.second .latest .box .card .details .left .info .rate{
  display: flex;
  align-items: center;
  margin-left: 10px;
  margin-right: 10px;
}

.second .latest .box .card .details .left .info .rate i{
  font-size: 12px;
  color: #ffd500;
  margin-right: 5px;
}

.second .latest .box .card .details .left .info .rate p{
  font-size: 12px;
}

.second .latest .box .card .details .left .info .time{
  display: flex;
  align-items: center;
```

```
}

.second .latest .box .card .details .left .info .time i{
  font-size: 12px;
  color: #ffd500;
  margin-left: 5px;
}

.second .latest .box .card .details .left .info .time p{
  font-size: 12px;
  margin-left: 5px;
}

.second .latest .box .card .details .right{
  position: absolute;
  right: 0;
  bottom: 0;
  color: #fff;
  transform: translateX(100px);
  transition: 0.4s;
  opacity: 0;
}

.second .latest .box .card:hover .details .right{
  opacity: 1;
  transform: translateX(0);
}
.second .latest .box .card .details .right i{
  width: 40px;
  height: 40px;
  line-height: 40px;
  font-size: 20px;
  text-align: center;
  background-color: #ffe100;
  border-radius: 50%;
  cursor: pointer;
  color: #282931;
  margin: 0 10px 8px 0;
}

.second .latest .box .card img{
  width: 100%;
  height: 100%;
```

```
  object-fit: cover;
  object-position: center;
}

/*---------*/

.upcoming{
  width: 100%;
  height: auto;
  margin: 0 auto;
  padding-bottom: 25px;
  background: url(https://www.desktopbackground.org/p/2010/12/24/131037_download-
black-wide-and-hd-wallpapers_1366x768_h.jpg);
}

.upcoming .movies_box{
  width: 95%;
  margin: 0 auto;
}

.upcoming .movies_box h1{
  font-size: 45px;
  color: #fff;
  margin-bottom: 20px;
}
.upcoming .movies_box .box{
  display: flex;
  align-items: center;
  justify-content: space-between;
}

.upcoming .movies_box .box .card{
  width: 250px;
  height: 370px;
  margin: 10px;
  overflow: hidden;
  border-radius: 8px;
}

.upcoming .movies_box .box .card .details{
  width: 250px;
  height: 370px;
  overflow: hidden;
```

```
  position: absolute;
  border-radius: 8px;
  transition: 0.3s;
  opacity: 0;
}

.upcoming .movies_box .box .card:hover .details{
  opacity: 1;
  background: linear-gradient(to bottom, rgba(0,0,0,0.0), rgba(0,0,0,0.7));
}

.upcoming .movies_box .box .card .details .left{
  position: absolute;
  bottom: 0;
  color: #fff;
  transform: translateX(-100px);
  transition: 0.4s;
  opacity: 0;
}

.upcoming .movies_box .box .card:hover .details .left{
  opacity: 1;
  transform: translateX(0);
}

.upcoming .movies_box .box .card .details .left p.name{
  width: 75%;
  font-size: 18px;
  font-weight: bold;
  margin-left: 10px;
  margin-bottom: 10px;
}

.upcoming .movies_box .box .card .details .left .date_quality{
  display: flex;
  align-items: center;
}

.upcoming .movies_box .box .card .details .left .date_quality p.quality{
  width: 30px;
  font-size: 13px;
  background: #ffe100;
  color: #282931;
```

```
  margin-left: 10px;
  margin-bottom: 8px;
  text-align: center;
  font-weight: bold;
}

.upcoming .movies_box .box .card .details .left .date_quality p.date{
  color: #fff;
  font-size: 15px;
  font-weight: bold;
  text-align: left;
  margin-left: 10px;
  margin-bottom: 8px;
}

.upcoming .movies_box .box .card .details .left p.category{
  margin-left: 10px;
  font-size: 13px;
  margin-bottom: 8px;
}

.upcoming .movies_box .box .card .details .left .info{
  display: flex;
  align-items: center;
  margin-bottom: 10px;
}

.upcoming .movies_box .box .card .details .left .info .time{
  display: flex;
  align-items: center;
}

.upcoming .movies_box .box .card .details .left .info .time i{
  font-size: 12px;
  color: #ffd500;
  margin-left: 5px;
}

.upcoming .movies_box .box .card .details .left .info .time p{
  font-size: 12px;
  margin-left: 5px;
}
```

```css
.upcoming .movies_box .box .card .details .right{
  position: absolute;
  right: 0;
  bottom: 0;
  color: #fff;
  transform: translateX(100px);
  transition: 0.4s;
  opacity: 0;
}

.upcoming .movies_box .box .card:hover .details .right{
  opacity: 1;
  transform: translateX(0);
}

.upcoming .movies_box .box .card .details .right i{
  width: 40px;
  height: 40px;
  line-height: 40px;
  font-size: 20px;
  text-align: center;
  background-color: #ffe100;
  border-radius: 50%;
  cursor: pointer;
  color: #282931;
  margin: 0 10px 8px 0;
}

.upcoming .movies_box .box .card img{
  width: 100%;
  height: 100%;
  object-fit: cover;
  object-position: center;
}



/*----------*/

.show{
  width: 100%;
  height: auto;
  margin: 0 auto;
```

```
  padding-bottom: 25px;
  background: url(https://www.desktopbackground.org/p/2010/12/24/131037_download-
black-wide-and-hd-wallpapers_1366x768_h.jpg);
}

.show .tv_show{
  width: 95%;
  margin: 0 auto;
}

.show .tv_show h1{
  font-size: 45px;
  color: #fff;
  margin-bottom: 20px;
}

.show .tv_show .box{
  display: flex;
  align-items: center;
  justify-content: space-between;
}

.show .tv_show .box .card{
  width: 250px;
  height: 370px;
  margin: 10px;
  overflow: hidden;
  border-radius: 8px;
}

.show .tv_show .box .card .details{
  width: 250px;
  height: 370px;
  overflow: hidden;
  position: absolute;
  border-radius: 8px;
  transition: 0.3s;
  opacity: 0;
}

.show .tv_show .box .card:hover .details{
  opacity: 1;
  background: linear-gradient(to bottom, rgba(0,0,0,0.0), rgba(0,0,0,0.7));
```

```
}

.show .tv_show .box .card .details .left{
  position: absolute;
  bottom: 0;
  color: #fff;
  transform: translateX(-100px);
  transition: 0.4s;
  opacity: 0;
}

.show .tv_show .box .card:hover .details .left{
  opacity: 1;
  transform: translateX(0);
}

.show .tv_show .box .card .details .left p.name{
  width: 75%;
  font-size: 18px;
  font-weight: bold;
  margin-left: 10px;
  margin-bottom: 10px;
}

.show .tv_show .box .card .details .left .date_quality{
  display: flex;
  align-items: center;
}

.show .tv_show .box .card .details .left .date_quality p.quality{
  width: 30px;
  font-size: 13px;
  background: #ffe100;
  color: #282931;
  margin-left: 10px;
  margin-bottom: 8px;
  text-align: center;
  font-weight: bold;
}

.show .tv_show .box .card .details .left .date_quality p.date{
  color: #fff;
  font-size: 15px;
```

```
  font-weight: bold;
  text-align: left;
  margin-left: 10px;
  margin-bottom: 8px;
}

.show .tv_show .box .card .details .left p.category{
  margin-left: 10px;
  font-size: 13px;
  margin-bottom: 8px;
}

.show .tv_show .box .card .details .left .info{
  display: flex;
  align-items: center;
  margin-bottom: 10px;
}

.show .tv_show .box .card .details .left .info .rate{
  display: flex;
  align-items: center;
  margin-left: 10px;
  margin-right: 10px;
}

.show .tv_show .box .card .details .left .info .rate i{
  font-size: 12px;
  color: #ffd500;
  margin-right: 5px;
}

.show .tv_show .box .card .details .left .info .rate p{
  font-size: 12px;
}

.show .tv_show .box .card .details .left .info .time{
  display: flex;
  align-items: center;
}

.show .tv_show .box .card .details .left .info .time i{
  font-size: 12px;
  color: #ffd500;
```

```
  margin-left: 5px;
}

.show .tv_show .box .card .details .left .info .time p{
  font-size: 12px;
  margin-left: 5px;
}

.show .tv_show .box .card .details .right{
  position: absolute;
  right: 0;
  bottom: 0;
  color: #fff;
  transform: translateX(100px);
  transition: 0.4s;
  opacity: 0;
}

.show .tv_show .box .card:hover .details .right{
  opacity: 1;
  transform: translateX(0);
}

.show .tv_show .box .card .details .right i{
  width: 40px;
  height: 40px;
  line-height: 40px;
  font-size: 20px;
  text-align: center;
  background-color: #ffe100;
  border-radius: 50%;
  cursor: pointer;
  color: #282931;
  margin: 0 10px 8px 0;
}

.show .tv_show .box .card img{
  width: 100%;
  height: 100%;
  object-fit: cover;
  object-position: center;
}
```

```css
/*---------global------*/
.container {
  max-width: 95%;
  margin: auto;
}

a {
  text-decoration: none;
  transition: 0.5s;
}

img {
  width: 100%;
  height: 100%;
}

ul {
  list-style: none;
}

.flex {
  display: flex;
}

.flex1 {
  display: flex;
  justify-content: space-between;
}

i {
  cursor: pointer;
  transition: 0.5s;
}

.nav-links{
flex: 1;
text-align: center;
}

/*---------global------*/
/*---------header------*/
.home {
  color: white;
```

```
  height: 100vh;
  width: 100%;
}

.logo img {
  width: 110px;
  height: 100%;
  margin-bottom: 110px;
  margin-left: 70px;
}

.headerbg {
  height: 100vh;
  width: 100%;
  background: linear-gradient(to right, rgba(34, 31, 31, 1) 0%, rgba(34, 31, 31,
0.4) 100%)
}

header {
  height: 14vh;
  background: rgba(0, 0, 0, 0.2);
}

.nav-links ul li{
list-style: none;
display: inline-block;
padding: 35px 15px;
margin-bottom: 105px;
position: relative;
}

.nav-links ul li a{
color: #fff;
text-decoration: none;
font-size: 16px;
font-family: Poppins;
}

.nav-links ul li::after{
content: '';
width: 0%;
height: 2px;
background: #FBA11D;
```

```
display: block;
margin: auto;
transition: 0.5s;
}

.nav-links ul li:hover::after{
width: 100%;
}

header .navbar {
  align-items: center;
  transition: 0.5s;
}

.subscribe i:nth-child(1) {
  margin-top: 17px;
}

.subscribe i {
  font-size: 20px;
  margin-right: 20px;
}

#playbtn {
  background: #FBA11D;
  width: 50px;
  height: 50px;
  color: #fff;
  line-height: 50px;
  text-align: center;
  border-radius: 50%;
  margin-right: 10px;
  margin-bottom: 110px;
}

.button {
  background: #FBA11D;
  border: none;
  outline: none;
  height: 55px;
  color: white;
  padding: 15px 30px;
  cursor: pointer;
```

```
  font-family: Poppins;
}

.subscribe button {
  margin-left: 15px;
}

@media only screen and (max-width:768px) {
.home {
    background-repeat: no-repeat;
    background-size: cover;
    background-position: center;
    height: 70vh;
  }

  .subscribe {
    display: none;
  }
}
h1 {
  font-size: 80px;
  letter-spacing: 2px;
  line-height: 90px;
  text-transform: uppercase;
}
</style>
</head>
<body>
<section class="home">
    <div class="headerbg ">
      <header>
        <div class="container ">
          <div class="navbar flex1">
            <div class="logo">
              <img src="https://seeklogo.com/images/M/movie-time-cinema-logo-
8B5BE91828-seeklogo.com.png" alt="">
            </div>
            <div class="nav-links">
<ul>
            <li><a href="index.html" target="_self">HOME</a></li>
            <li><a href="about.html">ABOUT</a></li>
            <li><a href="trending.html" target="_self">TRENDING MOVIES</a></li>
            <li><a href="contact.html" target="_self">CONTACT US</a></li>
```

```
            <li><a href="movie.html" target="_self">REVIEW PREDICTION</a></li>
            <li><a href="guide.html" target="_self">PARENTAL GUIDE</a></li>
</ul>
             </div>
            <div class="subscribe flex">
              <a href ="http://127.0.0.1:5000/"><i class="fas fa-search"
style="color: white; margin-top: 1rem;"></i></a>
              <a href ="login.html"><i id="playbtn" class="fas fa-user"
style="color: white; margin-top: 0.1rem;"></i></a>
               <a href="movie.html" class="button">SUBSCRIBE</a>
            </div>
          </div>
        </div>
      </header>

          <div class="swiper mySwiper">
            <div class="swiper-wrapper">
              <div class="swiper-slide">
                <div class="image">
                    <div class="black">
                        <h1>Toy Story</h1>
                        <div class="star">
                            <i class="fa-solid fa-star"></i>
                            <i class="fa-solid fa-star"></i>
                            <i class="fa-solid fa-star"></i>
                            <i class="fa-solid fa-star-half-stroke"></i>
                            <i class="fa-regular fa-star"></i>
                        </div>
                        <p>
                           Woody, Buzz, and the rest of Andy's toys haven't been
played with in years. With Andy about to go to college, the gang find themselves
                           accidentally left at a nefarious day care center. The
toys must band together to escape and return home to Andy.
                        </p>
                        <div class="genre">
                            <a href="#" class="category">Animation</a>
                            <a href="#" class="category">Family</a>
                            <a href="#" class="category">Comedy</a>
                            <a href="#" class="category"><span>4K</span></a>
                        </div>
                        <a href=https://www.youtube.com/watch?v=v-PjgYDrg70"
style="color: white;">
                        <div class="watch">
```

```
                            <i class="fa-solid fa-play"></i>
                            <p>Watch the Trailer</p>
                        </div></a>
                    </div>
                    <img
src="https://images.wallpapersden.com/image/download/toy-story-
4_a2llamiUmZqaraWkpJRmbmdlrWZnZWU.jpg">
                </div>
            </div>
            <div class="swiper-slide">
              <div class="image">
                  <div class="black">
                      <h1>The Avengers</h1>
                      <div class="star">
                          <i class="fa-solid fa-star"></i>
                          <i class="fa-solid fa-star"></i>
                          <i class="fa-solid fa-star"></i>
                          <i class="fa-solid fa-star-half-stroke"></i>
                          <i class="fa-regular fa-star"></i>
                      </div>
                      <p>
                           Earth's mightiest heroes unite to combat Loki and
his alien army,
                           facing internal conflicts while saving the world
with their unique powers and teamwork.
                           Action-packed superhero film combines humor, and
epic battles in a thrilling cinematic experience.
                      </p>
                      <div class="genre">
                          <a href="#" class="category">Action</a>
                          <a href="#" class="category">Adventure</a>
                          <a href="#" class="category"><span>4K</span></a>
                      </div>
                      <a href=https://www.youtube.com/watch?v=v-PjgYDrg70"
style="color: white;">
                      <div class="watch">
                          <i class="fa-solid fa-play"></i>
                          <p>Watch the Trailer</p>
                      </div></a>
                  </div>
                  <img
src="https://wallpapersmug.com/download/1920x1200/2262c4/avengers-infinity-war-
superheroes-marvel-comics-2018.jpg">
```

```
                </div>
            </div>
            <div class="swiper-slide">
                <div class="image">
                    <div class="black">
                        <h1>Interstellar</h1>
                        <div class="star">
                            <i class="fa-solid fa-star"></i>
                            <i class="fa-solid fa-star"></i>
                            <i class="fa-solid fa-star"></i>
                            <i class="fa-solid fa-star-half-stroke"></i>
                            <i class="fa-regular fa-star"></i>
                        </div>
                        <p>
                            Interstellar chronicles the adventures of a group of
explorers who make use of a newly discovered wormhole to surpass the
                            limitations on human space travel and conquer the
vast distances involved in an interstellar voyage.
                        </p>
                        <div class="genre">
                            <a href="#" class="category">Adventure</a>
                            <a href="#" class="category">Drama</a>
                            <a href="#" class="category"><span>4K</span></a>
                        </div>
                        <a href=https://www.youtube.com/watch?v=zSWdZVtXT7E"
style="color: white;">
                        <div class="watch">
                            <i class="fa-solid fa-play"></i>
                            <p>Watch the Trailer</p>
                        </div></a>
                    </div>
                    <img
src="https://i.pinimg.com/originals/e9/24/0b/e9240b7c5b29b0fb4aa86304a03287e8.jpg
">
                </div>
            </div>
<div class="swiper-slide">
                <div class="image">
                    <div class="black">
                        <h1>YOUNG SHERLOCK HOLMES</h1>
                        <div class="star">
                            <i class="fa-solid fa-star"></i>
                            <i class="fa-solid fa-star"></i>
```

```
                        <i class="fa-solid fa-star"></i>
                        <i class="fa-solid fa-star-half-stroke"></i>
                        <i class="fa-regular fa-star"></i>
                    </div>
                    <p>
                    Young Sherlock Holmes and Watson embark on a thrilling
investigation, confronting supernatural forces and showcasing Holmes' budding
detective skills.
                    </p>
                    <div class="genre">
                        <a href="#" class="category">Animation</a>
                        <a href="#" class="category">Family</a>
                        <a href="#" class="category">Comedy</a>
                        <a href="#" class="category"><span>4K</span></a>
                    </div>
                    <a href=https://youtu.be/QWvMK5j-fXU" style="color:
white;">
                    <div class="watch">
                        <i class="fa-solid fa-play"></i>
                        <p>Watch the Trailer</p>
                    </div></a>
                </div>
                <img
src="https://i.pinimg.com/originals/3f/7a/bf/3f7abf449ea8be6ca7d2eff2b7021f88.jpg
">
            </div>
        </div>
<div class="swiper-slide">
            <div class="image">
                <div class="black">
                    <h1>The Omega Code</h1>
                    <div class="star">
                        <i class="fa-solid fa-star"></i>
                        <i class="fa-solid fa-star"></i>
                        <i class="fa-solid fa-star"></i>
                        <i class="fa-solid fa-star-half-stroke"></i>
                        <i class="fa-regular fa-star"></i>
                    </div>
                    <p>
                        In this spiritual thriller, an ancient prophecy is
about to be fulfilled as a secret code brings the world to the edge of
Apocalypse.
                    </p>
```

```
                        <div class="genre">
                            <a href="#" class="category">Action</a>
                            <a href="#" class="category">Horror</a>
                            <a href="#" class="category"><span>4K</span></a>
                        </div>
                       <a href=https://www.youtube.com/watch?v=QclB9U8Swwc"
style="color: white;">
                        <div class="watch">
                            <i class="fa-solid fa-play"></i>
                            <p>Watch the Trailer</p>
                        </div></a>
                    </div>
                    <img
src="https://i.ytimg.com/vi/tE7Jf3sIHBI/maxresdefault.jpg">
                </div>
              </div>
<div class="swiper-slide">
                <div class="image">
                    <div class="black">
                        <h1>Pirates of the Caribbean</h1>
                        <div class="star">
                            <i class="fa-solid fa-star"></i>
                            <i class="fa-solid fa-star"></i>
                            <i class="fa-solid fa-star"></i>
                            <i class="fa-solid fa-star-half-stroke"></i>
                            <i class="fa-regular fa-star"></i>
                        </div>
                        <p style="font-size: 13px">
                            The series follows the charismatic pirate Captain
Jack Sparrow as
                            he navigates treacherous seas, battles supernatural
forces, and seeks hidden treasures.
                        </p>
                        <div class="genre">
                            <a href="#" class="category">Action</a>
                            <a href="#" class="category">Adventure</a>
                            <a href="#" class="category"><span>4K</span></a>
                        </div>
                     <a href="https://www.youtube.com/watch?v=Hgeu5rhoxxY"
style="color: white;">
                        <div class="watch">
                            <i class="fa-solid fa-play"></i>
                            <p>Watch the Trailer</p>
```

```
                        </div></a>
                    </div>
                    <img src="https://wallpaperaccess.com/full/1103973.jpg">
                </div>
              </div>
            <div class="swiper-button-next"></div>
            <div class="swiper-button-prev"></div>
            <div class="swiper-pagination"></div>
          </div>
    </section>

<div class="second">
        <div class="latest">
            <h1>Popular Movies</h1>
            <div class="box">
                <div class="card">
                    <div class="details">
                        <div class="left">
                            <p class="name">The Exorcist</p>
                            <div class="date_quality">
                                <p class="quality">HD</p>
                                <p class="date">1973</p>
                            </div>
                            <p class="category">Horror/Suspense</p>
                            <div class="info">
                                <div class="rate">
                                    <i class="fa-solid fa-star"></i>
                                    <p>8.1</p>
                                </div>
                                <div class="time">
                                    <i class="fa-regular fa-clock"></i>
                                    <p>2h2min</p>
                                </div>
                            </div>
                        </div>
                        <div class="right">
                        <a href="https://www.youtube.com/watch?v=YDGw1MTEe9k">
                            <i class="fa-solid fa-play"></i></a>
                        </div>
                    </div>
                    <img src="https://m.media-
amazon.com/images/M/MV5BNWY0ZTEzNDItY2E2Yy00YjI5LWFhYTgtNDVlODIxNTUzMjg5XkEyXkFqc
GdeQXVyMTY3ODkyNDkz._V1_.jpg">
```

```
                </div>
<div class="card">
                    <div class="details">
                        <div class="left">
                            <p class="name">Big Hero 6</p>
                            <div class="date_quality">
                                <p class="quality">HD</p>
                                <p class="date">2014</p>
                            </div>
                            <p class="category">Animation/Comedy</p>
                            <div class="info">
                                <div class="rate">
                                    <i class="fa-solid fa-star"></i>
                                    <p>7.8</p>
                                </div>
                                <div class="time">
                                    <i class="fa-regular fa-clock"></i>
                                    <p>1h45min</p>
                                </div>
                            </div>
                        </div>
                        <div class="right">
                     <a href="https://www.youtube.com/watch?v=z3biFxZIJOQ">
                            <i class="fa-solid fa-play"></i></a>
                        </div>
                    </div>
                    <img
src="https://image.tmdb.org/t/p/original/rzEmN6l7yWDH9KttH4yXrDftL53.jpg">
                </div>
<div class="card">
                    <div class="details">
                        <div class="left">
                            <p class="name">Indiana Jones and the Kingdom of the
Crystal Skull</p>
<div class="date_quality">
                                <p class="quality">HD</p>
                                <p class="date">2008</p>
                            </div>
                            <p class="category">Action/War</p>
                            <div class="info">
                                <div class="rate">
                                    <i class="fa-solid fa-star"></i>
                                    <p>6.2</p>
```

```
                                </div>
                                <div class="time">
                                    <i class="fa-regular fa-clock"></i>
                                    <p>2h2min</p>
                                </div>
                            </div>
                        </div>
                        <div class="right">
                <a href="https://www.youtube.com/watch?v=kTJy1rFBtVw">
                            <i class="fa-solid fa-play"></i></a>
                        </div>
                    </div>
                    <img src="https://m.media-
amazon.com/images/M/MV5BZmY5ZTk3ZDMtZjA1MS00NzU4LTk5ZDItYmNhOTkxMGYxMjRlXkEyXkFqc
GdeQXVyMjM4MzQ4OTQ@._V1_FMjpg_UX1000_.jpg">
                </div>
 <div class="card">
                    <div class="details">
                        <div class="left">
                            <p class="name">Jaws</p>
                            <div class="date_quality">
                                <p class="quality">HD</p>
                                <p class="date">1975</p>
                            </div>
                            <p class="category">Documentary/Adventure</p>
                            <div class="info">
                                <div class="rate">
                                    <i class="fa-solid fa-star"></i>
                                    <p>8.1</p>
                                </div>
                                <div class="time">
                                    <i class="fa-regular fa-clock"></i>
                                    <p>2h4min</p>
                                </div>
                            </div>
                        </div>
                        <div class="right">
                <a href="https://www.youtube.com/watch?v=U1fu_sA7XhE">
                            <i class="fa-solid fa-play"></i></a>
                        </div>
                    </div>
```

```
                    <img src="https://m.media-
amazon.com/images/M/MV5BMmVmODY1MzEtYTMwZC00MzNhLWFkNDMtZjAwM2EwODUxZTA5XkEyXkFqc
GdeQXVyNTAyODkwOQ@@._V1_.jpg">
                </div>

<div class="card">
                    <div class="details">
                        <div class="left">
                            <p class="name">The Rock</p>
                            <div class="date_quality">
                                <p class="quality">HD</p>
                                <p class="date">1996</p>
                            </div>
                            <p class="category">Action/Thriller</p>
                            <div class="info">
                                <div class="rate">
                                    <i class="fa-solid fa-star"></i>
                                    <p>7.4</p>
                                </div>
                                <div class="time">
                                    <i class="fa-regular fa-clock"></i>
                                    <p>2h9min</p>
                                </div>
                            </div>
                        </div>
                        <div class="right">
              <a href="https://www.youtube.com/watch?v=a3qcNyjj9ZQ">
                            <i class="fa-solid fa-play"></i></a>
                        </div>
                    </div>
                    <img src="https://m.media-
amazon.com/images/M/MV5BZDJjOTE0N2EtMmRlZS00NzU0LWE0ZWQtM2Q3MWMxNjcwZjBhXkEyXkFqc
GdeQXVyNDk3NzU2MTQ@._V1_QL75_UX190_CR0,0,190,281_.jpg">
                </div>
            </div>
        </div>
    </div>

<div class="upcoming">
        <div class="movies_box">
            <h1>Upcoming Movies</h1>
            <div class="box">
                <div class="card">
```

```
                    <div class="details">
                        <div class="left">
                            <p class="name">Impossible Things</p>
                            <div class="date_quality">
                                <p class="quality">HD</p>
                                <p class="date">2021</p>
                            </div>
                            <p class="category">Drama/Comedy</p>
                            <div class="info">
                                <div class="time">
                                    <i class="fa-regular fa-clock"></i>
                                    <p>1h2min</p>
                                </div>
                            </div>
                        </div>
                        <div class="right">
                  <a href="https://www.dailymotion.com/video/x8hszca">
                            <i class="fa-solid fa-play"></i></a>
                        </div>
                    </div>
                    <img src="https://m.media-
amazon.com/images/M/MV5BZjE4YTE1NzYtMzFlZi00NDg0LWFlNzItMTM1MWIxNGIwMzFkXkEyXkFqc
GdeQXVyMjM0Mzk5NTU@._V1_.jpg">
                </div>
                <div class="card">
                    <div class="details">
                      <div class="left">
                            <p class="name">Top Gun: Maverick</p>
                            <div class="date_quality">
                                <p class="quality">HD</p>
                                <p class="date">2022</p>
                            </div>
                            <p class="category">Action/Drama</p>
                            <div class="info">
                                <div class="time">
                                    <i class="fa-regular fa-clock"></i>
                                    <p>2h11min</p>
                                </div>
                            </div>
                        </div>
                        <div class="right">
                <a href="https://www.youtube.com/watch?v=qSqVVswa420">
                            <i class="fa-solid fa-play"></i></a>
```

```
                    </div>
                </div>
                <img src="https://m.media-
amazon.com/images/M/MV5BZWYzOGEwNTgtNWU3NS00ZTQ0LWJkODUtMmVhMjIwMjA1ZmQwXkEyXkFqc
GdeQXVyMjkwOTAyMDU@._V1_FMjpg_UX1000_.jpg">
            </div>
            <div class="card">
                <div class="details">
                    <div class="left">
                        <p class="name">Finch</p>
                        <div class="date_quality">
                            <p class="quality">HD</p>
                            <p class="date">2021</p>
                        </div>
                        <p class="category">Science Fiction/Adventure</p>
                        <div class="info">
                            <div class="time">
                                <i class="fa-regular fa-clock"></i>
                                <p>1h55min</p>
                            </div>
                           </div>
                    </div>
                    <div class="right">
                <a href="https://www.youtube.com/watch?v=-0bYWnP3jH4">
                        <i class="fa-solid fa-play"></i></a>
                    </div>
                </div>
                <img src="https://m.media-
amazon.com/images/M/MV5BMmExZDc4NjEtZjY1ZS00OWU5LWExZGYtYTc4NDM1ZmRhMDZhXkEyXkFqc
GdeQXVyMTEyMjM2NDc2._V1_.jpg">
            </div>
            <div class="card">
                <div class="details">
                    <div class="left">
                        <p class="name">Back to the Outback</p>
                        <div class="date_quality">
                            <p class="quality">HD</p>
                            <p class="date">2021</p>
                        </div>
                        <p class="category">Animation/Comedy</p>
                        <div class="info">
                            <div class="time">
                                <i class="fa-regular fa-clock"></i>
```

```
                                <p>1h35min</p>
                            </div>
                        </div>
                    </div>
                    <div class="right">
                <a href="https://www.youtube.com/watch?v=dDNhtB7L8Lk">
                        <i class="fa-solid fa-play"></i></a>
                    </div>
                </div>
                <img src="https://m.media-
amazon.com/images/M/MV5BMmU0YzExYjctOGM0Yy00NzdkLTgxOGEtYjAwNzcxZGY4YTBiXkEyXkFqc
GdeQXVyMTEzMTI1Mjk3._V1_FMjpg_UX1000_.jpg">
            </div>
            <div class="card">
                <div class="details">
                    <div class="left">
                        <p class="name">The Sea Beast</p>
                        <div class="date_quality">
                            <p class="quality">HD</p>
                            <p class="date">2022</p>
                        </div>
                        <p class="category">Animation/Fantasy</p>
                        <div class="info">
                            <div class="time">
                                <i class="fa-regular fa-clock"></i>
                                <p>1h55min</p>
                            </div>
                        </div>
                    </div>
                    <div class="right">
            <a href="https://www.youtube.com/watch?v=P-E-IGQCsPo">
                        <i class="fa-solid fa-play"></i></a>
                    </div>
                </div>
                <img src="https://m.media-
amazon.com/images/M/MV5BODkyYTRlMDItMDlhMC00MzkzLWI3NzQtOGVlMjdjYThlNDM3XkEyXkFqc
GdeQXVyMTA1OTcyNDQ4._V1_FMjpg_UX1000_.jpg">
            </div>
        </div>
    </div>
</div>

<div class="show">
```

```
        <div class="tv_show">
            <h1>Aplauded TV Shows</h1>
            <div class="box">
                <div class="card">
                    <div class="details">
                        <div class="left">
                            <p class="name">Star Wars: Episode I - The Phantom
Menace</p>
                            <div class="date_quality">
                                <p class="quality">HD</p>
                                <p class="date">1999</p>
                            </div>
                            <p class="category">Action</p>
                            <div class="info">
                                <div class="rate">
                                    <i class="fa-solid fa-star"></i>
                                    <p>6.5</p>
                                </div>
                                <div class="time">
                                    <i class="fa-regular fa-clock"></i>
                                    <p>133min</p>
                                </div>
                            </div>
                        </div>
                        <div class="right">
                        <a href="https://www.youtube.com/watch?v=bD7bpG-zDJQ">
                            <i class="fa-solid fa-play"></i></a>
                        </div>
                    </div>
                    <img src="https://m.media-
amazon.com/images/M/MV5BYTRhNjcwNWQtMGJmMi00NmQyLWE2YzItODVmMTdjNWI0ZDA2XkEyXkFqc
GdeQXVyNTAyODkwOQ@@._V1_.jpg">
                </div>
                <div class="card">
                    <div class="details">
                        <div class="left">
                            <p class="name">GoldenEye</p>
                            <div class="date_quality">
                                <p class="quality">HD</p>
                                <p class="date">1995</p>
                            </div>
                            <p class="category">Spy</p>
                            <div class="info">
```

```
                                <div class="rate">
                                    <i class="fa-solid fa-star"></i>
                                    <p>7.2</p>
                                </div>
                                <div class="time">
                                    <i class="fa-regular fa-clock"></i>
                                    <p>2h10min</p>
                                </div>
                            </div>
                        </div>
                        <div class="right">
                           <a href="https://www.youtube.com/watch?v=lcOqUE0u1LM">
                            <i class="fa-solid fa-play"></i></a>
                        </div>
                    </div>
                    <img src="https://m.media-
amazon.com/images/M/MV5BOWI0MTJlMjEtYTI0ZS00NWZiLWE1ZmItYzBlZTE5YTk3NTJiXkEyXkFqc
GdeQXVyMTEwNDU1MzEy._V1_UY1200_CR85,0,630,1200_AL_.jpg">
                </div>
                <div class="card">
                    <div class="details">
                        <div class="left">
                            <p class="name">The Hunger Games: Catching Fire</p>
                            <div class="date_quality">
                                <p class="quality">HD</p>
                                <p class="date">2013</p>
                            </div>
                            <p class="category">Mystery</p>
                            <div class="info">
                                <div class="rate">
                                    <i class="fa-solid fa-star"></i>
                                    <p>7.5</p>
                                </div>
                                <div class="time">
                                    <i class="fa-regular fa-clock"></i>
                                    <p>2h26min</p>
                                </div>
                            </div>
                        </div>
                        <div class="right">
                        <a href="https://www.youtube.com/watch?v=EAzGXqJSDJ8">
                            <i class="fa-solid fa-play"></i></a>
                        </div>
```

```
                    </div>
                    <img src="https://m.media-
amazon.com/images/M/MV5BMTAyMjQ3OTAxMzNeQTJeQWpwZ15BbWU4MDU0NzA1MzAx._V1_FMjpg_UX
1000_.jpg">
                </div>
                <div class="card">
                    <div class="details">
                        <div class="left">
                            <p class="name">Jurassic Park</p>
                            <div class="date_quality">
                                <p class="quality">HD</p>
                                <p class="date">1993</p>
                            </div>
                            <p class="category">Animation</p>
                            <div class="info">
                                <div class="rate">
                                    <i class="fa-solid fa-star"></i>
                                    <p>8.2</p>
                                </div>
                                <div class="time">
                                    <i class="fa-regular fa-clock"></i>
                                    <p>2h7min</p>
                                </div>
                            </div>
                        </div>
                        <div class="right">
                    <a href="https://www.youtube.com/watch?v=_jKEqDKpJLw">
                            <i class="fa-solid fa-play"></i></a>
                    </div>
                    </div>
                    <img src="https://m.media-
amazon.com/images/M/MV5BMjQzODQyMzk2Nl5BMl5BanBnXkFtZTcwNTg4MjQ3OA@@._V1_.jpg">
                </div>
                <div class="card">
                    <div class="details">
                        <div class="left">
                            <p class="name">Charlie's Angels</p>
                            <div class="date_quality">
                                <p class="quality">HD</p>
                                <p class="date">2019</p>
                            </div>
                            <p class="category">Crime Fiction</p>
                           <div class="info">
```

```
                                <div class="rate">
                                    <i class="fa-solid fa-star"></i>
                                    <p>5</p>
                                </div>
                                <div class="time">
                                    <i class="fa-regular fa-clock"></i>
                                    <p>1h38min</p>
                                </div>
                            </div>
                        </div>
                        <div class="right">
                         <a href="https://www.youtube.com/watch?v=RSUq4VfWfjE">
                            <i class="fa-solid fa-play"></i></a>
                        </div>
                    </div>
                    <img src="https://m.media-
amazon.com/images/M/MV5BZTg5MTgzNDctOWVmNC00NzhkLWFiZmQtZWFkZTk4ZDU4N2Y2XkEyXkFqc
GdeQXVyMTAyOTE2ODg0._V1_.jpg">
                </div>
            </div>
        </div>
    </div>

<script src="https://cdn.jsdelivr.net/npm/swiper@9/swiper-
bundle.min.js"></script>
 <!-- Initialize Swiper -->
  <script>
    var swiper = new Swiper(".mySwiper", {
      slidesPerView: 1,
      spaceBetween: 30,
      loop: true,
      pagination: {
        el: ".swiper-pagination",
        clickable: true,
      },
      navigation: {
        nextEl: ".swiper-button-next",
        prevEl: ".swiper-button-prev",
      },
    });
  </script>
</body>
</html>
```

- **RECOMMEND.html**

```
<!DOCTYPE html>
<html>
<head>
    <title>NEW</title>

    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1, shrink-
to-fit=no">

    <!-- Google Fonts -->
    <link
href="https://fonts.googleapis.com/css?family=IBM+Plex+Sans&display=swap"
rel="stylesheet">
    <link
href="https://fonts.googleapis.com/css2?family=Noto+Sans+JP&display=swap"
rel="stylesheet">

    <!-- Font Awesome -->
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/4.7.0/css/font-awesome.min.css">

    <!-- Bootstrap -->
    <link rel="stylesheet"
href="https://maxcdn.bootstrapcdn.com/bootstrap/4.0.0/css/bootstrap.min.css"
integrity="sha384-
Gn5384xqQ1aoWXA+058RXPxPg6fy4IWvTNh0E263XmFcJlSAwiGgFAW/dAiS6JXm"
crossorigin="anonymous">

    <link rel= "stylesheet" type= "text/css" href= "{{
url_for('static',filename='style.css') }}">

</head>

<body id="content">
    <div class="results">
        <center>
          <h2 id="name" class="text-uppercase">{{title}}</h2>
        </center>
    </div>
```

```
    <br>

<div id="mycontent">
    <div id="mcontent">
      <div class="poster-lg">
        <img class="poster" style="border-radius: 40px;margin-left: 90px;"
height="400" width="250" src={{poster}}>
      </div>
      <div class="poster-sm text-center">
        <img class="poster" style="border-radius: 40px;margin-bottom: 5%;"
height="400" width="250" src={{poster}}>
      </div>
      <div id="details">
        <br/>
        <h6 id="title" style="color:white;">TITLE:  {{title}}</h6>
        <h6 id="overview" style="color:white;max-width: 85%">OVERVIEW:
<br/><br/>       {{overview}}</h6>
        <h6 id="vote_average" style="color:white;">RATING:
 {{vote_average}}/10 ({{vote_count}} votes)</h6>
        <h6 id="genres" style="color:white;">GENRE:  {{genres}}</h6>
        <h6 id="date" style="color:white;">RELEASE DATE:
 {{release_date}}</h6>
        <h6 id="runtime" style="color:white;">RUNTIME:  {{runtime}}</h6>
        <h6 id="status" style="color:white;">STATUS:  {{status}}</h6>
      </div>
    </div>
</div>
<br/>

{% for name, details in cast_details.items() if not cast_details.hidden %}
  <div class="modal fade" id="{{details[0]}}" tabindex="-1" role="dialog" aria-
labelledby="exampleModalLabel3" aria-hidden="true">
    <div class="modal-dialog modal-lg" role="document">
      <div class="modal-content">
        <div class="modal-header" style="background-color: #e50914;color:
white;">
          <h5 class="modal-title" id="exampleModalLabel3">{{name}}</h5>
          <button type="button" class="close" data-dismiss="modal" aria-
label="Close">
            <span aria-hidden="true" style="color: white">&times;</span>
          </button>
        </div>
```

```
        <div class="modal-body">
          <img class="profile-pic" src="{{details[1]}}" alt="{{name}} - profile"
style="width: 250px;height:400px;border-radius: 10px;" />
          <div style="margin-left: 20px">
            <p><strong>Birthday:</strong> {{details[2]}} </p>
            <p><strong>Place of Birth:</strong> {{details[3]}} </p>
            <p>
              <p><strong>Biography:</strong><p>
              {{details[4]}}
            </p>
          </div>
        </div>
        <div class="modal-footer">
          <button type="button" class="btn btn-secondary" data-
dismiss="modal">Close</button>
        </div>
      </div>
    </div>
  </div>
  {% endfor %}

  {% if casts|length > 1 %}
  <div class="movie" style="color: #E8E8E8;">
        <center>
        <h3>TOP CAST</h3>
        <h5>(Click on the cast to know more)</h5>
    </center>
  </div>

  <div class="movie-content">
        {% for name, details in casts.items() if not casts.hidden %}
          <div class="castcard card" style="width: 15rem;" title="Click to know
more about {{name}}" data-toggle="modal" data-target="#{{details[0]}}">
            <div class="imghvr">
              <img class="card-img-top cast-img" id="{{details[0]}}" height="360"
width="240" alt="{{name}} - profile" src="{{details[2]}}">
              <figcaption class="fig">
                <button class="card-btn btn btn-danger"> Know More </button>
              </figcaption>
            </div>
            <div class="card-body">
              <h5 class="card-title">{{name|title}}</h5>
```

```
              <h5 class="card-title"><span style="color:#756969;font-size:
20px;">Character: {{details[1]|title}}</span></h5>
            </div>
          </div>
      {% endfor %}
    </div>
{% endif %}

  <center>
    {% if reviews %}
    <h2 style="color:white">USER REVIEWS</h2>
     <div class="col-md-12" style="margin: 0 auto; margin-top:25px;">
        <table class="table table-bordered" bordercolor="white"
style="color:white">
            <thead>
                <tr>
                    <th class="text-center" scope="col" style="width:
75%">Comments</th>
                    <th class="text-center" scope="col">Sentiments</th>
                </tr>
            </thead>

            <tbody>
              {% for review, status in reviews.items() if not reviews.hidden %}
              <tr style="background-color:#161616; opacity: 0.8">
                  <td>{{review}}</td>
                  <td><center>
                      {{status}} :
                      {% if status =='Good' %}
                        &#128515;
                      {% else %}
                        &#128534;
                      {% endif %}
                    </center>
                  </td>
                </tr>
                {% endfor %}
            </tbody>
        </table>
    </div>
    {% else %}
    <div style="color:white;">
      <h1 style="color:white"> This movie is not released yet. Stay tuned! </h1>
```

```
    </div>
    {% endif %}
  </center>

      {% if movie_cards|length > 1 %}


      <div class="movie" style="color: #E8E8E8;">
      <center><h3>RECOMMENDED MOVIES FOR YOU</h3><h5>(Click any of the movies to
get recommendation)</h5></center>
    </div>

    <div class="movie-content">
      {% for poster, title in movie_cards.items() if not movie_cards.hidden %}
        <div class="card" style="width: 15rem;" title="{{title}}"
onclick="recommendcard(this)">
          <div class="imghvr">
            <img class="card-img-top" height="360" width="240" alt="{{title}} -
poster" src={{poster}}>
            <figcaption class="fig">
              <button class="card-btn btn btn-danger"> Click Me </button>
            </figcaption>
           </div>
          <div class="card-body">
            <h5 class="card-title">{{title|capitalize}}</h5>
          </div>
        </div>
    {% endfor %}
    </div>

    {% endif %}


    </div>
  </div>

  <script
src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js"></script>
  <script type="text/javascript" src="{{url_for('static',
filename='recommend.js')}}"></script>
  <script
src="https://cdnjs.cloudflare.com/ajax/libs/popper.js/1.12.9/umd/popper.min.js"
integrity="sha384-
```

```
ApNbgh9B+Y1QKtv3Rn7W3mgPxhU9K/ScQsAP7hUibX39j7fakFPskvXusvfa0b4Q"
crossorigin="anonymous"></script>
  <script
src="https://maxcdn.bootstrapcdn.com/bootstrap/4.0.0/js/bootstrap.min.js"
integrity="sha384-
JZR6Spejh4U02d8jOt6vLEHfe/JQGiRRSQQxSfFWpi1MquVdAyjUar5+76PVCmYl"
crossorigin="anonymous"></script>
</body>
</html>
```

- **PARENTAL GUIDANCE**

```
<!DOCTYPE html>
<html lang="en" dir="ltr">
<head>
<title>Movie Recommendation System and Review Engine</title>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-
fit=no">
<!-- Google Fonts -->
  <link href='https://fonts.googleapis.com/css?family=Poppins' rel='stylesheet'>

  <!-- Font Awesome -->
  <link rel="stylesheet"
href="https://pro.fontawesome.com/releases/v5.10.0/css/all.css"
integrity="sha384-
AYmEC3Yw5cVb3ZcuHtOA93w35dYTsvhLPVnYs9eStHfGJvOvKxVfELGroGkvsg+p"
crossorigin="anonymous" />
<!-- Owl Carousel css-->
  <link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/OwlCarousel2/2.3.4/assets/owl.carous
el.css" />
  <link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/OwlCarousel2/2.3.4/assets/owl.carous
el.min.css" />
  <!-- Owl Carousel css-->


  <!-- jquery css-->
  <script src="https://code.jquery.com/jquery-1.12.4.min.js" integrity="sha256-
ZosEbRLbNQzLpnKIkEdrPv7lOy9C27hHQ+Xp8a4MxAQ=" crossorigin="anonymous"></script>
  <!-- jquery css-->
```

```
<style>
@import
url('https://fonts.googleapis.com/css2?family=Poppins:wght@200;300;400;500;600;70
0;800;900&display=swap');
*{
    padding: 0;
    margin: 0;
    box-sizing: border-box;
}
body{
    width: 100%;
    height: 100%;
    color: white;
    background: #161616;
    font-family: Poppins;
}
.nav-links{
flex: 1;
text-align: center;
}
.nav-links ul li{
list-style: none;
display: inline-block;
padding: 35px 15px;
position: relative;
margin-bottom: 105px;
}
.nav-links ul li a{
color: #fff;
text-decoration: none;
font-size: 15.5px;
font-family: Poppins;
}
.nav-links ul li::after{
content: '';
width: 0%;
height: 2px;
background: #FBA11D;
display: block;
margin: auto;
transition: 0.5s;
}
.nav-links ul li:hover::after{
```

```
width: 100%;
}

/*---------global------*/
.container {
  max-width: 95%;
  margin: auto;
}

a {
  text-decoration: none;
  transition: 0.5s;
}

img {
  width: 100%;
  height: 100%;
}

ul {
  list-style: none;
}

.flex {
  display: flex;
}

.flex1 {
  display: flex;
  justify-content: space-between;
}

i {
  cursor: pointer;
  transition: 0.5s;
}

/*---------global------*/
/*---------header------*/
.home {
  color: white;
  background-image: url("https://wallpapers.com/images/hd/iron-man-movie-
kqwjqnganqze6pjs.jpg");
```

```
  background-size: cover;
  height: 100vh;
  width: 100%;
}

.logo img {
  width: 118px;
  height: 100%;
  margin-bottom: 110px;
  margin-left: 70px;
}

.headerbg {
  height: 100vh;
  width: 100%;
  background: linear-gradient(to right, rgba(34, 31, 31, 1) 0%, rgba(34, 31, 31,
0.4) 100%)
}

header {
  height: 12vh;
  background: rgba(0, 0, 0, 0.2);
}

header .navbar {
  align-items: center;
  transition: 0.5s;
}

.subscribe i:nth-child(1) {
  margin-top: 17px;
}

.subscribe i {
  font-size: 20px;
  margin-right: 20px;
}

#palybtn {
  background: #FBA11D;
  width: 50px;
  height: 50px;
  color: #fff;
```

```
  line-height: 50px;
  text-align: center;
  border-radius: 50%;
  margin-right: 10px;
  margin-bottom: 110px;
}

.button4 {
  background: #FBA11D;
  border: none;
  outline: none;
  height: 53px;
  color: white;
  padding: 15px 30px;
  cursor: pointer;
  font-family: Poppins;
}

.subscribe button {
  margin-left: 15px;
}

header.sticky .headerbg {
  position: relative;
}

header.sticky {
  z-index: 10;
  position: fixed;
  top: 0;
  width: 100%;
  background: #000;
  transition: 0.5s;
  height: 10vh;
  box-shadow: 2px 2px 2px 1px rgba(0, 0, 0, 0.1);
  transition: 0.5s;
}

header.sticky .subscribe {
  display: none;
}

@media only screen and (max-width:768px) {
```

```
  header nav ul {
    position: absolute;
    top: 100px;
    left: 0;
    width: 100%;
    background: #000;
    overflow: hidden;
    transition: max-height 0.5s;
    z-index: 100;
  }

  header nav ul li {
    display: block;
    margin: 50px;
    margin-top: 10px;
    margin-bottom: 10px;
  }

  header .navbar span {
    color: white;
    display: block;
    cursor: pointer;
  }

  .home {
    background-repeat: no-repeat;
    background-size: cover;
    background-position: center;
    height: 70vh;
  }

  .subscribe {
    display: none;
  }
}

/*---------home_content------*/
.left, .right {
  width: 50%;
}

.top {
  margin-top: 10%
```

```
}

.mtop {
  margin-top: 5%
}

h1 {
  font-size: 80px;
  letter-spacing: 2px;
  line-height: 90px;
  text-transform: uppercase;
}

.time {
  margin-top: 20px;
  margin-bottom: 20px;
  line-height: 40px;
}

.time label {
  width: 40px;
  height: 40px;
  border: 1px solid white;
  text-align: center;
  line-height: 40px;
}

.time i {
  font-size: 5px;
  line-height: 40px;
  margin-left: 20px;
  margin-right: 20px;
}

.home_content p {
  font-size: 18px;
  line-height: 28px;
}

.btn {
  padding: 20px 40px;
  margin-right: 20px;
  background: #FF8E11;
```

```
  width: 25%;
  height: 10vh;
  color: white;
  border: none;
}

.home_content #palybtn {
  width: 60px;
  height: 60px;
  line-height: 60px;
  background: white;
  color: #FF8E11;
}

.button {
  margin-top: 30px;
}

.btn2 {
padding: 10px 20px;
      background-color: #FF8E11;
      color: white;
      border: none;
      cursor: pointer;
      font-size: 16px;

}

.button p {
  margin-top: 12px;
  margin-left: 20px;
  font-size: 18px;
}
.last-btn {
      display: flex;
      justify-content: left;
      align-items: left;
      margin-top: 2rem;
    }

.button123 {
      padding: 10px 20px;
      background-color: #E50916;
```

```
      color: white;
      border: none;
      cursor: pointer;
      font-size: 16px;
    }

    .button123:hover {
      background-color: darkred;
    }
.about-us{
  height: 100vh;
  width: 100%;
  padding: 90px 0;
  background: #161616;
}
.pic{
  height: auto;
  width:  350px;
}
.about{
  width: 1130px;
  max-width: 85%;
  margin: 0 auto;
  display: flex;
  align-items: center;
  justify-content: space-around;
}
.text{
  width: 650px;
  text-align: justify;
}
.text h2{
  font-size: 90px;
  font-weight: 600;
  margin-bottom: 10px;

}
.text h5{
  font-size: 22px;
  font-weight: 500;
  margin-bottom: 20px;
}
span{
```

```
  color: red;
}
.text p{
  font-size: 16.5px;
  line-height: 25px;
  letter-spacing: 1px;
}
</style>
</head>
<body>
<section class="home">
    <div class="headerbg ">
      <header>
        <div class="container ">
          <div class="navbar flex1">
            <div class="logo">
              <img src="https://seeklogo.com/images/M/movie-time-cinema-logo-
8B5BE91828-seeklogo.com.png" alt="">
            </div>

            <div class="nav-links">
<ul>
            <li><a href="index.html" target="_self">HOME</a></li>
            <li><a href="about.html">ABOUT</a></li>
            <li><a href="trending.html" target="_self">TRENDING MOVIES</a></li>
            <li><a href="contact.html" target="_self">CONTACT US</a></li>
            <li><a href="movie.html" target="_self">REVIEW PREDICTION</a></li>
            <li><a href="guide.html" target="_self">PARENTAL GUIDE</a></li>
</ul>
</div>

            <div class="subscribe flex">
              <a href ="http://127.0.0.1:5000/"><i class="fas fa-search"
style="color: white; margin-top: 1rem;"></i></a>
              <a href ="login.html"><i id="palybtn" class="fas fa-user"
style="color: white; margin-top: 0.1rem;"></i></a>
              <a href="movie.html" class="button4">SUBSCRIBE</a>
            </div>
          </div>
        </div>
      </header>
      <div class="home_content mtop">
        <div class="container">
```

```
          <div class="left">
            <h1>PARENTAL GUIDANCE</h1><br>
            <h2>Nurturing Family-Friendly Movie Experiences</h2>

            <div class="time flex">
              <label>R</label>
              <i class="fas fa-circle"></i>
              <p>1hrs 50mins</p>
              <i class="fas fa-circle"></i>
              <p>2021</p>
              <i class="fas fa-circle"></i>
              <button class="btn2">Action</button>
            </div>
            <p>
Welcome to our parental guidance section, where we prioritize the well-being and
enjoyment of your family's movie journey.
We understand the importance of making informed decisions about the content your
loved ones engage with.
            </p>
            <div class="button flex">
              <button class="btn">PLAY NOW</button>
              <i id="palybtn" class="fas fa-play"></i>
              <p>WATCH TRAILER</p>
            </div>
          </div>
        </div>
      </div>
    </div>
  </section>
<section class="about-us">
    <div class="about">
      <img src="https://www.wallpapersun.com/wp-content/uploads/2020/09/Parental-
Advisory-Wallpaper-9.jpg" class="pic">
      <div class="text">
        <h2>Flick Counsel</h2>
        <h5>Enlightened Guidance for <span>Film </span>Parenting</h5>
          <p>Our comprehensive content evaluations go beyond surface-level
ratings, delving into themes, language, violence, and more, to ensure you have a
clear understanding of what each film offers. We also provide detailed
descriptions of potentially sensitive content, allowing you to make decisions
that align with your family values.
    <br><br>
```

```
    Discover our age-specific recommendations, designed to cater to different
developmental stages and interests. From delightful animations for little ones,
to inspiring coming-of-age stories for teenagers, our curated suggestions offer a
diverse range of movies that entertain, educate, and inspire.
    <br><br>
    Join us in creating a safe and enriching movie environment for your loved
ones. Explore our parental guidance resources and embark on a cinematic journey
that's tailored to your family's values and aspirations.</p>
  <div class="last-btn">
    <a href="https://help.imdb.com/article/contribution/titles/parental-
guide/GF4KYKYJA4PKQB32#" class="button123">Know More About It</a>
  </div>
<script>
    window.addEventListener("scroll", function() {
      var header = document.querySelector("header");
      header.classList.toggle("sticky", window.scrollY > 50);
    })
  </script>

</body>
</html>
```

GLOSSARY

1. **Big Data:** Refers to large and complex datasets that cannot be easily managed or processed through traditional data processing techniques.
2. **IMDb:** Stands for Internet Movie Database, a popular online database that provides information about movies, TV shows, actors, and other related information.
3. **Content-based recommendation system:** A recommendation system that suggests items (in this case, movies) to users based on the similarity of their content features, such as genre, actors, directors, and plot.
4. **Cosine similarity:** A mathematical technique used to measure the similarity between two vectors by calculating the cosine of the angle between them.
5. **Sentiment analysis:** The process of determining the sentiment or opinion expressed in a piece of text, such as a movie review, using natural language processing techniques.
6. **Logistic regression:** A statistical model used to predict binary outcomes, often used in sentiment analysis to classify text as positive or negative.
7. **User experience (UX):** Refers to the overall experience of a user when interacting with a system or product, including ease of use, satisfaction, and effectiveness.
8. **Dataset:** A collection of structured or unstructured data that is used for analysis and research purposes.
9. **Acronym:** A word formed from the initial letters of a series of words, pronounced as a separate word, such as IMDb (Internet Movie Database).
10. **Abbreviation:** A shortened form of a word or phrase, such as UX for user experience.